

Performance benchmarking of open-source service meshes using load testing

Master Thesis

Submitted in partial fulfillment of the requirements for the degree of

Master of Science in Engineering

to the University of Applied Sciences FH Campus Wien
Master Degree Program: Software Design and Engineering

Author:

Ing. Michael Schneider, BSc

Student identification number:

2110838013

Supervisor:

FH-Prof. FH-Hon.Prof. Priv.-Doz. Mag. DI. DI. Dr. Karl Göschka

Date:

2023-05-13

Declaration of authorship:

I declare that this Master Thesis has been written by myself. I have not used any other than the listed sources, nor have I received any unauthorized help.

I hereby certify that I have not submitted this Master Thesis in any form (to a reviewer for assessment) either in Austria or abroad.

Furthermore, I assure that the (printed and electronic) copies I have submitted are identical.

Date: 2023-05-13

Signature:

Danksagung

An dieser Stelle möchte ich vor allem meiner Familie und meinen Freunden für ihre Geduld und Liebe während meiner gesamten Studienzeit danken.

Besonderen Dank gilt meinem Kommilitonen und persönlichen Freund Nichil Strasser, der mir durch interessante Gespräche, wertvolles Feedback gegeben hat.

Ich möchte mich auch bei meinem Betreuer, Herrn Karl Michael Göschka, bedanken, der immer geduldig auf meine Fragen antwortete und meine Anliegen verstand.

Vielen Dank an wichtige Personen aus meinem Studiengang, darunter Katharina Bühn, Daniela Duh, Karim Schierbauer und David Schedler.

Abstract

Through Docker and consequently Kubernetes, a trend from monolithic to more distributed applications, so-called microservice architecture, has established itself in the industry. Microservices aim to break down a complex monolith into small, independently deployable, and maintainable services. In distributed systems such as microservices, new problems arise such as fault tolerance, resilience, and security. Depending on the programming language, different libraries were developed for resilience. However, each library has a different implementation that makes different assumptions, which could lead to partial implementation for some languages and inconsistency in the overall implementation. This inconsistency makes it challenging to reason about failures, possibly contributing to obscuring or propagating them. To address these concerns, Service Mesh has emerged as a promising approach. Service Mesh introduces a dedicated infrastructure layer over microservices without modifying the service implementations.

Since 2014, a large number of service meshes have appeared. At first glance, they all promise the same features, such as improved reliability, observability, and security. Due to the similar features, it is difficult to find the right service mesh in practice. Aiming this problem, this thesis compares the four most popular open-source service meshes under load. In addition to the performance benchmarking evaluation, Istio, Linkerd, Kuma, and Traefik Mesh are also subjected to a quantitative literature study using a questionnaire.

The quantitative analysis of the performance benchmarking shows that Traefik Mesh is the clear winner in terms of response times and throughput, but scores lower than the others in terms of features such as security and observability. Traefik Mesh and Kuma are ahead in response times. Istio plays to its strengths with extensive configuration options and consumes little CPU and memory in the application data plane. In addition, Istio meets most of the quantitative literature study's security, observability and traffic control criteria. Linkerd convinces with minimal memory consumption in the Sidecar proxy and its simplicity in configuration. Kuma achieves very good response times and, unlike the other service meshes, can also be used in non-Kubernetes environments. Based on the evaluation, a decision tree can be derived. This decision tree helps to select a service mesh for a microservice architecture. It has also been demonstrated that the Service Mesh Performance specification can be implemented in a load test tool as in the proof of concept. This provides an alternative to Meshery, which implements much more than the Service Mesh Performance specification and is not essential for load testing. It has also been shown that it is possible to improve the performance of a service mesh using a combination of different components such as the Ingress - The HAProxy Ingress improves performance in load testing, but in terms of resilience, the Google Kubernetes Engine Ingress is better.

Kurzfassung

Durch Docker und in der Folge Kubernetes hat sich in der Branche ein Trend von monolithischen zu verteilten Anwendungen, der sogenannten Microservice Architektur, etabliert. Microservices zielen darauf ab, einen komplexen Monolithen in kleine, unabhängig einsetzbare und wartbare Services zu zerlegen. In verteilten Systemen mit Microservices ergeben sich neue Probleme wie Fehlertoleranz, Ausfallsicherheit und Security. Je nach Programmiersprache wurden verschiedene Bibliotheken für diese Probleme entwickelt. Jede Bibliothek hat jedoch eine andere Implementierung, die von unterschiedlichen Annahmen ausgeht, was bei einigen Sprachen zu einer Teilimplementierung und bei der Gesamtimplementierung zu Inkonsistenzen führen kann. Diese Inkonsistenz macht es schwierig, Schlussfolgerungen zu Fehlern zu ziehen und trägt möglicherweise dazu bei, diese zu verschleiern oder zu verbreiten. Zur Lösung dieser Probleme hat sich der Service Mesh Ansatz als vielversprechend erwiesen. Ein Service Mesh führt eine dedizierte Infrastrukturebene über Microservices ein, ohne die Implementierung der Services zu verändern.

Seit 2014 ist eine große Anzahl von Service Meshes erschienen. Auf den ersten Blick versprechen sie alle die gleichen Eigenschaften, wie z.B.: eine verbesserte Reliability, Observability und Security. Aufgrund der ähnlichen Eigenschaften ist es in der Praxis schwierig, das richtige Service Mesh zu finden. Um dieses Problem zu lösen, vergleicht diese Arbeit die vier populärsten open-source Service Meshes unter Last. Neben dem Leistungsvergleich werden Istio, Linkerd, Kuma und Traefik Mesh auch einer quantitativen Literaturstudie mit Hilfe eines Fragebogens unterzogen.

Die quantitative Analyse des Leistungsvergleiches zeigt, dass Traefik Mesh der klare Sieger in Bezug auf Antwortzeiten und Durchsatz ist, aber in Bezug auf Eigenschaften wie Security, Traffic Control und Observability weniger gut abschneidet als die anderen. Traefik Mesh und Kuma liegen bei den Antwortzeiten voran. Istio spielt seine Stärken mit umfangreichen Konfigurationsoptionen aus und verbraucht wenig CPU und Speicher in der Data plane. Darüber hinaus erfüllt Istio die meisten der in der quantitativen Literaturstudie genannten Kriterien für Security, Observability und Traffic control. Linkerd überzeugt durch minimalen Speicherverbrauch im Sidecar-Proxy und seine Einfachheit in der Konfiguration. Kuma erreicht sehr gute Antwortzeiten und kann im Gegensatz zu den anderen Service Meshes auch in nicht-Kubernetes Umgebungen eingesetzt werden. Aus der Auswertung lässt sich ein Entscheidungsbaum ableiten. Dieser Entscheidungsbaum hilft bei der Auswahl eines Service Meshes für eine Microservice Architektur.

Es wurde im Proof of Concept auch gezeigt, dass die Service Mesh Performance Spezifikation in ein Lasttest-Tool implementiert werden kann. Dies stellt eine Alternative zu Meshery dar, das viel mehr als die Service Mesh Performance Spezifikation implementiert und für Lasttests nicht unbedingt erforderlich ist. Außerdem wurde festgestellt, dass es möglich ist, die Leistung eines Service Meshes durch eine Kombination einer anderen Komponenten wie dem Ingress zu verbessern - Der HAProxy Ingress verbessert die Leistung bei Lasttests, aber in Bezug auf die Ausfallsicherheit ist der Google Kubernetes Engine Ingress besser geeignet.

List of Abbreviations

API	Application Programming Interface
CDR	Custom Definition Resource
CLI	Command Line Tool
CNCF	Cloud Native Computer Foundation
CPU	Central Processing Unit
CRI	Container Runtime Interface
CSV	Comma-separated values
DNS	Domain Name Service
DSL	Domain Specific Language
eBPF	extended Berkeley Packet Filter
GCP	Google Cloud Platform
GKE	Google Kubernetes Engine
HCL	HashiCorp Configuration Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IaC	Infrastructure as Code
IP	Internet Port
ISO	International Organization for Standardization
MSA	Microservice Architecture
mTLS	mutual Transport Layer Security
NIST	National Institute of Standards and Technology
PoC	Proof of Concept
RAM	Random Access Memory
RPC	Remote Procedure Call
RPI	Remote Procedure Invocation
SDN	Software-defined networking
SMP	Service Mesh Performance
SM-DR	Service Mesh - Deployment Recommendation
SSL	Secure Sockets Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TPS	Transactions per seconds
YAML	Yet Another Markup Language

Key Terms

Benchmarking
Decision tree
Istio
Kubernetes
Kuma
Linkerd
Load testing
Open-source
Microservice Architecture
Performance
Service Mesh
Traefik Mesh

Contents

1	Introduction	1
1.1	From Microservices to Service Meshes	1
1.2	Aims and Objectives of Thesis	2
1.3	Methodology	3
1.4	Organization of Thesis	3
2	Background Information	4
2.1	Terminology	4
2.2	Kubernetes	8
2.3	Service Meshes	12
3	Performance Benchmarking and Proof of Concept	20
3.1	Selection of Service Meshes	20
3.2	Comparison of Istio, Linkerd, Kuma and Traefik Mesh architecture	21
3.3	Test environment	23
3.4	Configuration of Service Meshes	25
3.5	Deployment	29
3.6	Tools and Languages	33
3.7	Collection of aggregated results	35
3.8	Proof of Concept for Service Mesh Performance Output Extension	36
3.9	Lessons Learned	39
4	Discussion	42
4.1	Quantitative Literature Study	42
4.2	Quantitative Analysis of Performance Benchmarking	47
4.3	Summary and Decision tree	54
5	Final Considerations	58
5.1	Summary	58
5.2	Evaluation of the Objectives	58
5.3	Limitations and Reflection of Thesis	60
5.4	Related Work	60
5.5	Future Work	61
5.6	Conclusion	62
	Bibliography	68
	List of Figures	68
	List of Tables	70
	List of Listenings	71
	Appendix	72

1 Introduction

In 2013, the Docker container platform was released. This technology was a groundbreaking innovation, and it gave microservice architecture (MSA) a new impetus. These architectures can be run in production with orchestrators like Kubernetes and Apache Mesos. However, this rapid evolution led to extremely distributed systems. To achieve high availability and reliability with these MSA, resilience frameworks/libraries were developed to provide various mechanisms such as service discovery and fault tolerance. However, the approach led to drawbacks such as increased code complexity, low reusability of microservice artifacts, and limited choice of programming languages. To address the previous issues, an infrastructure-related code was offloaded from microservices to a separate layer. This approach is called service mesh.[1]

1.1 From Microservices to Service Meshes

Over the past few decades, the cloud computing paradigm has undergone significant changes, leading to shifts in the software service delivery domain. Modern applications are now designed with cloud-native architectures for orchestrated environments, in contrast to on-premises design. This shift has contributed to the emergence of DevOps, which combines development and operations. DevOps has become mainstream in the enterprise due to recent technology developments in microservices that standardize software packaging and deployment, leading to faster and more agile software service delivery. Microservices aim to break down a complex monolith into small, independently deployable, and maintainable services. Proponents of microservices believe that this not only solves the problems associated with integrating enterprise applications, but also simplifies the plumbing required to build a service-oriented architecture. Microservices are now widely adopted in different areas, including big data processing, Internet of things applications, and high-performance computing, due to their flexibility, simplicity, and scalability. Significant efficiency and productivity improvements achieved during the development and deployment phase, but there has been no reduction in operational complexity during the service runtime. A cloud-native application may comprise a large number of microservices, each implemented using different languages and belonging to different tenants, with thousands of constantly changing service instances caused by the orchestration platform scheduling. Debugging microservices applications in such dynamic environments is challenging due to the complexity of service dependencies and the temporary inaccessibility of any service to its consumers. Traffic control becomes necessary, but is currently missing for runtime operation. To address these concerns, Service Mesh has emerged as a promising approach. Service Mesh introduces a dedicated infrastructure layer over microservices without modifying the service implementations. It serves as a fully manageable service-to-service communication platform that standardizes the runtime operations of applications. Service Mesh technologies within the microservices ecosystem hold the promise of addressing communication-related concerns such as interoperability, traffic segmentation, dependency control, and runtime enforcement.[2]

1.2 Aims and Objectives of Thesis

Currently, there are a variety of implementations, such as Istio, Consul, Linkerd, Kuma, Open Service Mesh, Traefik Mesh, and many more, which have already implemented the Service Mesh architecture. All of them provide similar, but to a different degree, non-functional requirements such as reliability, observability, and security without editing the program code. Although some functionalities are already known, how service meshes complement MSA in a useful way there are still opportunities for future research. In [3], the following five possibilities are mentioned:

1. Unified architecture
2. Performance
3. Data analytics
4. Adaptation of service meshes in edge environments
5. extended Berkeley Packet Filter (eBPF) trend

The focus of this thesis is on the benchmarking of service meshes. One goal of the paper is to answer a Future Work finding from an existing paper:

There are a number of studies that have tried to improve the performance of service mesh solutions, but the performance analyses of existing solutions are unknown.[3]

From this, the following goal can be derived:

At least Istio, Linkerd, Kuma and Traefik Mesh with the same MSA in the Google Cloud Platform should be compared for their performance based on previously defined test cases.

At the same time, by comparing the performance of the four service meshes under consideration, an attempt is made to provide an answer to the second part of the paper:

Additionally, there is no guideline or best practices to optimize the tuning of service mesh features.[3]

Optimally, benchmarking will yield new insights into which set screws in a service mesh requires modification to achieve better results in the evaluation. Based on these two objectives, this thesis main questions are:

1. How do Istio, Linkerd, Kuma and Traefik Mesh differ in architecture?
2. Is the Service Mesh Performance (SMP) specification also applicable to other load testing tools like k6?
3. How do the service meshes differ in the areas of traffic control, observability and security?
4. How do Istio, Linkerd, Kuma and Traefik Mesh differ in performance benchmarking?
 - How do Istio, Linkerd, Kuma and Traefik Mesh differ in load testing?
 - How do Istio, Linkerd, Kuma and Traefik Mesh differ in stress testing?
 - How resource intensive (Central Processing Unit(CPU) and Random Access Memory(RAM)) are Istio, Linkerd, Kuma and Traefik Mesh in comparison?
5. Is it possible to improve the performance of the service mesh?

1.3 Methodology

To answer the first research question of chapter 1.2, a literature study was conducted for Istio, Linkerd, Kuma, and Traefik Mesh. For the second research question, a proof of concept is created using the popular k6 load test tool. The aim is to implement the vendor-independent SMP specification and offer it as an extension for the tool. A quantitative literature study will answer the third research question. For this purpose, a questionnaire of twelve questions is derived from National Institute of Standards and Technology (NIST) Special Publication - 800-204A, covering the areas of traffic control, observability and security. The fourth research question and its sub-questions are answered through the quantitative analysis of performance benchmarking. This is done by using Google Kubernetes Engine (GKE) to deploy a selected MSA application on Google Cloud Platform (GCP). After deployment, each of the selected service meshes is tested using four self-created test cases. The test cases consist of load and stress tests. To answer the last question, a different configuration is used in the service meshes and the same tests are performed as for the fourth research question.

1.4 Organization of Thesis

Chapter 2 defines concepts that are referenced throughout the paper. First, you need to understand the foundation, such as Kubernetes, MSA and other terminology, in order to understand the concept of service meshes.

The chapter 3 describes the steps that are necessary to set up one of the service meshes in practice. In addition, the different types of high-level service mesh architecture are also explained. Furthermore, the different test cases as well as the test scenario are described. The proof of concept is also presented. First, the problem is explained, which is solved by the SMP output extension. Then, the implementation using k6 is described and design choices are considered. Tools, languages and data collection of aggregated results are also explained. The chapter 3 ends with lessons learned.

The quantitative literature study between service meshes and the quantitative results of the performance benchmarking are presented in chapter 4. From the two evaluations, a decision tree for service meshes is derived at the end of the chapter.

A summary of the work is given in chapter 5. This is followed by an evaluation of the objectives. The chapter also explains the differences with related work and discusses possible future work. In the end, it is explained that service meshes are not the silver bullet for everything.

2 Background Information

This chapter provides the basics which are necessary for the understanding of the chapters 3 and 4. Chapter 2.1 discusses the different terms such as Performance, Domain Specific Language, Infrastructure as Code, Helm, JavaScript, Go, C4 and microservices. The most important Kubernetes resources for this thesis are explained in chapter 2.2. The principles, benefits and alternatives to a service mesh are explained in chapter 2.3. This chapter also covers the Service Mesh Performance specification.

2.1 Terminology

2.1.1 Performance

Key Performance Indicators

These are the terms which Martin Fowler use in [4] to describe performance:

Response time is the time it takes for a system to process an external request, which could be a User interface action or a server Application programming interface (API) call.

Responsiveness refers to how quickly a system acknowledges a request, as opposed to processing it. It's important for user satisfaction, as low responsiveness can be frustrating even if response time is good. If a system waits until the request is complete, responsiveness and response time are the same. However, if the system indicates that it has received the request before it's finished, responsiveness is better. For example, providing a progress bar during a file copy improves the responsiveness of the user interface, even though it doesn't impact response time.

Latency is the minimum amount of time it takes to get a response, even if there is no work to be done. This is often a challenge for remote systems, as the time taken for the request and response to travel across the wire can cause a delay. Latency cannot usually be improved by application developers, and minimizing remote calls can help reduce it.

Throughput is how much work a system can do in a certain time. Throughput can be measured in bytes per second when copying a file, for example. For enterprise applications, transactions per second (TPS) is a typical measure, although it depends on the complexity of the transaction. Choosing a common set of transactions for a particular system can help standardize the measurement of throughput.

Performance is either throughput or response time, depending on what matters more to the system. Talking about performance when a technique increases throughput but decreases response time can be a challenge, so it's best to use the more precise term. Improving responsiveness, even if it means sacrificing response time or throughput, can improve performance from the user's point of view, as responsiveness may be more important than response time.

Load indicates how much stress a system is under, usually measured by the number of current users. Load is often used in context with other measurements, such as response time.

Load sensitivity refers to how response time varies with the load. For example, system A might have a response time of 0.5 seconds for 10-20 users, while system B has a response time of 0.2 seconds for 10 users but increases to 2 seconds for 20 users. In this case, system A has lower load sensitivity than system B. Degradation can also be used to describe how much more System B degrades than System A.

Efficiency is performance divided by resources. A system that achieves 30 TPS on two CPUs is more efficient than a system that achieves 40 TPS on four identical CPUs.

[4]

Test Types

To measure Key Performance Indicators, performance tests must be designed in a specific way. The following list from [5] outlines three common types or designs of performance testing:

Load/Volume tests: These tests are used to validate if an application can handle the expected volume of users, also known as concurrency/throughput. For example, a search function should respond within 2 seconds for a volume of 300 users. A performance test is designed to simulate this volume and validate that the application is meeting the expected target for the response time. It may be necessary to repeat the test multiple times to observe consistency and measure the average to establish a benchmark for the application.

Stress tests: An application's performance may begin to degrade as more users stress it. For instance, for X number of users, it can perform within acceptable limits, but it may start to respond with delays beyond X users, and finally, with X+n users, it may respond with errors. It is important to know the exact measures of these figures for infrastructure planning while scaling the application to new countries or during sales. To determine at what point an application begins to fail, the performance test gradually increases the load on the system in small increments beyond the limits of the volume test. This process of subjecting the system to extreme conditions to determine its breaking point is known as stress testing.

Soak Tests: Long-term operation of an application under heavy load can lead to a decrease in responsiveness due to infrastructure problems, memory leaks, or other factors. To see how the application behaves under sustained load and evaluate its performance, a soak test is performed. This performance test is designed to maintain a constant load over a long period.

[5]

Load patterns

We discussed earlier the different types of performance tests that can help measure Key Performance Indicators. These tests involve generating various load patterns on the application by adjusting four parameters: think time, ramp-up time, pacing and number of users at the same time. There are three tested load patterns [5]:

Steady Ramp-Up Pattern: In the steady ramp-up pattern, the number of users is steadily increased within a specific time frame, and then the load is maintained at a constant level for a sustained period to measure performance. This is the typical scenario for

most real-world applications, such as Black Friday sales, where users gradually enter the application and stay there for some time before leaving.

Step Ramp-Up Pattern: In the step ramp-up pattern, users are ramped up in batches periodically, such as 100 users every 2 minutes. This pattern is useful for observing and measuring the application's performance for each step count of users, which can help benchmark the application for different loads. The step ramp-up pattern is often used in performance tuning and infrastructure capacity planning.

Peak-Rest Pattern: The peak-rest pattern involves ramping up the system to reach peak load and then ramping it down to complete rest in repeated cycles. This pattern applies to some applications such as social networking, where the peak usage comes and goes in cycles throughout the day.

[5]

2.1.2 Domain Specific Language

Martin Fowler's definition in [6] consists of four essential components.

Computer programming language : A Domain Specific Language (DSL) is a language created by humans to instruct computers. It should have a structure that is easily comprehensible by humans, while also being executable by computers, like any modern programming language.

Linguistic nature: A DSL is a programming language and therefore should possess fluency that stems not only from individual expressions but also from their ability to be combined together.

Limited expressiveness: General-purpose programming languages have diverse capabilities. Although these are useful, they make learning and usage harder. A DSL should have the minimum number of features necessary to support its domain. One cannot create an entire software system using a DSL; instead, a DSL is employed for a particular aspect of a system.

Domain focus: Limited language is only valuable if it has a clear focus on a small domain. The domain focus is what gives a limited language its worth.

[6]

2.1.3 Infrastructure as Code

Infrastructure as Code (IaC) is a methodology for automating infrastructure tasks using practices derived from software development. Its focus is on establishing reliable and repeatable processes for provisioning and altering system configurations. In IaC, after making changes to the code, you test via automation and implement those modifications on your systems.[7]

2.1.4 Helm

In 2015, the Helm project was initiated a few months before the first KubeCon. Setting up Kubernetes was challenging, with new users needing to compile the Kubernetes source code and use shell scripts to run it. Additionally, once the cluster was up and running, users had to create Yet Another Markup Language (YAML) from scratch with few basic or production-ready examples available. The primary focus was to simplify the process of getting started

with Kubernetes. To achieve this, it should take less than five minutes for a new Helm user with an existing Kubernetes cluster to download and install an application.[8]

2.1.5 JavaScript

As the primary programming language, web development relies heavily on JavaScript. Nearly all websites use JavaScript, and modern web browsers on various devices such as desktops, tablets, and smartphones include JavaScript interpreters, making it the most widely deployed programming language in history. Over the past decade, Node.js has extended the use of JavaScript beyond web browsers. The immense success of Node.js has also resulted in JavaScript becoming the most popular programming language among software developers.[9]

2.1.6 Go

Go, also known as Golang, is an open-source programming language created by Google. Its developers, Robert Griesemer, Rob Pike, and Ken Thompson, aimed to design a language for modern systems programming that addresses practical issues encountered when building large-scale systems. Unlike other statically typed and compiled languages, Go is designed with features that make it feel dynamic, and its runtime includes garbage collection. The language is intended for use in large codebases developed by large teams. While Go is primarily a programming language, it also comes with a toolchain that supports other elements of application development, such as testing, documentation, formatting, and package management. This toolchain has facilitated the creation of an ecosystem around Go, thanks to its built-in package system and common toolchain for essential development elements.[10]

2.1.7 C4

The C4 diagrams can be linked to maps of a software system that allows you to zoom at different levels of detail. Importantly, the C4 model is not a design process but rather a set of diagrams that can be utilized to depict the static structure of a software system. C4 stands for[11]:

- *Context*: provides an overview of the system by depicting key system dependencies and actors/roles/personas.
- *Container*: illustrates the high-level technology choices, the distribution of responsibilities across them, and how the containers communicate.
- *Component*: displays the key logical components and their relationships within each container.
- *Classes or Code*: is optional, and a small number of high-level UML class diagrams can be drawn to explain the implementation of a particular component.

2.1.8 Microservices

Microservices are modular and autonomous services that revolve around a particular business domain. These services encapsulate specific functionality and make it available to other services through networks, allowing for the creation of a complex system from these building blocks. For example, one microservice could handle inventory management, another could manage orders, and a third could manage to ship, ultimately forming a complete e-commerce system. The concept of microservices is a strategic architectural approach that prioritizes

flexibility and versatility in addressing potential challenges. Microservices are a type of service-oriented architecture that emphasizes clear service boundaries and independent deployability. They are not limited to a specific technology, offering technological neutrality as a significant advantage. From an external perspective, a microservice operates as a black box, hosting its business functionality on one or more network endpoints such as a queue or REST API, using the most appropriate protocols. Other microservices or programs access this functionality through these networked endpoints, while the internal details, such as the technology used or data storage method, are hidden from the outside world. This design principle, known as information hiding, minimizes the amount of information revealed through external interfaces and maximizes information contained within the component. As a result, changes within the microservice boundary will not impact upstream consumers, promoting independent releasability of functionality. In most cases, microservices avoid shared databases, with each microservice housing its database as needed. The clear separation between what is easily changeable and what is more difficult to change, as well as the stable service boundaries that do not change with internal implementation, results in systems with weaker coupling and stronger cohesion. It is also worth mentioning the Hexagonal Architecture pattern, first described by Alistair Cockburn. This pattern emphasizes the separation of internal implementation from external interfaces, allowing for the same functionality to be accessed through different types of interfaces. The hexagonal shape used to represent microservices pays homage to this architectural pattern, as well as serves as a visual differentiation from traditional services.[12]

2.2 Kubernetes

In the past, most software applications were large monoliths that ran as a single process or on a few servers. These outdated systems had slow release cycles and infrequent updates, requiring developers to package the entire system and give it to the operations team for deployment and monitoring. In the event of hardware failure, manual migration was necessary. However, these monolithic applications are now being replaced by smaller microservices, independently running components. This allows for faster development, deployment, and updating, as well as quick changes to meet changing business requirements. However, managing the system effectively becomes more difficult as the number of components increases and datacenters grow. This is where Kubernetes comes in. It allows developers to deploy and update their applications without relying on the operations team, while also automatically monitoring and rescheduling in case of hardware failure. Kubernetes abstracts away the underlying hardware and exposes the datacenter as a single computational resource. This makes it ideal for both on-premises datacenters and large cloud providers, who can offer a simple platform for running any type of application without knowledge of the specifics of each individual one. As more companies adopt the Kubernetes model, it has become the standard way of running distributed applications in the cloud as well as on the local infrastructure. The increase in the number of deployable application components in a system can lead to difficulties in managing them. Google was likely one of the pioneers in recognizing the need for an improved method for deploying and managing software components and infrastructure globally. As one of the few companies operating hundreds of thousands of servers, it has been forced to tackle the challenge of deploying at such a large scale. This led to the development of solutions to make the process of developing and deploying thousands of software components manageable and cost-effective. Over time, Google running hundreds of thousands of servers, they encountered challenges in managing such a massive deployment. That prompted the development of an internal system, first Borg and later Omega, to simplify the process and

improve infrastructure utilization. The increased efficiency resulted in significant cost savings for the company. In 2014, after a decade of keeping Borg and Omega confidential, Google decided to release Kubernetes, an open-source system, which was built on the knowledge gained from the use of Borg, Omega, and other internal Google systems.[13]

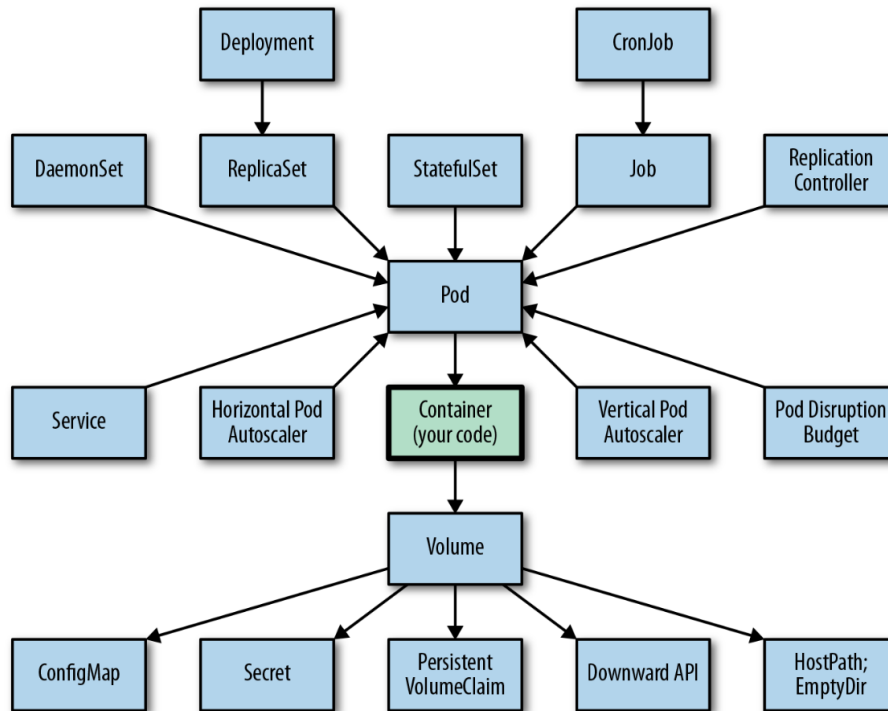


Figure 2.1: Overview of Kubernetes resources[14]

Figure 2.1 displays an overview of available Kubernetes resources. In the following chapters 2.2.1, 2.2.2, 2.2.3, 2.2.4, 2.2.5, 2.2.6, 2.2.7 and 2.2.8, the most important types for this thesis are explained.

2.2.1 Container

Containers offer repeatability as each container can include all the necessary dependencies to ensure consistent behavior across different environments. By decoupling applications from the underlying infrastructure, containers make it easier to deploy applications in various cloud or operating system environments. In a Kubernetes cluster, each node runs containers that form the Pods assigned to it, with the containers in a Pod co-located and co-scheduled to run on the same node. A container image is a self-contained software package containing everything needed to run an application, including the code, runtime, libraries, and default settings. Stateless and immutable are key characteristics of containers, meaning that any changes to the code of a running container require building a new image and recreating the container to run from the updated image. Responsible for running containers is the container runtime. Kubernetes supports different container runtimes, including containerd, CRI-O, and other implementations of the Kubernetes Container Runtime Interface(CRI). Typically, the cluster's default container runtime is used for a Pod, but you can specify a particular container runtime using RuntimeClass for Pods that require a different container runtime. [15]

2.2.2 Pod

The smallest computing units within Kubernetes are known as pods. A Pod is a group of one or more containers sharing network and storage resources and are described in a specification. The containers within a Pod are co-located, co-scheduled, and run in a shared context. Applications executed on the same machine in non-cloud environments are similar to cloud applications executed on the same logical host in this context. Apart from application containers, a Pod can include init containers that run during startup and ephemeral containers for debugging purposes, depending on whether your cluster supports these features. Although Kubernetes supports a number of other container runtimes in addition to Docker, it is the most widely recognized runtime, and it can be useful to use some of its terminologies when describing Pods. The shared context of a Pod comprises a set of Linux namespaces, cgroups, and other facets of isolation that containers also use. Within a Pod's context, individual applications may have additional sub-isolations. To summarize, a Pod is similar to a group of containers that share namespaces and filesystem volumes, making it a basic building block for running applications in Kubernetes.[16]

2.2.3 Service

With Kubernetes, there's no need to modify your application to use an unfamiliar service discovery mechanism. Pods are assigned their Internet Protocol (IP) addresses and a single Domain Name Service (DNS) name for a group of Pods. In addition, load balancing is automatically performed across them. However, Kubernetes Pods are created and destroyed according to the desired state of your cluster and are therefore non-permanent resources. While each Pod has its own unique IP address, the set of Pods running at one time may differ from the set of Pods running the application later. This presents a challenge: if a group of Pods (known as "backends") provide the functionality to other Pods (known as "frontends") within your cluster, how do the frontends discover and keep track of the appropriate IP address to connect to the backend? This is where Services come into play. A Kubernetes Service is an object that acts as a stable address for a set of Pods and provides a consistent way for frontends to access backends. By assigning a single IP address and DNS name to a group of Pods, Services enable frontends to easily access backends even as Pods come and go in a Deployment.[17]

2.2.4 Label

Objects in Kubernetes, such as pods, can be labeled with key/value pairs to specify meaningful and relevant attributes, without implying semantics to the core system. Labels allow for efficient organization and selection of subsets of objects and can be modified at any time. Each object can have a unique set of labels defined. Labels are particularly useful for efficient queries and watches and are ideal for use in User Interfaces and Command Line Tools (CLI). Annotations that are explained in chapter 2.2.5 should be used to record non-identifying information. With labels, users can loosely map their organizational structures onto system objects without requiring clients to store these mappings. In complex systems, such as service deployments and batch processing pipelines, multiple dimensions such as deployments, release tracks, tiers, and microservices per tier are common. Cross-cutting operations are often required, breaking the encapsulation of strictly hierarchical representations. Rigid hierarchies determined by the infrastructure, rather than by users, can be problematic for management. Labels provide a solution to this problem.[18]

2.2.5 Annotation

Kubernetes annotations allow you to attach any non-identifying metadata to objects, which can be retrieved by clients like tools and libraries. Labels and annotations both provide a way to attach metadata to Kubernetes objects, but they serve different purposes. Labels are used to select objects and find collections of objects that meet certain conditions, while annotations are not used for identification and selection. Annotation metadata can be small or large, structured or unstructured, and can contain characters not permitted by labels. Annotations and labels are both key/value maps, but the keys and values in the map must be strings. Numeric, boolean, list and other types cannot be used for neither the keys nor the values. Annotations are useful for recording a variety of information, such as fields managed by a declarative configuration layer, build, release, or image information, pointers to logging and monitoring repositories, client library or tool information for debugging purposes, user or tool/system provenance information, lightweight rollout tool metadata, and contact information for persons responsible. While this type of information could be stored in an external database or directory, using annotations allows for easier production of shared client libraries and tools for deployment, management, and introspection.[19]

2.2.6 Namespace

Kubernetes namespaces make it possible for resources to be isolated within a single cluster, ensuring that the names of resources are unique within a namespace but not across namespaces. Only namespaced objects such as Deployments and Services can be scoped using namespace-based scoping, and not for cluster-wide objects like StorageClass, Nodes, and PersistentVolumes. Namespaces are particularly useful in environments with multiple users spread across various teams or projects. However, for clusters with only a few to tens of users, namespaces may not be necessary. As such, start using namespaces when their features are needed. Namespaces are primarily used to provide scope for names, with each Kubernetes resource only able to exist within a single namespace. Namespaces cannot be nested, and are used to divide cluster resources between multiple users via resource quota. Using multiple namespaces is not necessary for separating slightly different resources, such as different versions of the same software; labels can be used instead to distinguish resources within the same namespace. Kubernetes always starts with four namespaces - default, kube-node-lease, kube-public, and kube-system.[20]

2.2.7 Deployment

Deployments are a Kubernetes resource that enables declarative updates to Pods and ReplicaSets. By specifying a desired state in a Deployment, the Deployment Controller controls the gradual transition from current to desired state. Deployments can create new ReplicaSets, update existing ones, or even remove them altogether and adopt their resources with new Deployments. Here are some typical use cases for Deployments[21]:

- Rollout a new ReplicaSet by creating a Deployment. The ReplicaSet will create Pods in the background, and you can monitor the status of the rollout to ensure success.
- Update the PodTemplateSpec of a Deployment to declare the new state of the Pods. The Deployment will create a new ReplicaSet and move the Pods from the old ReplicaSet to the new one at a controlled rate. Each new ReplicaSet will update the revision of the Deployment.

- Rollback to an earlier revision of a Deployment if the current state is unstable. Each rollback will update the revision of the Deployment.
- Scale up the Deployment to handle the increased load.
- Use the status of the Deployment as an indicator if a rollout is stuck.
- Clean up old ReplicaSets that are no longer needed.

[21]

2.2.8 Ingress and Ingress controller

The Ingress resource provides external access to Hypertext Transfer Protocol(HTTP) and Hypertext Transfer Protocol Secure(HTTPS) routes into the cluster and the routing of traffic on the basis of rules that are defined on the ingress resource. Ingress configuration can be used to load balance traffic, offer name-based virtual hosting, terminate Secure Sockets Layer(SSL) / Transport Layer Security(TLS), and provide externally-reachable URLs for Services. The Ingress controller, which is typically a load balancer, is responsible for fulfilling the Ingress, but may also configure your edge router or additional frontends to help handle the traffic. It is important to note that an Ingress does not expose arbitrary ports or protocols. In cases where services other than HTTP and HTTPS need to be exposed to the Internet, a Service of type NodePort or type LoadBalancer can be used. To make use of an Ingress, an Ingress controller must be in place. Simply creating an Ingress resource will not have any effect.[22] For the Ingress resource to function properly, an ingress controller must be operational within the cluster. Ingress controllers, unlike other types of controllers that run as part of the kube-controller-manager binary, are not automatically launched with a cluster. Kubernetes provides support and maintenance for Amazon Web Services, Google Compute Engine, and Nginx ingress controllers, giving users the option to choose the implementation that best meets their needs.[23]

Although all Ingress controllers should ideally conform to the reference specification, in reality, various Ingress controllers may operate slightly differently. [22]

2.3 Service Meshes

Service Meshes serve as a central infrastructure layer that manages both modernizing existing systems, such as monolithic architecture and controlling the growth of microservices. The rise of microservices in the cloud-native approach, which focuses on creating scalable and independently delivered services, has led to the growing popularity of Service Meshes. They allow for seamless communication between services through remote procedure calls (RPCs) and provide independence in language and technology choices for service teams. Service meshes provide a developer-focused network solution, eliminating the need for developers to incorporate network concerns into their code and allowing operators to define network behavior, identity, and traffic flow through policies. This approach is distinct from traditional software-defined networking (SDN), which is focused on network administrators. Most Service Meshes today are software-based, though hardware implementations may become available in the future. With its policy-based control, a service mesh can be considered a cloud-native SDN. Service Meshes use service proxies to carry traffic, which is intercepted and redirected through iptable rules in the pod namespace. This uniform layer of infrastructure along with service deployments is known as a service mesh. [24]

2.3.1 Service Mesh Concept

The fundamental concept is to direct all incoming and outgoing traffic from microservices through the service mesh layer, allowing traffic to be observed and modified. The service mesh includes APIs that enable integration with logging platforms, telemetry, and policy systems. Essentially, a service mesh can provide three key features: security, traffic management, and observability. To seamlessly implement service meshes into MSA, the sidecar pattern is employed, which will be explained in chapter 2.3.2.[1]

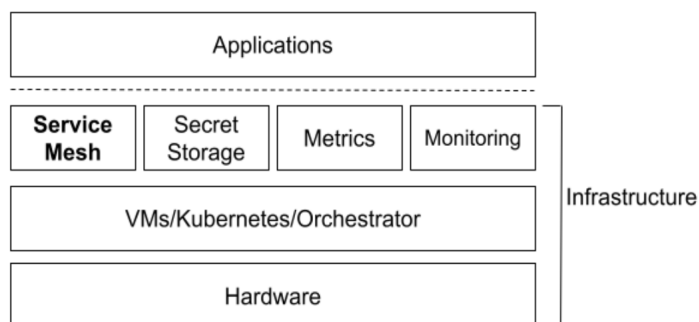


Figure 2.2: Service mesh is situated at the first infrastructure layer[25]

The Figure 2.2 shows the standard architecture where services and applications operate on top of the infrastructure. The service mesh is situated at the first infrastructure layer, alongside storage, metrics, and other high-level infrastructure needs. Beneath this layer are the Virtual Machines, Kubernetes, or any other compute provider or orchestrator, on which everything is running. Below the previous layer is the physical hardware, or bare metal, that forms the foundation of the entire architecture.[25]

2.3.2 Sidecar Pattern

The Sidecar pattern is a widely used pattern that involves running a separate container (application or microservice) alongside the main microservice. Sidecar containers add extra features and capabilities to the main container. Related to service connectivity, the Sidecar pattern is frequently employed to implement intersystem and interservice connectivity logic outside of the main microservice. In the context of service connectivity, we'll have a look at the use of the sidecar pattern. Suppose there are two microservices, Microservice A and B, and you need to establish interservice communication between them, as shown in Figure 2.3. In cases where the communication logic involves complex network communication that is unrelated to the business capabilities of the service, it may take service developers a long time to integrate it into the service. Moreover, when multiple microservices are built, we may have to replicate the same capability multiple times and across multiple programming languages or frameworks.[26]

The Sidecar pattern allows us to offload the interservice communication logic into a separate runtime. This runtime is co-located with the main microservice. Deploying cloud-native applications with containers frequently involves utilizing a Sidecar, which is typically a co-located container running alongside the main container responsible for executing the business logic. When the Sidecar is utilized for interservice communication, the business logic of each service is relieved of the responsibility of managing the underlying network communication

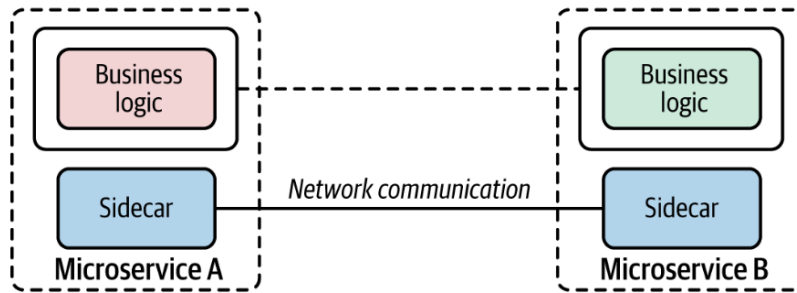


Figure 2.3: Interservice communication over Sidecar[26]

since the Sidecar is already handling it. As a developer, we only need to plug the Sidecar into the main container and configure it to achieve the preferred interservice communication logic. The primary container interacts with the co-located Sidecar component, typically through localhost, and the Sidecar handles all external communication. By adopting this approach, microservices are not expected to communicate directly with other services or systems. The runtime environment used for Sidecar should support all the essential features required to facilitate interservice communication. Using a high-level configuration language, we should also be able to configure the Sidecar. The same lifecycle is shared between the sidecar and the main microservice. It's important to keep in mind that a Sidecar can be used for any purpose that enhances the capability of the main microservice.[26]

2.3.3 Service Proxy

Generally, the Sidecar pattern is commonly implemented using a platform such as Kubernetes because it allows for easy encapsulation of a Sidecar-based microservices application as a Kubernetes pod. This results in a multi-container pod, consisting of the main container and the Sidecar container, as illustrated in Figure 2.4. By preserving a clear distinction between the extension or enhancement logic and the business logic, the entire application can be efficiently managed and scaled as a single unit. The Sidecar pattern can be utilized in various ways when building connectivity between microservices or external systems. One is a sidecar proxy, which can mediate inbound and outbound communication to the main microservices it's connected to. The main container calls the Sidecar running on localhost as it invokes an external service or system, and the Sidecar proxy then requests additional network communication features, such as security or service discovery logic. Here is an example: Two microservices, Checkout, and Inventory, communicate with each other via Envoy as the Sidecar proxy. The Envoy proxy is used to communicate between the two services via HTTP. The sidecars can be configured using Envoy's configuration management APIs to modify the behavior of the Sidecar proxy. As an example, when communication between two services occurs via the Envoy proxy, Envoy features can be utilized to establish security and reliability. Furthermore, the communication between the Inventory service and MongoDB can be externally monitored by enabling Envoy to intercept the communication using Envoy's MongoDB sniffing filters and forwarding the data to an external monitoring tool.[26]

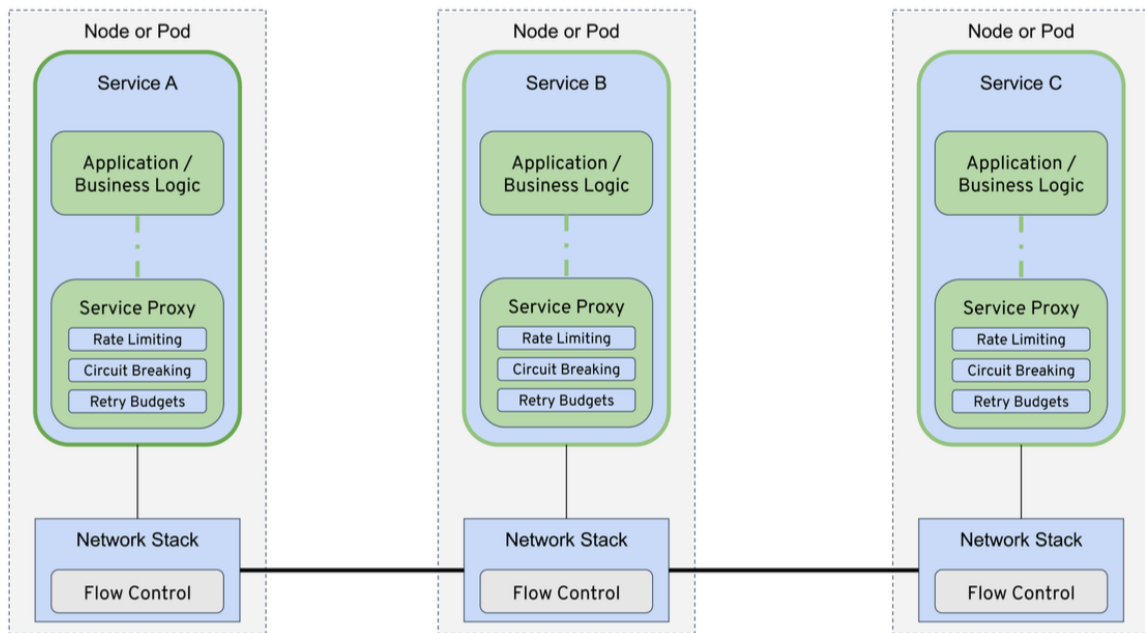


Figure 2.4: Service Architecture with service proxies[24]

2.3.4 Benefits of Service Meshes

Observability

Service meshes initially attract many organizations due to their ability to offer uniform observability. As no complex system is ever entirely healthy, service-level telemetry can illuminate where the system is performing poorly. This can help answer challenging questions, such as why requests are slow to respond. Although identifying when a service is offline is relatively straightforward, pinpointing the source and cause of slow performance can be a challenging endeavour. Service meshes provide a comprehensive approach to monitoring service-to-service communication that includes opaque monitoring of metrics and logs from outside the application, as well as code-level monitoring that provides observable signals from inside the application. Some service meshes integrate with distributed tracing libraries to enable code-level monitoring, while others support protocol-specific filters as a feature of their proxies, enabling a more granular level of visibility. Proxies, which comprise the data plane, are well-positioned to generate metrics, logs, and traces, providing uniform and comprehensive observability throughout the mesh. Organisations typically use separate monitoring solutions for different functions such as distributed tracing, logging, security and access control. Service meshes provide a centralised approach to addressing these observability challenges. Here are some of the benefits of using service meshes for this purpose: *Logging* is a crucial tool for gaining insight into the access requests made across your entire service fleet. Service mesh logging provides a comprehensive view of source and destination information, request protocol, endpoint (URL), associated response codes, and response time and size, enabling you to establish a baseline level of visibility. *Metrics* provide an alternative to relying on code instrumentation in the development process to collect data. When metrics are consistently collected across the entire cluster, they allow for new insights and can enable automation such as autoscaling. Telemetry data provided by service meshes includes service mesh metrics, which offer insights such as total request volume, success rate across the

service mesh, and individual service response data categorized by version, source, and time. *Tracing* is crucial for debugging slow services and visualizing service dependencies, request volumes, and failure rates. Without tracing, manually enumerating service dependencies in a spreadsheet can be time-consuming and error-prone. Service meshes simplify the integration of tracing functionality through automatically generated span identifiers. While individual services still need to forward context headers, Sidecar proxies in the mesh can trace the flow of requests across services without requiring manual instrumentation, unlike many application performance management (APM) solutions.[27]

Traffic control

Service meshes enable precise and explicit management of network traffic, enabling actions such as routing requests for canary releases. They also offer a range of resilience features, including circuit breaking, load balancing that takes into account latency, service discovery that is eventually consistent, timeouts, deadlines, and retries. *Timeouts* cancel service requests that exceed a predetermined time limit. They restrict the duration of any given request and are enforced when a response is deemed invalid or too lengthy for the client (user) to tolerate. *Deadlines* are a sophisticated service mesh functionality that enables feature-level timeouts for a group of requests, instead of individual service timeouts, thus helping to prevent retry storms. With deadlines, the remaining time to process a request is continuously deducted at each step, and the elapsed time is propagated with each downstream service call as the request traverses the mesh. In the event of a service timeout or unsuccessful return, you may consider retrying the request. However, simple *retries* can exacerbate the issue by repeatedly calling a service already struggling with the load (e.g., three retries equal 300% more service load). Retry budgets, also known as maximum retries, offer the advantage of multiple attempts but with a constraint to avoid overloading an already burdened service. Certain service meshes go a step further in alleviating client contention by incorporating a jitter and an exponential back-off algorithm to determine the timing of the subsequent retry attempt. Alternatively, instead of retrying and contributing to the service's load, you can choose to fail quickly and terminate the service, preventing further calls to it. *Circuit breaking* is a strategy that uses configurable timeouts or failure thresholds to set safe limits and enable graceful failure, particularly for services that have slow response times. By implementing circuit breaking in a separate layer using a service mesh, the overhead on applications can be reduced, especially during times of high demand. *Rate limiting*, also known as throttling, is another technique used to ensure service stability by preventing spikes in requests from disrupting other clients. Rate limits are typically measured over a specific period and can be calculated using various algorithms such as fixed or sliding windows, sliding logs, and more. The main objective of rate limiting is to ensure operational efficiency and prevent oversubscription of services. By limiting how often a client, mostly identified by a user access token, can access your services, rate limiting protects your services from overuse and ensures operational resilience, such as limiting service A to handle only 500 requests per second. *Quota management*, also known as conditional rate limiting, differs from rate limiting in that it focuses on managing the consumption of requests based on business requirements, rather than just ensuring operational stability. While the same service mesh capability can be used for both features, they are presented differently to users. An example of a policy that can be implemented with Service Mesh is quota management, which limits the number of client requests that can be made to a service in a given time period. For example, a free service plan might limit user Lee to 10 requests per day. To enforce these limits, a distributed counter is used to track incoming requests, typically using an in-memory datastore such as Redis. *Conditional rate limits* is

a more flexible capability which enables users to define a set of attributes that trigger rate limiting, such as limiting the number of requests based on the user's subscription plan or other arbitrary criteria. They are a powerful tool for service consumption management in complex environments.[27]

Security

Many service meshes offer a certificate authority to manage keys and certificates that ensure secure communication between services. Each service generates its certificate, which gives it a unique identity. In the case of Sidecar proxies (discussed in chapter 2.3.2 and 2.3.3), these proxies take on the identity of the service and manage the lifecycle of certificates, including generation, distribution, refresh, and revocation. In a deployment using Sidecar proxies, local Transmission Control Protocol(TCP) connections are typically established between the service and its Sidecar proxy, while mutual Transport Layer Security (mTLS) connections are established between the proxies themselves, as depicted in Figure 1-9. Securing internal traffic within an application is a crucial security consideration. Service calls within an application used to be limited to a single monolith via localhost, but now they are exposed over the network. Allowing service calls without TLS on the transport can lead to security issues. Mesh-enabled services have strong cryptographic proof of their peers' identities when they communicate. These identities are used to construct access-control policies that determine whether a request should be serviced. Depending on the service mesh used, the policy controls the configuration of the key management system (such as the certificate refresh interval), and whether a request is accepted is determined by operational access control - Block and allow lists are employed to differentiate between permitted and prohibited connection requests. Additionally, they are employed to implement fine-grained access controls, including time-based restrictions. [27]

2.3.5 Alternatives to Service Meshes - Client libraries

Large internet companies were the first to successfully run their applications and services in a cloud environment, and many of them played a pioneering role in developing cloud infrastructure as we know it today. These organizations dedicated significant resources and effort to building libraries and frameworks tailored to a specific set of languages, which helped overcome the challenges of running services in a cloud-native architecture. Google created Stubby, Twitter developed Finagle, and in 2012, Netflix made its microservices libraries, known as NetflixOSS, open-source to the community. These libraries are specifically designed for Java developers and address cloud-native concerns. Due to their Java runtime focus, the libraries were restricted to be used only in Java projects. To incorporate them into our application, we must create a dependency on them, pull them into our classpath, and then integrate them into our application code. Distributing the responsibility of building resilience into each application eliminates central bottlenecks and is an advantageous trait in large-scale deployments on an unstable cloud infrastructure. While decentralizing and distributing the implementation of application resilience into the applications themselves mitigates some concerns about large-scale services architectures, it also introduces new challenges. The first challenge arises due to the expected assumptions of any application. If a new service is to be introduced into the architecture, it would be restricted by the implementation decisions made by other teams, which may lead to implicit constraints around an undefined protocol for interacting with the system. For instance, using NetflixOSS Hystrix requires Java or a Java Virtual Machine based technology, and load balancing with Netflix Ribbon necessitates a registry to discover service

2 Background Information

endpoints, such as Eureka. Additionally, using libraries for resilience patterns means using both circuit-breaking and load-balancing libraries, which could limit the implementation of resilience strategies. The second issue arises when introducing a new language or framework to implement a service.[28]. Figure 2.5 shows a service architecture with the client libraries discussed above.

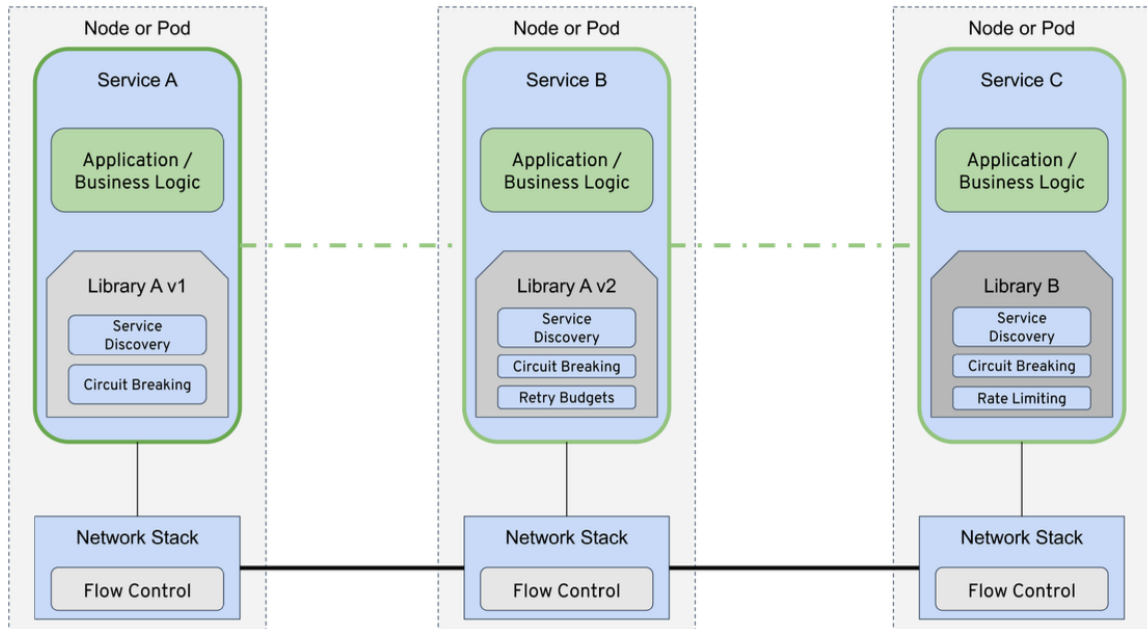


Figure 2.5: Service Architecture with client libraries [24]

For example, while NodeJS might be a better fit for implementing user-facing APIs, the rest of the architecture could use Java and NetflixOSS, necessitating the search for different sets of libraries for resilience patterns. Alternatively, finding analogous packages like resilient or hystrixjs for JavaScript could be an option. However, each library has a different implementation that makes different assumptions, which could lead to partial implementation for some languages and inconsistency in the overall implementation. This inconsistency makes it challenging to reason about failures, possibly contributing to obscuring or propagating them. While microservices enable a polyglot development environment, it is usually best to standardise on a handful of languages. Maintaining a handful of libraries across various programming languages and frameworks requires significant discipline and is challenging to accomplish. Consistency and correctness are critical to ensure that all implementations function correctly. A single deviation can introduce unpredictability, undermining the entire system. Additionally, updating and changing a fleet of services simultaneously can be overwhelming. While the decentralization of application networking is optimal for cloud architectures, the operational burden and constraints that it places on a system can be difficult for most organizations to handle. Even if organizations take on the challenge, achieving it accurately is still challenging.[28]

2.3.6 Service Mesh Performance

SMP is a standardized approach for capturing and describing infrastructure capacity, service mesh configuration, and workload metadata. This approach provides numerous advantages,

2 Background Information

such as the capability to assess the effectiveness of cloud-native infrastructure within the service mesh and its network functions, measure SMP (Service Mesh Performance), establish a shared vocabulary and measurement to exchange performance information between systems and meshes, enable performance comparisons of service mesh tools and deployments, and establish a universal performance index to evaluate a service mesh's effectiveness relative to other organizations' environments. SMP is a vendor-agnostic specification that captures details such as environment and infrastructure information, the number and size of nodes, orchestrator details, service mesh configuration, workload/application information, and statistical analysis to characterize the performance. The Meshery [29] implementation serves as the canonical example of this specification. The specification defines a consistent set of statistical analyses to be performed for every performance test, as shown in the following excerpt.[30]

```
message PerformanceTestResult {  
    message Latency {  
        double min = 1;  
        double average = 2;  
        double p50 = 3;  
        double p90 = 4;  
        double p99 = 5;  
        double max = 6;  
    } ...  
}
```

Figure 2.6: Snippet of the SMP [30]

3 Performance Benchmarking and Proof of Concept

In chapter 2, we discussed the fundamentals of a Service Mesh and Kubernetes, as well as the terminology used in this thesis. Chapter 3 explains the technological implementation of the performance benchmarking and the proof of concept. The chapter 3.1 starts with the selection of the open-source service meshes. Chapter 3.2 is a comparison of service mesh architectures, which is relevant to the first research question. Chapter 3.3 describes the different test cases and test scenarios as well as the selected microservice architecture. Chapter 3.4 explains each service mesh configuration in detail. Chapter 3.5 then explains how to deploy service meshes in the Google Cloud Platform. The tools and languages used are listed in chapter 3.6. The collection of data, which will be analysed in Chapter 4.2, is described in Chapter 3.7. This is essential for answering the fourth research question. Chapter 3.8 describes in detail all the information needed to answer the second research question. Lessons learned are highlighted at the end of chapter 3.9.

3.1 Selection of Service Meshes

Two criteria were used to select the service meshes. On the one hand, a selected service mesh must be open-source. On the other hand, the popularity of the service meshes was evaluated on the basis of the GitHub stars awarded. A good place to find different services meshes is the landscape of the Cloud Native Computer Foundation (CNCF). On 15.October 2022, the order in the Table 3.1 was as follows:

Service Mesh	License	Written in	GitHub Stars	Backer
Istio	Apache License 2.0	Go	31625	CNCF
Consul	Mozilla Public License 2.0	Go	25496	HashiCorp
Linkerd	Apache License 2.0	Go	8940	CNCF
Kuma	Apache License 2.0	Go	2950	CNCF
Open Service Mesh	Apache License 2.0	Go	2492	CNCF
Traefik Mesh	Apache License 2.0	Go	1750	Traefik Labs

Table 3.1: Overview of Service Mesh selection

In the beginning, the selection was limited to three service meshes. The reasons for this are described in chapter 5.3. In addition, there are other service meshes, but service meshes under 1000 GitHub Stars were not considered. The data for the comparison comes from GitHub.com.

3.2 Comparison of Istio, Linkerd, Kuma and Traefik Mesh architecture

Istio, Linkerd, Kuma, and Traefik Mesh are all based on the concepts outlined in 2.3.1, 2.3.2 and 2.3.3. Each service mesh uses a Sidecar deployment. What is the same for all Service Meshes, there is always a Control plane and a Data plane. To keep the separation of responsibilities, the control and data planes always run in separate Kubernetes namespaces, practically they could also be deployed in the same namespace, but this reduces readability. However, there is a difference between the components used per service mesh in the data plane and where the Sidecar is located. In Figure 3.1, Figure 3.2 and Figure 3.4, the Sidecar is located next to the containers in the same Pod. However, in figure 3.3, the Sidecar is in the Kubernetes node. Therefore, a distinction can be made here once between two variants: Sidecar per application or Sidecar per node. The first choice benefits that we have more control over the application and are not completely dependent on a service proxy in the data plane. The disadvantage is the increased coordination effort and the need to run multiple containers. Sidecar per node has the advantage that I only have to run one container per Kubernetes node but has the disadvantage being a bottleneck to the control plane.

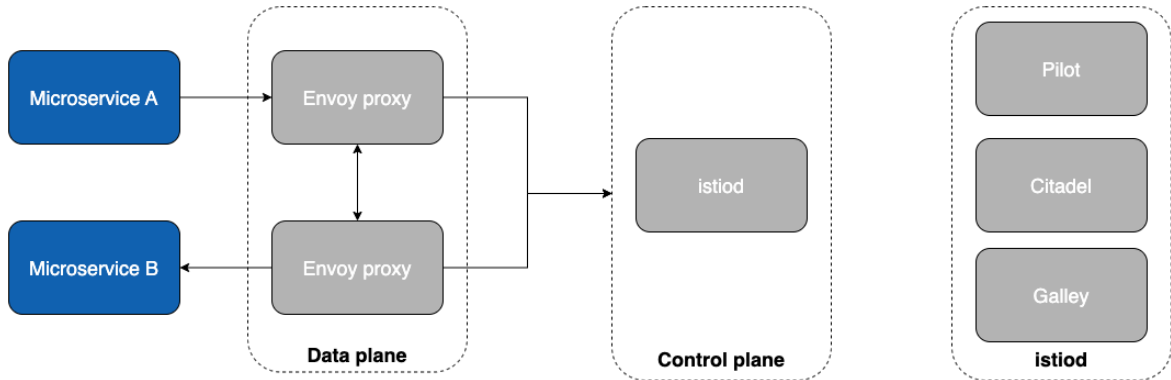


Figure 3.1: Istio Architecture [31]

Figure 3.1 shows the architecture of Istio. Istio uses an Envoy service proxy as a Sidecar deployment. No other service proxy can be used for the data plane. The Control Plane consists of Istiod. Istiod in turn consists of three components: Pilot, Citadel, and Galley. The current description for istiod [31] describes that it covers service discovery, configuration, and certification management, but it does not describe which component of istiod is responsible for this. In [32] the three components are described as follows:

- Pilot: The primary configuration service for the data plane is responsible for providing service discovery to the Envoy proxies, enabling intelligent routing with traffic management capabilities such as A/B testing, canary rollouts, and ensuring resilience through timeouts, retries, circuit breakers, and other mechanisms.
- Citadel: The responsibility of this service is to enforce robust authentication for both service-to-service and end-users, which includes built-in identity and credential management.
- Galley: The service is accountable for validating, ingesting, processing, and distributing configurations to other services.

Figure 3.2 shows the architecture of Linkerd. Linkerd is one of the few service meshes that use a self-developed service proxy, namely the Linkerd proxy as a Sidecar. In [34] the inventors

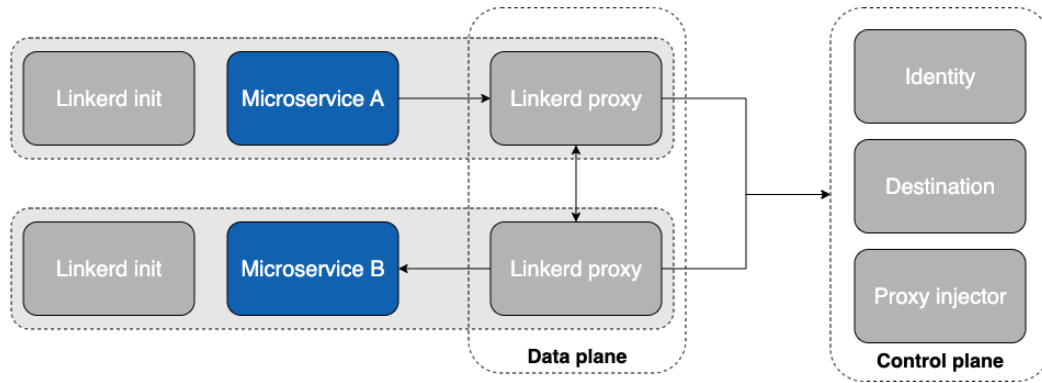


Figure 3.2: Linkerd Architecture [33]

of Linkerd describe why they think Envoy is not suitable for their service mesh.

The data plane proxies utilize the destination service to determine several aspects of their behavior. This includes fetching information related to service discovery, such as where to direct a specific request and the expected TLS identity on the receiving end, as well as policy information indicating which request types are permitted. Additionally, the destination service is responsible for retrieving service profile information that is used to inform per-route metrics, retries, timeouts, and other relevant details. The identity service role is to function as a TLS Certificate Authority that receives Certificate Signing Requests from proxies and issues signed certificates. These certificates are issued during the initialization of the proxies and are employed for proxy-to-proxy connections, allowing for mutual Transport Layer Security (mTLS) authentication. The proxy injector's primary function is to examine resources for a specific annotation that is unique to Linkerd. If this annotation is present, the injector modifies the pod's specification and adds both the Linkerd init and Linkerd proxy containers to the pod, as well as any relevant start-time configuration. Unlike in Istio an init container known as Linkerd init is included in every pod that is part of the service mesh. This container is added as a Kubernetes init container and is executed before all other containers within the pod. Its primary function is to utilize iptables to route all TCP traffic to and from the pod via the proxy. [33]

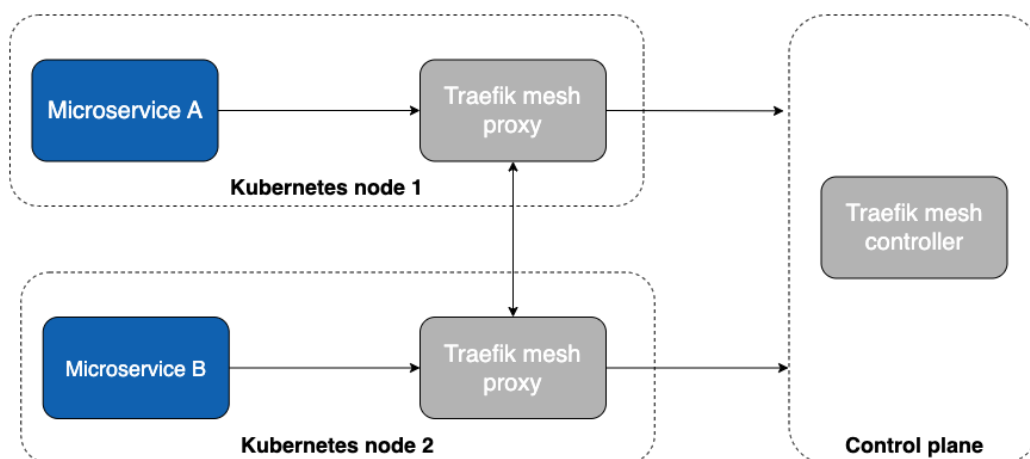


Figure 3.3: Traefik Mesh Architecture [35]

Traefik Mesh doesn't utilize Sidecar containers but instead manages routing via proxy endpoints that are operational on each Kubernetes node. A dedicated pod houses the mesh

controller, which manages the configuration parsing and deployment of the proxy nodes. Traefik Mesh provides several configuration choices, including annotations on user service objects and Service Mesh Interface objects. By not employing sidecars, Traefik Mesh doesn't alter Kubernetes objects or traffic without the user's knowledge.[35] Figure 3.3 shows the architecture of Kuma.

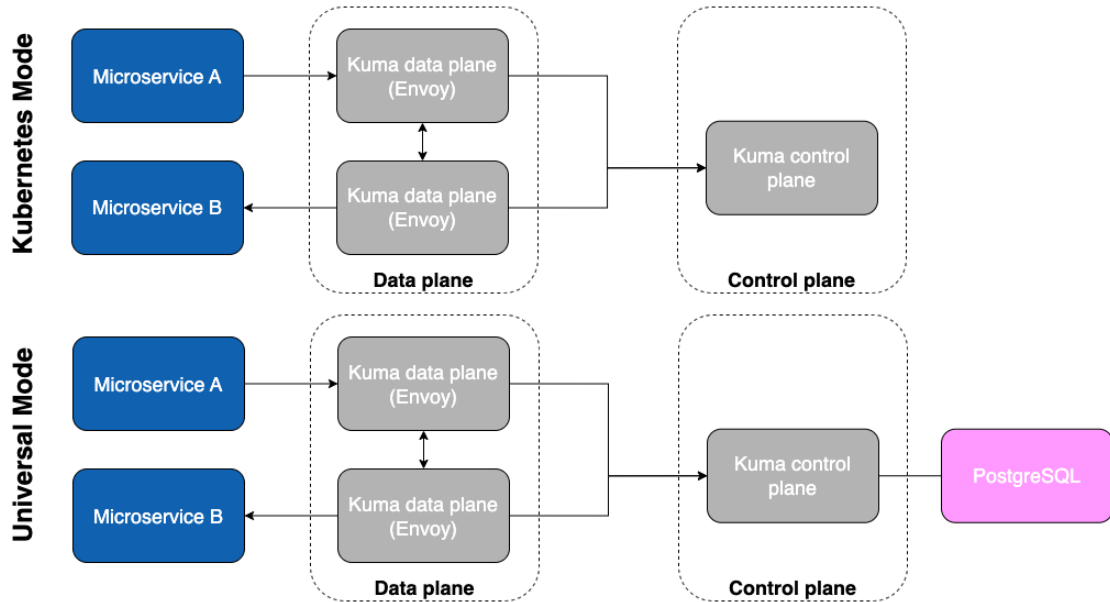


Figure 3.4: Kuma Architecture [36]

Figure 3.4 shows the architecture of Kuma. The special thing about Kuma is that it can be deployed in two variants, see Figure 3.4. In [36] the difference between the two variants is well explained. The bottom line is that the Universal Mode is available to run Kuma on an orchestrator other than Kubernetes. For the Universal Mode, a PostgreSQL database is required. Otherwise, the architecture is identical to the Istio architecture because an Envoy proxy is also used, but with a different naming.

3.3 Test environment

The different test types and load patterns are explained in chapter 2.1.1. Chapter 3.3.1 describes the four test cases, followed by the corresponding test scenario in chapter 3.3.2. Chapter 3.3.3 describes the microservice architecture application used for the service meshes. Why did we not create our own microservice architecture application for this thesis? Because the focus of the work is primarily on the performance comparison of the service meshes. Since a service mesh is available on the infrastructure layer, as described in Chapter 2.2, an existing microservice architecture application can be used on top of it. This has several advantages. On the one hand, it is faster, and on the other hand, a mature architecture can already be chosen, which is therefore less prone to bugs.

3.3.1 Test cases

Test case 1: Perform a 10-minute load test with 25 virtual users

Test case 2: Perform a 10-minute load test with 50 virtual users

Test case 3: Perform a 10-minute load test with 100 virtual users

Test case 4: Perform a 10-minute stress test, ramping up from 0 to 500 virtual users

3.3.2 Test scenario

The test scenario applies to all test cases described in chapter 3.3.1. First, the start page of the application is accessed. Then a currency is selected using a random number. Five currencies are available (Euro, US Dollar, Canadian Dollar, British Pound, Japanese Yen, Turkish Lira). After that, one of the ten products is selected at random. A random amount (max 5) of this product will be added to your shopping cart. At the end, the current shopping cart is displayed and the purchase is completed at the checkout. The scenario is repeated until ten minutes have elapsed. Figure 3.5 additionally shows the process as a flowchart. The test scenario is written in JavaScript, which is explained in chapter 2.1.5.

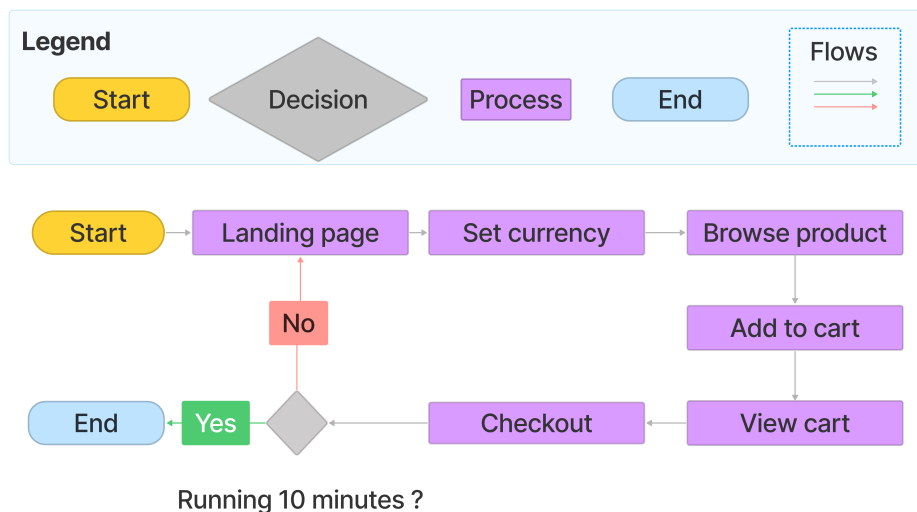


Figure 3.5: Flowchart of Test scenario

3.3.3 Microservice Architecture

The MSA [37] was used as the basis for the different service meshes. This consists of the following ten microservices (+ one Redis container as database) [37]:

- *Ad-Service* written in Java: Generates text ads based on given contextual keywords.
- *Cart-Service* written in C#: Stores the items present in the user's shopping cart and retrieve them as needed.
- *Checkout-Service* written in Go: Retrieves the user's cart, manages the payment, shipping, email notification process and prepares the order for shipment.
- *Currency-Service* written in JavaScript: Uses actual values obtained from the European Central Bank to convert currency amounts.

- *Email-Service* written in Python: Sends a confirmation email (mock) to users regarding their order.
- *Frontend-Service* written in Go: A HTTP server for website hosting and also generates unique session IDs for each user automatically.
- *Payment-Service* written in JavaScript: Accepts credit card information and processes a charge for a specified amount, returning a unique transaction ID.
- *Productcatalog-Service* written in Go: Offers a list of products, allowing users to search and view individual products.
- *Recommendation-Service* written in Python: Recommends products based on the shopping cart contents.
- *Shipping-Service* written in Go: Estimates shipping costs based on the items present in the shopping cart and delivers them to the address provided.

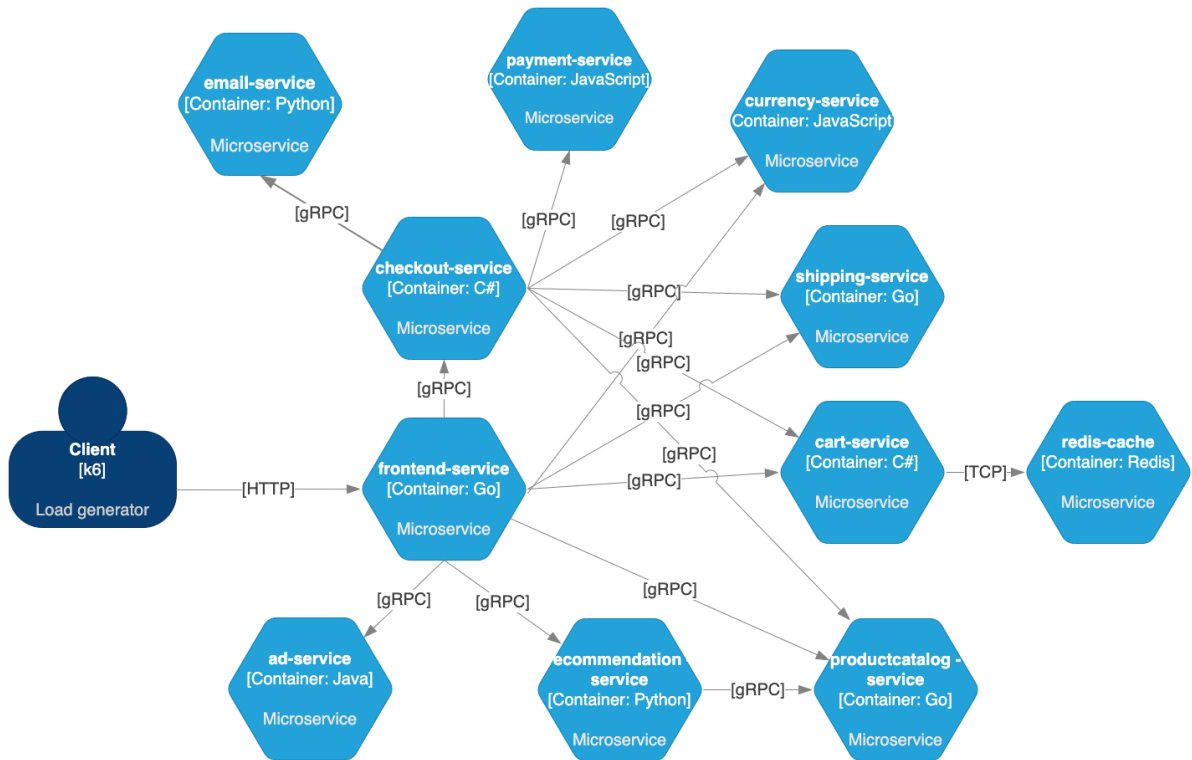


Figure 3.6: Microservice architecture based on [37] illustrated as C4 model

C4 is explained in chapter 2.1.7.

3.4 Configuration of Service Meshes

In order to operate a service mesh, a MSA as described in chapter 3.3.3 is required and an orchestration tool like Kubernetes is necessary. Kubernetes is explained in chapter 2.2 and Microservices in chapter 2.1.8. Additionally, each service must be containerized, and the Kubernetes resources must be described by YAML files. For the selected MSA from [37], a YAML description is already available, but without configuration for the respective microservice. To make the service mesh measurable later for quantitative comparison, automatic injection via the pod was chosen as the injection policy. It should be mentioned here that

in addition to this variant, different variants per service mesh are possible. Traefik Mesh is the only service mesh where no injection policy is needed because Sidecar deployment takes place per node. All four consist of eleven services and eleven deployment resources. In addition, there is one ingress resource per service mesh. In the following, the structure of a deployment and a service resource is explained on the basis of the email service. The other deployments/services differ only in other unique names and different ports.

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: emailservice
5  spec:
6    selector:
7      matchLabels:
8        app: emailservice
9    template:
10     metadata:
11       labels:
12         app: emailservice
13     spec:
14       serviceAccountName: default
15       terminationGracePeriodSeconds: 5
16       containers:
17       - name: server
18         image: gcr.io/google-samples/microservices-demo/emailservice:v0.5.0
19         ports:
20         - containerPort: 8080
21         env:
22         - name: PORT
23           value: "8080"
24         readinessProbe:
25           periodSeconds: 5
26           exec:
27             command: ["/bin/grpc_health_probe", "-addr=:8080"]
28         livenessProbe:
29           periodSeconds: 5
30           exec:
31             command: ["/bin/grpc_health_probe", "-addr=:8080"]
```

Listing 3.1: Deployment resource of email-service

Listing 3.1 shows a configuration of a deployment resource without a service mesh. In Line 1, the Kubernetes version for the object is determined. Then in line 2, the type of object is determined, other possibilities are shown in figure 2.1. In Line 3, a unique name is defined for the object. Optionally, the namespace can also be specified, if it is not present, default is used. From line 4 the state of the object is described. Lines 6 -8 describe how the mapping for the ReplicaSet to the pod is done. From lines, 9-31 pod properties are defined. Line 12 defines a unique name for the pod. From the containers field, the containers in the pod are described, in this description, there is only one. After that in line 17 a name for the pod is given and in lines 18-19, on the one hand, the location of the image is given and the port of the container is defined. In lines 27 and line 28 a readiness and a liveness probe are defined. Important is Line 15 This is the number of seconds to wait before the process of the Pod will be terminated. Between lines 21-23, another environment variable is defined for the container in the pod.

In Listing 3.2 is an example of a Service Resource. Lines 1 - 4 are similar to the previous

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: emailservice
5  spec:
6    type: ClusterIP
7    selector:
8      app: emailservice
9    ports:
10     - name: grpc
11       port: 5000
12       targetPort: 8080
```

Listing 3.2: Service resource of email-service

Listing 3.1, except that in line 2, instead of Deployment, the Service Resource is selected. In line 7, the type of service is selected, in this case ClusterIP. Besides ClusterIP, there are also LoadBalancer and NodePort. In line 8, it is specified to which pod the request should be forwarded. In this case, the previously labeled emailservice. From line 9, the ports are defined. How do port and targetPort differ? port is used for communication within the cluster between the pods. At targetPort, the requests are forwarded. The container listens on this port.

3.4.1 Istio

Listing 3.3 shows how a Sidecar injection can be performed in Istio. The Listing from above 3.1 remains the same, only a key/value pair must be added to labels in line 9 as in Listing 3.3. Istio uses the following pair: `sidecar.istio.io/inject: "true"`. If no injection is desired for a pod, the value is set to `"false"`. In Istio, automatic injection is also possible for a complete namespace or manual injection via the supplied `istioctl` CLI.

```
1  # Omitted for brevity
2  spec:
3    selector:
4      matchLabels:
5        app: emailservice
6    template:
7      metadata:
8        labels:
9          app: emailservice
10         sidecar.istio.io/inject: "true"
11  # Omitted for brevity
```

Listing 3.3: Automatic Sidecar injection in Istio

3.4.2 Linkerd

With Linkerd, an annotation is used instead of a label. The difference is explained in chapter 2.2.5. In Listing 3.4, an annotation for automatic injection is added in lines 8-9. Linkerd uses the following key/value pair `linkerd.io/inject: enabled`. If no injection is desired for a pod, the value is set to `disabled`. In addition to this variant, an injection can also be carried out in Linkerd via the CLI `linkerd`.

```
1  # Omitted for brevity
2  spec:
3    selector:
4      matchLabels:
5        app: emailservice
6    template:
7      metadata:
8        annotations:
9          linkerd.io/inject: enabled
10     labels:
11       app: emailservice
12  # Omitted for brevity
```

Listing 3.4: Automatic Sidecar injection in Linkerd

3.4.3 Kuma

The configuration of Kuma is similar to Istio, see Listing 3.5. The only thing that changes is the name of the key/value pair `kuma.io/sidecar-injection: enabled`. If no injection is desired for a pod, the value is set to `disabled`. With this service mesh, there is no other way to inject a Sidecar.

```
1  # Omitted for brevity
2  spec:
3    selector:
4      matchLabels:
5        app: emailservice
6    template:
7      metadata:
8        labels:
9          app: emailservice
10     kuma.io/sidecar-injection: enabled
11  # Omitted for brevity
```

Listing 3.5: Automatic Sidecar injection in Kuma

3.4.4 Ingress

In addition to the configurations from the previous chapter, access to the Kubernetes cluster must also be established. For this, an ingress is used as described in chapter 2.2.8. Two different providers were used for this work. The two ingresses from GKE and HAProxy are described below in Listing 3.6 and 3.7. The two resources are the same for all service meshes only the `metadata.name` is different. Lines 1-4 are the same as the previous listings. Starting from line 5 the Kubernetes resource is defined as where the requests should be forwarded to. In Listing 3.6 it is the frontend service with port 8080. Since there is only one service, it is not necessary to define rules.

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: linkerd-ingress
5  spec:
6    defaultBackend:
7      service:
8        name: frontend
9        port:
10         number: 8080
```

Listing 3.6: Ingress resource for Service Meshes with GKE

For the HAProxy Ingress, more fields have to be set than for the GKE Ingress. It is also important to distinguish which HAProxy is used because there is one version of HAProxy Ingress Controller and one of HAProxy Technologies. In this work, the Community Edition of HAProxy Technologies is used. For the HAProxy Ingress, more fields have to be set than for the GKE Ingress. In Listing 3.7, two additional annotations are set. One is the ingress class with `haproxy` and the other is the DNS for the frontend service with the defined port for the field `haproxy.org/request-set-header`. Starting from line 9 an ingress rule is defined for the frontend service. In line 13 the path is defined and then again the name of the service and the corresponding port. All three elements must match the respective service so that the incoming request can be forwarded correctly. Only one Ingress is ever used per service mesh. Theoretically, two or more Ingress could also be used.

3.5 Deployment

3.5.1 Provision a GKE Cluster

Terraform is used to create the cluster. Terraform is an Infrastructure as Code (IaC) tool to automatically create and modify infrastructure. IaC is explained in chapter 2.1.3. The creation of a cluster in GKE consists of several steps via the console, which is why it can be automated. This has the advantage that the results are comparable because each cluster is created in the same way and it saves time if the process is repeated several times, which is the case in this work. HashiCorp, the company that provides Terraform, provides an excellent tutorial [38] on how to create a cluster with two nodes. For this work, it has been adapted to create three nodes in a European region and to comply with the quota regulations for my account of GCP. The IaC requires the following five files, all written in the HashiCorp Configuration Language (HCL), a DSL (explained in chapter 2.1.2) for Terraform:

- `gke.tf`

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: linkerd-ingress
5    annotations:
6      kubernetes.io/ingress.class: haproxy
7      haproxy.org/request-set-header: |
8        15d-dst-override frontend.default.svc.cluster.local:8080
9  spec:
10    rules:
11      - http:
12        paths:
13          - path: /
14            pathType: Prefix
15            backend:
16              service:
17                name: frontend
18                port:
19                  number: 8080
```

Listing 3.7: Ingress resource for Service Meshes with HAProxy

- output.tf
- terraform.tfvars
- version.tf
- vpc.tf

3.5.2 Initialize Google Cloud Platform

1. `gcloud init`
2. `gcloud auth application-default login`

3.5.3 Provision a cluster

1. Initialize your Terraform workspace with `terraform init` - once only
2. Apply the configuration with `terraform apply`
3. Confirm your apply with *yes*
4. You need to configure `kubectl` with `gcloud container clusters get-credentials $(terraform output -raw kubernetes_cluster_name) -region $(terraform output -raw zone)`
5. Verify with `kubectl get` all the deployments/services are running

3.5.4 Deprovision a cluster

1. Running `terraform destroy` will de-provision the deployments and services we have created
2. Confirm your destroy with a *yes*
3. Check the status with `gcloud container clusters list`

3.5.5 Deployment of Service Meshes

The following paragraphs list the necessary commands for the respective service mesh to create the Kubernetes resources as shown in Figure 3.7. The commands required to install the control plane have been taken from the respective installation guides for Istio[39], Linkerd[40], Kuma[41] and Traefik mesh[42]. In general, the cluster must first be created using Terraform see chapter 3.5.1. The creation and destruction of a cluster take on average between 5-6 minutes. In addition, Figure 3.7 clearly shows how the individual components are deployed in the GCP. For this work, a Kubernetes cluster with 3 nodes was created in the Europe-West4 region (Netherlands). A separate namespace was used for the control and data plane. Additionally, it should be mentioned that the configuration for Istio, Linkerd, Kuma and Traefik Mesh cover the first variant with a GKE ingress. Paragraph HAProxy explains the additional steps for the HAProxy variant. Helm was used as the Kubernetes package manager for the various configurations, which is explained in chapter 2.1.4. The exact configuration is stored in a public GitHub repository [43], only the most important excerpts are shown below to be able to understand the configurations.

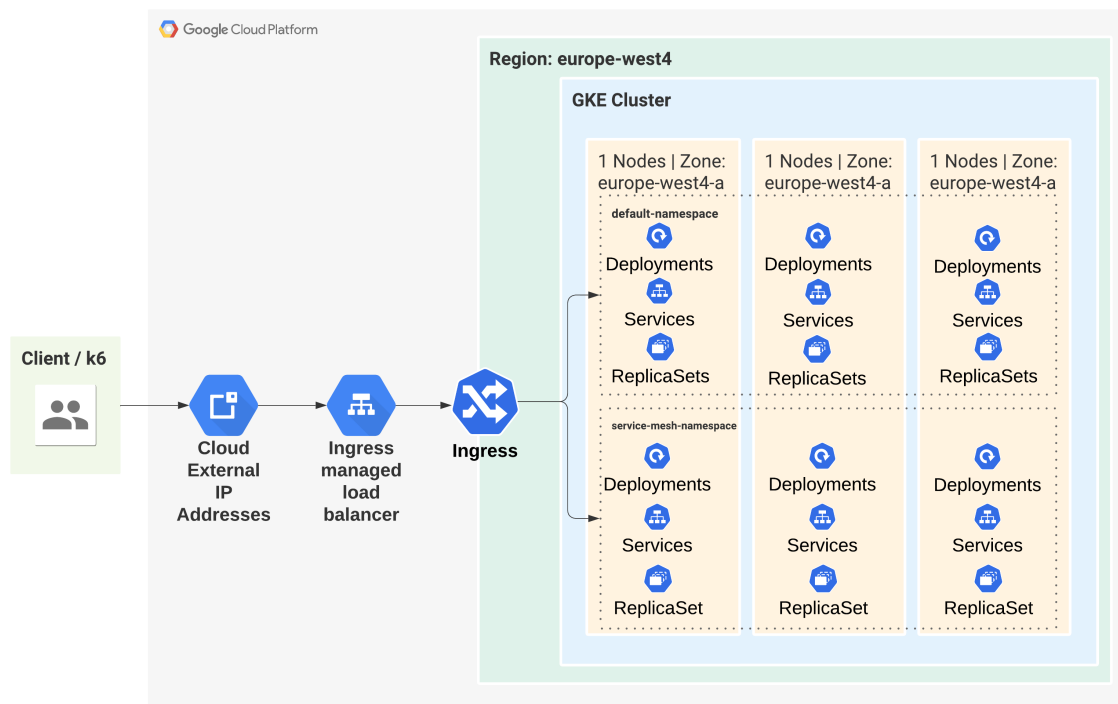


Figure 3.7: Deployment on GKE

Istio

1. Adding the Chart repository to the helm package manager `helm repo add istio https://istio-release.storage.googleapis.com/charts`
2. Install Custom Definition Resources(CDRs) and create namespace `istio-system`
3. Install Control Plane
4. Installing the Data Plane on the Cluster `helm install microservices ../helm-chart -f microservices-values.yaml`

```
1 # Omitted for brevity
2 sidecars:
3   istio: true
4   linkerd: false
5   kuma: false
6   traefik: false
```

Listing 3.8: Sidecar configuration in microservices-values.yaml

Depending on the service mesh used, the microservices-values.yaml must be customized. In the end, only flags have to be set differently so that the correct labels/annotations are used in the Kubernetes resources. Listing 3.8 shows the configuration for Istio. Depending on the service mesh, the flag is set from false to true. Theoretically, multiple flags of sidecar can also be set to true in parallel, but this combination has not been tried.

Linkerd

1. Install step - `brew install step`
2. Trust anchor certificate - `step certificate create root.linkerd.cluster.local ca.crt ca.key \ -profile root-ca -no-password -insecure`
3. Issuer certificate and key - `step certificate create identity.linkerd.cluster.local issuer.crt issuer.key \ -profile intermediate-ca -not-after 8760h -no-password -insecure \ -ca ca.crt -ca-key ca.key`
4. Setup Helm repo `helm repo add linkerd https://helm.linkerd.io/stable`
5. Install CRDs and create namespace `linkerd-system - helm install linkerd-crds linkerd/linkerd-crds -n linkerd-system -create-namespace`
6. Install Control plane - `helm install linkerd-control-plane -n linkerd \ -set-file identityTrustAnchorsPEM=ca.crt \ -set-file identity.issuer.tls \ -set-file identity.issuer.tls.keyPEM=issuer.key \ linkerd/linkerd-control-plane`
7. Installing the Data Plane on the Cluster - `helm install microservices ../helm-chart -f microservices-values.yaml`

Kuma

1. Add Chart repository with helm `helm repo add kuma https://kumahq.github.io/charts`
2. Install CDRs/Control Plane and create namespace `kuma-system - helm install -create-namespace -namespace kuma-system kuma kuma/kuma`
3. Installing the Data Plane on the Cluster - `helm install microservices ../helm-chart -f microservices-values.yaml`

Traefik Mesh

1. Adding the Chart repository to the helm package manager `helm repo add traefik https://traefik.github.io/charts`
2. Install Control Plane and create namespace `traefik-system - helm install traefik-mesh traefik-mesh/traefik-mesh -f traefik-values.yaml -create-namespace -namespace traefik-system -set kubedns=true`

3. Installing the Data Plane on the Cluster - `helm install microservices ../helm-chart -f microservices-values.yaml`

HAProxy

1. Adding the Chart repository to the helm package manager `helm repo add haproxytech https://haproxytech.github.io/helm-charts`
2. Install HAProxy - `helm install kubernetes-ingress haproxytech/kubernetes-ingress -set controller.service.type=LoadBalancer`

When the HAProxy is deployed, bullet points one and two must already be executed before the respective other points in previous configurations for Istio, Linkerd, Kuma and Traefik Mesh. In addition, the `microservice-values.yaml` must be adjusted as shown in Listing 3.9. The value of `ingressGke.create` needs to be set to false and `ingressHaproxy.create` must be set to true.

```
1  # Omitted for brevity
2  ingressGke:
3    create: false
4
5  ingressHaproxy:
6    create: true
7  # Omitted for brevity
```

Listing 3.9: Ingress configuration in `microservices-values.yaml`

3.6 Tools and Languages

The languages and tools, and where they were used, are listed in the following chapter.

3.6.1 Languages

JavaScript is described in chapter 2.1.5. The test cases for the last test tool k6 are written in this programming language. See more in the appendix under .1.

Go is described in chapter 2.1.6. The programming language is used in chapter 3.8 for the proof of concept.

HCL is a DSL which are described in chapter 2.1.2. This is required for Terraform in chapter 3.5.

YAML is mainly used for the Kubernetes resources. Descriptions of the Kubernetes resources of each service mesh can be found in Chapter 3.4.

3.6.2 Tools

Command Line Interface (CLI) are available with any service mesh, as well as with Google Cloud Platform, Helm, Terraform and k6. The version number is also shown here:

- `istioctl - v1.61.1`

- linkerd – v2.12.2
- kumactl – v2.0.2
- gcloud - v412.0.0
- helm - v3.10.1
- terraform - v1.3.4
- k6 - v0.39.0

Kubernetes is described in chapter 2.2. It has been used both locally and in the cloud. Minikube was used locally and GCP in the cloud. Kubernetes always consists of a client - *kubectl* and a server - Kubernetes API server. The Kubernetes API server in this case is minikube or GCP. Below are the versions used and the hardware setup for each environment:

- *Local*
 - Minikube - v1.28.0
 - Client – v1.25.2
 - Server - v1.28.0
 - 1 Node | 1 vCPUs | 8 GB RAM
- *Cloud*
 - GCP
 - Client – v1.25.2
 - Server – v1.24.7-gke.900
 - 3 Nodes | 6 vCPUs | 22,5 GB RAM | Storage 30 GB (10 GB per Node)

Helm is described in chapter 2.1.4. The Kubernetes package manager is used in chapter 3.5.5 for the Deployment.

Terraform is an IaC described in the chapter 2.1.3. This tool is used to provision the GKE cluster in chapter 3.5.1.

k6 is a load testing tool. It is used for both execution and proof of concept development.

Microsoft Excel is used in chapter 3.7 for the evaluation of the collected data.

Service Mesh is described in chapter 2.3. A comparison is given in chapter 3.2. Below are the versions used:

- Istio - v1.61.1
- Linkerd – v2.11.5
- Kuma – v2.0.2
- Traefik Mesh – v4.1.1

3.7 Collection of aggregated results

At the end of each test, data was saved from the following two sources:

- k6
- GCP Monitoring

The Load test tool k6 provides very extensive result outputs, but these can be extended or reduced with flags. In this work, the following two results of the k6 were used. On the one hand, the end-of-test summary, which includes many metrics, see 3.8. On the other hand, only p90, p95, RPS and failed requests per second were used for the evaluation.

```

● michaelsschneider@Michaels-MacBook-Pro load-testing % k6 --out csv=test_results_use_case_one_istio.csv run 1_use-case-one.js

      / \      / \      / \
     /   \    /   \    /   \
    /_____\  /_____\  /_____\
   /         \         \         \
  /           \           \           \
 /             \             \             \
/               \               \               \
\               /               /               /
 \             /             /             /
  \           /             /             /
   \         /             /             /
    \       /             /             /
     \     /             /             /
      \___/             /             /
         \             /             /
          \           /             /
           \         /             /
            \       /             /
             \     /             /
              \___/             /
                 \             /
                  \           /
                   \         /
                    \       /
                     \     /
                      \___/

execution: local
script: 1_use-case-one.js
output: csv (test_results_use_case_one_istio.csv)

scenarios: (100.00%) 1 scenario, 25 max VUs, 10m30s max duration (incl. graceful stop):
 * default: 25 looping VUs for 10m0s (gracefulStop: 30s)

running (10m08.9s), 00/25 VUs, 2778 complete and 0 interrupted iterations
default ✓ [=====] 25 VUs 10m0s

data_received.....: 293 MB 481 kB/s
data_sent.....: 7.0 MB 12 kB/s
http_req_blocked.....: avg=57.21µs min=1µs med=4µs max=80.27ms p(90)=6µs p(95)=6µs
http_req_connecting.....: avg=52.21µs min=0µs med=0µs max=78.08ms p(90)=0µs p(95)=0µs
http_req_duration.....: avg=81.16ms min=30.48ms med=80.7ms max=1s p(90)=113.4ms p(95)=127.84ms
  { expected_response:true }...: avg=81.16ms min=30.48ms med=80.7ms max=1s p(90)=113.4ms p(95)=127.84ms
http_req_failed.....: 0.00% / 0 / 33336
http_req_receiving.....: avg=2.26ms min=20µs med=1.91ms max=164.69ms p(90)=5.31ms p(95)=6.07ms
http_req_sending.....: avg=25.69µs min=5µs med=24µs max=451µs p(90)=39µs p(95)=41µs
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=78.87ms min=30.36ms med=77.56ms max=1s p(90)=110.55ms p(95)=124.67ms
http_reqs.....: 33336 54.748906/s
iteration_duration.....: avg=5.43s min=769.2ms med=5.22s max=12.12s p(90)=9.28s p(95)=9.94s
iterations.....: 2778 4.562409/s
vus.....: 1 min=1 max=25
vus_max.....: 25 min=25 max=25

```

Figure 3.8: Example of stdout from test case 1 with Istio

Additionally, k6 also generates a Comma-separated values (CSV) with the following columns: metric_name, timestamp, metric_value, check, error, error_code, expected_response, group, method, name, proto, scenario, service, status, subproto, tls_version, url, and extra_tags. In other words, a timestamp of each request with more meta information. Unfortunately, no new insights into the tests could be gained from this data, which were already available from the previous end-of-test summary. Also, Excel reaches its limit when more than 1 048 576 rows are occupied by simply truncating the results. In order for k6 to additionally write the queries to a CSV the flag `-out` must be used as shown in Listing 3.10.

```
1 k6 --out csv=test_results_use_case_one_istio.csv run 1_use-case-one.js
```

Listing 3.10: Command to run k6 with test case 1

In addition to k6, the RAM and CPU usage of the pods were measured during the test using the built-in monitoring tool from GCP. To do this, a separate dashboard would be created for each service mesh in the monitoring service as shown in Figure 3.9. After each test run, a CSV file can then be generated for each graph.

For chapter 4.2, the stored data were processed and made comparable using Excel.

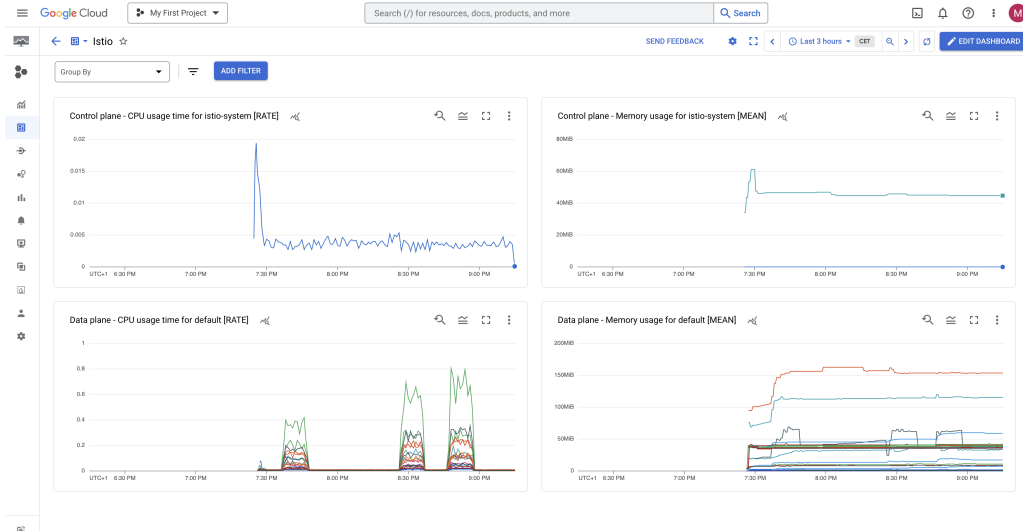


Figure 3.9: Self-built GCP Dashboard for Istio

3.8 Proof of Concept for Service Mesh Performance Output Extension

3.8.1 Problem Description

At the time of writing, there was a specification (SMP) for comparing service meshes, but only one implementation. What exactly the SMP is and what purpose it serves is already described in chapter 2.3.6. The previously mentioned implementation is called Meshery. Meshery is an open-source management plane that must be deployed into a Kubernetes cluster like a service mesh. Below is the functionality of Meshery based on [29]:

- Service Mesh
 - Performance Management
 - Configuration Management
 - Lifecycle Management
 - Interoperability and Federation
- Data Plane Intelligence

As listed above it can do much more than just compare the performance of service meshes, for a single load test that is a lot of overhead, plus Meshery skews the results as it runs in Kubernetes and therefore shares in the resources. To reduce this overhead, the idea was to extend an open-source tool to enable SMP-compliant load testing. The question then arose as to whether it was really necessary to write a load testing tool from scratch. There is a wide range of open-source load testing tools available. The goal was to adapt and extend an open-source tool to perform SMP tests.

3.8.2 Selection of load test tool

As mentioned in the previous chapter: No completely new load test tool was written. Instead, an overview of the current open-source products in this area was first evaluated, and then a load testing tool was selected based on two specific requirements. Only tools that have an open-source license are relevant. The second parameter refers to the popularity by

means of the GitHub stars. Furthermore, it should be extensible to be able to use it for the aforementioned problem. The table 3.2 shows the five most popular tools. There are a variety of other tools, but currently, they are ranked by GitHub stars to limit the number. Afterwards, small tests were performed with each tool and the documentation was compared. In the end, k6 was chosen over Locust because it is written in Go. This language is often used in the cloud environment, and the documentation of k6 is very extensive and detailed. The comparison was carried out on 1. November 2022, and the data was again taken from GitHub.com.

Load test tool	License	Written in	GitHub Stars
Locust	MIT License	Python	21k
k6	AGPL-3.0 License	Go	19.9k
JMeter	Apache License 2.0	Java	6.9k
Artillery	MPL-2.0 License	JavaScript	6.3k
Gatling	Apache License 2.0	ScalaJava	5.9k

Table 3.2: Overview of Load test tool selection

3.8.3 Implementation

To be able to extend k6 there is the possibility of extensions. k6 made a distinction between JavaScript and Output Extension. For the problem in chapter 3.8.1, the output extension is just right, because we collect the metrics, process them and then represent them as YAML. Since there are over 50 extensions, it was also checked whether such an extension is possibly already available, but many different specifications and protocols are supported, and the SMP is not available.

Structure

The extension consists of three files:

- go.mod - includes direct dependencies, our go module name xk6-output-logger and the go version
- go.sum - enumerates the checksum of both direct and indirect dependencies needed by the module.
- main.go - main program implements the output.Output interface.

Code

Implementation will be based on a tutorial [44] and an interface description in [45].

The main.go file consists of eleven functions:

1. **func** init()
2. **func** new(params output.Params) (output.Output, **error**)
3. **func** (*Logger) Description() **string**
4. **func** (o *Logger) Start() **error**
5. **func** (l *Logger) AddMetricSamples(samples []metrics.SampleContainer)

3 Performance Benchmarking and Proof of Concept

6. **func** metricKeyValues(samples []metrics.Sample) **string**
7. **func** (*Logger) Stop() **error**
8. **func** latenciesInMilliseconds(p90Indexlatencies []**float64**)
9. **func** createYamlFile(duration time.Duration)
10. **func** createFileNameAsIso8601() **string**
11. **func** kubectVersion() **string**

The functions 3,4,5 and 7 are method signatures of `output.Output` Interface which must be used to extend the binary with the corresponding function. At application startup, the Go runtime invokes the `init()` function.

The function `New(params output.Params)` creates and returns a new instance of the `Logger`.

`Description()` returns a human-readable description which will be displayed in `k6 run` output.

Function 4 is called by the engine prior to using the output. In general, it performs lengthy initialisation tasks and starts a goroutine that asynchronously flushes metrics to the output. This function starts a stopwatch.

Function 5 provides the latest metric samples from the Engine and is never called concurrently. Therefore, it is advised not to include any blocking operations that may take a considerable amount of time. This is where the helper function 6 is used.

The function `metricKeyValues(samples []metrics.Sample)` generates a string containing key-value pairs for all the metrics in the sample and increments a counter for each iteration if a certain condition is true.

`Stop()` flushes all the remaining metrics and concludes the test run.

Function 8 provides the calculated measured values such as minimum, maximum, mean, p50, p90, and p99.

`CreateYamlFile(duration time.Duration)` is called from the previous function `Stop()`. Here the data is serialized and written into a YAML file.

`createFileNameAsIso8601()` here, a timestamp is converted into a string according to International Organization for Standardization(ISO) 8601. This serves as the name of the generated file. `kubectVersion()` derives the current version from the command line tool - `kubectl`.

Build and use Extension

Go is the prerequisite for the commands to work.

1. Install `xk6` with `go install xk6`
2. To create a `k6` binary with this extension - `xk6 build -with xk6-output-logger=.`
3. Use the extension `./k6 run script.js -o smp -quiet -no-summary -vus 5 -duration 10s`
4. Generates a YAML File as Output with the current timestamp

Conclusion of the proof of concept

The previous chapter 3.8.3 has shown that it is possible to answer the second research question. Yes, it is possible to apply the service mesh performance specification to another load

test tool like k6. This means that it is no longer necessary to deploy Meshery on a Kubernetes cluster to perform performance tests. Another advantage of the proof of concept is that other test types described in 2.1.1, such as soak tests, can be performed because k6 supports them out of the box.

3.9 Lessons Learned

Subsequent subsections present the main findings from the benchmarking and the proof of concept.

3.9.1 Cluster debugging

Unfortunately, one of the main lessons learned from the many different configurations and test runs is debugging the Kubernetes cluster. The learning curve from a MSA to containerizing and orchestrating is very steep, but just because `kubectl apply` or `helm install` creates the appropriate Kubernetes resources doesn't mean the application will work as intended. Therefore, here is a list of the most common troubleshooting approaches in this work based on [46]:

- Debug Pods
 - Understanding Pod status
 - Pod stays pending - Insufficient resource
 - Pod stays pending - hostPort in use
 - Pod stays waiting - Image name is incorrect
- Debug Services
 - Missing Service
 - Use wrong Service type
 - Pod not working

Also, a great help is Lens [47] for monitoring and debugging Kubernetes clusters. A graphical interface for the CLI tool `kubectl`. Another helpful tool is Kubeshark [48], a traffic analyzer for the Kubernetes cluster.

3.9.2 Automation of infrastructure

During the configurations and benchmarking, one thing, in particular, stood out, which saved a lot of time. Automating the deployment of the cluster via Terraform and the creation of the Helm charts costs more time in the short term, but saves time in the longer term. It also reduces the error-proneness in the whole process because machines are more accurate than humans at monotonous and repetitive tasks.

3.9.3 Standardization of service meshes

It is very pleasing that there is already a second specification in addition to the SMP, namely the Service Mesh Interface. The key points of SMP have already been discussed in chapter 2.6. The most important points of Service Mesh Interface based on [49] are described below:

- Provider agnostic

- Kubernetes native - installable as CRDs
- Covers the most common features of the service mesh
 - Traffic policy
 - Traffic telemetry
 - Traffic management

On the one hand, the specifications disentangle the service mesh jungle of configurations. On the other hand, they increase interoperability, reusability, and flexibility among each other. Furthermore, it is certainly not a disadvantage that companies such as Microsoft, VMware, or RedHat are partners of the specification.

3.9.4 Lack of Documentation

In general, there is relatively much quantitative documentation in the respective service meshes, but less qualitative documentation. An exception is Istio, but it is also currently the most popular service mesh. This assumption can be confirmed by the fact that there are no reference implementations with more than three services in the respective documentation.

3.9.5 Built-in tools versus third-party tools

Especially in the area of observability with logging, tracing, and monitoring there are a lot of good open-source tools like Jaeger, Zipkin, Logstash, Grafana Loki, Prometheus, and many more. In my first conception and in the first test run a separate pod was used for Prometheus and Grafana, the data was unfortunately wrong, because an error was present in the test scenario. We also found that the third-party tools were very good at supporting us when we were doing everything ourselves, but did not justify the extra effort when we were using a hyperscaler like GCP. The modern hyperscalers already have all these tools integrated by default. This can save a lot of time because configuring observability tools takes some time.

3.9.6 k6 Ecosystem and Extension

At first glance, k6 is "only" a load testing tool. When you implement it and read the available documentation, you quickly realise that a large performance testing ecosystem has already been developed behind it. If a protocol or specification is not supported, it is simply possible to extend the binary as shown in the Proof of Concept(PoC). The ecosystem leaves nothing to be desired; besides messaging protocols such as Advanced Message Queuing Protocol and Message Queuing Telemetry Transport, serialization via Avro, or even distributed tracing, extensions are already being offered. There is an open-source version, which was used for this PoC, and a paid Software-as-a-Service version for enterprise use, with even more out-of-the-box functionality.

3.9.7 Go

Return of more than one value

A very useful language function, which I did not know from other languages like Java or C#, is the return of multiple values. It is shown in Listing 3.11. Here the function `LatenciesInMilliseconds()` returns five metrics which are later assigned to the five

3 Performance Benchmarking and Proof of Concept

variables `min`, `max`, `average`, `p50`, `p90`, `p99` in the function `CreateYamlFile()`. It is not an all-purpose solution, but it is very useful in certain situations.

```
1 // Omitted for brevity
2 func LatenciesInMilliseconds(p90Indexlatencies []float64) (min float64, max float64,
3 average float64, p50 float64, p90 float64, p99 float64) {
4 // Omitted for brevity
5     return min, max, sum / (float64)(len(latencies)), latencies[int(p50Index)],
6     latencies[int(p90Index)], latencies[int(p99Index)]
7 }
8
9 func CreateYamlFile(duration time.Duration) {
10
11     id := uuid.New()
12     requestsPerSeconds := float64(requests) / duration.Seconds()
13
14     min, max, average, p50, p90, p99 := LatenciesInMilliseconds(latencies)
15 // Omitted for brevity
```

Listing 3.11: Excerpt from the `main.go`

Standard Libraries

Unlike other languages, Go has a very modern and mature library of methods/functions. In addition, libraries for `crypto`, `database` and `html` are available as listed in [50], but these were not needed for the PoC. This makes programming much easier, as fewer dependencies simplify testing and the central package management in Go also makes searching extremely simple.

Toolchain

It must be said that Go is a very young language and has therefore benefited from the mistakes of older languages or learned from them. This includes the toolchain. In other languages like JavaScript, additional packages/tools are necessary to do the following things already described in chapter 2.1.6:

- Package management
- Testing
- Code coverage
- Format

Very valuable and makes learning a new language much easier.

4 Discussion

The discussion is divided into quantitative literature study and quantitative analysis of performance benchmarking. The quantitative literature study is explained in chapter 4.1, relevant to the third research question. The quantitative analysis of performance benchmarking is then presented in Chapter 4.2. In this way, the fourth research question and its sub-questions will be answered. In the end, the decision tree is presented. This can be an aid in the selection of an open-source service mesh.

4.1 Quantitative Literature Study

For the four service meshes Istio, Linkerd, Kuma, and Traefik Mesh, the following evaluation criteria were defined see Table 4.1. If a criterion is exceeded, a plus is awarded. If it corresponds to the description without additional features, a zero is awarded. If it is not present, a minus is awarded.

Assessment	Description
++ (Two)	Exceeds the criterion
+ (One)	Meets the criterion
0 (Zero)	Doesn't meet the criterion

Table 4.1: Assessment criteria

The criteria catalog was derived from [51], a NIST specification regarding the creation of secure microservice applications using service mesh architecture. In the specification, they call it the Service Mesh - Deployment Recommendation(SM-DR). A total of twelve criteria are described in table 4.2. They are divided into the following three categories: Observability(5), Traffic Control(4), and Security(3).

Criterion number	Derived from	Criterion description	Area
1	SM-DR1	It is necessary to include functionality that allows users to define a particular range of protocols and ports for a service proxy to receive traffic intended for its associated service. Unless otherwise specified by this configuration, the service proxy should block all traffic by default.	Traffic Control

Table 4.2: Derived criteria catalog from [51]

Criterion number	Derived from	Criterion description	Area
2	SM-DR2	It is essential to restrict the scope of services accessible by a service proxy. This can be achieved through capabilities that limit access based on the namespace, a specific named service within a namespace, or the service's run-time identity.	Traffic Control
3	SM-DR3	The service proxy ought to possess inherent capabilities to facilitate clients that communicate using different protocols than the target microservice. For instance, the proxy should be able to transform REST/HTTP requests into gRPC requests or upgrade HTTP/1.1 to HTTP/2.	Traffic Control
4	SM-DR6	For secure communication between service proxies, it is necessary to establish a mutual TLS (mTLS) session wherein all exchanged data packets are encrypted. This would ensure that only authorized parties can access the data.	Security
5	SM-DR21	The proxy should record errors arising from input validation, as well as errors associated with extraneous (unexpected) parameters, crashes, and core dumps.	Observability
6	SM-DR22	The proxy is expected to log the Common Log Format fields, at a minimum, for requests such as those resulting in non-200 HTTP responses.	Observability
7	SM-DR22	The log messages should comprise the name/identity of microservice, trace id of request, event message, event date, event time and other relevant contextual information.	Observability
8	SM-DR23	The configuration for the Service Mesh metrics collection for external client and microservice calls shall include, as a minimum, the number of client/service requests within a specified duration, the number of unsuccessful client/service requests categorised by failure code, and the average latency per service and the average total latency per complete request lifecycle.	Observability
9	SM-DR24	When configuring service proxies to enable distributed tracing, it is imperative to ensure that the application services are appropriately instrumented to propagate the headers of the communication packets they receive.	Observability

Table 4.2: Derived criteria catalog from [51]

Criterion number	Derived from	Criterion description	Area
10	SM-DR25	Data associated with retries, timeouts, circuit-breaking settings, or canary deployments, including all control plane configuration data, ought to be stored in resilient data stores, such as Key/Value stores.	Security
11	SM-DR26	To ensure the integrity of the load balancing information, it is necessary to closely integrate the health checking function for service instances with the service discovery function.	Security
12	SM-DR27	To configure the CORS policy for an edge service, it is recommended to utilize the Service Mesh capability instead of dealing with it through the microservice application service code.	Traffic Control

Table 4.2: Derived criteria catalog from [51]

4.1.1 Istio

Istio performs best in all three categories compared to the other service meshes and also offers many other functions in the area of security in addition to mTLS, such as peer authentication, request authentication, permissive mode, secure naming, authentication, and authorization policies. Moreover, the documentation is very extensive. The evaluations are shown in Table 4.3.

Criterion number	Verifiability through	Assessment
1	[52]	++
2	[53]	++
3	Not found, but in Envoy - [54]	0
4	[55]	++
5	[56]	+
6	[56]	+
7	[56]	+
8	[57]	++
9	[57]	+
10	[58]	+
11	[59]	+
12	[60]	+

Table 4.3: Quantitative Assessment for Istio

4.1.2 Linkerd

The results of Linkerd are summarised in Figure 4.1.2. According to the evaluation, it is the service mesh that shows no particular weaknesses, but in no area also very good. Linkerd

can score with simplicity though, so it is quickly deployable to the Kubernetes cluster via the included CLI.

Criterion number	Verifiability through	Assessment
1	[61]	++
2	[61]	++
3	Not found	0
4	[62]	+
5	[63]	+
6	[63]	+
7	[64]	+
8	[65]	+
9	[66]	+
10	[58]	+
11	[59]	+
12	Not Found	0

Table 4.4: Quantitative Assessment for Linkerd

4.1.3 Kuma

Kuma performs slightly better than Linkerd see Figure 4.5. However, there is an important difference in deployment. It can be deployed on non-Kubernetes platforms (Amazon Linux, CentOS, Red Hat, or CentOS) instead of Kubernetes, unlike all the service meshes compared here.

Criterion number	Verifiability through	Assessment
1	[67]	+
2	[68]	+
3	[69]	+
4	[70]	+
5	[71]	++
6	[71]	+
7	[71]	+
8	[72]	++
9	[73]	+
10	[58]	+
11	[74]	++
12	Not found	0

Table 4.5: Quantitative Assessment for Kuma

4.1.4 Traefik Mesh

Traefik Mesh cannot play to its strengths in any of the three categories. Later in chapter 4.2, it is shown that the strength of this service mesh lies in its performance and achieves very high throughput with the requests. The results for Traefik Mesh are shown in Figure 4.6.

Criterion number	Verifiability through	Assessment
1	[75]	+
2	[75]	+
3	Not found	0
4	Not found	0
5	Not found, but in Traefik Proxy [76]	0
6	[77]	+
7	Not found, but in Traefik Proxy [76]	0
8	[75]	+
9	Not found	0
10	[58]	+
11	[59]	+
12	Not found, but in Traefik Proxy [78]	0

Table 4.6: Quantitative Assessment for Traefik Mesh

4.1.5 Comparison

Figure 4.1 shows the total ratings of the service meshes for each assessment category. Overall, Istio scores the best and Traefik Mesh the worst. In between, tied on points, are Kuma and Linkerd. In chapter 4.3.1, the results are presented according to the three areas in Figure 4.15.

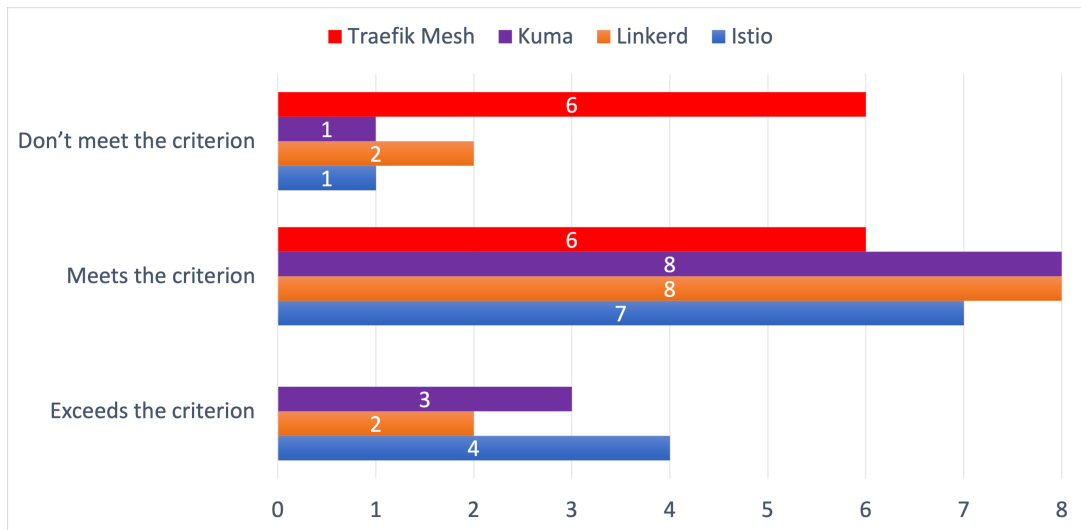


Figure 4.1: Comparison of quantitative literature study result

4.2 Quantitative Analysis of Performance Benchmarking

A total of three rounds of test runs were conducted. In the first and second round, the GKE Ingress was used. In the third round, HAProxy was used instead of GKE Ingress. In total, only two rounds would have been necessary, but in the first round, there was a bug in the test script that was only noticed at the end of the first round. A round was completed when all service meshes had run each test case once. In total, there were 48 runs. At the end of the three rounds, about 4.97 GB of data was generated for evaluation. The net time of the test runs was 480 minutes and approximately 2.31 million requests were generated.

4.2.1 Control Plane

Memory usage

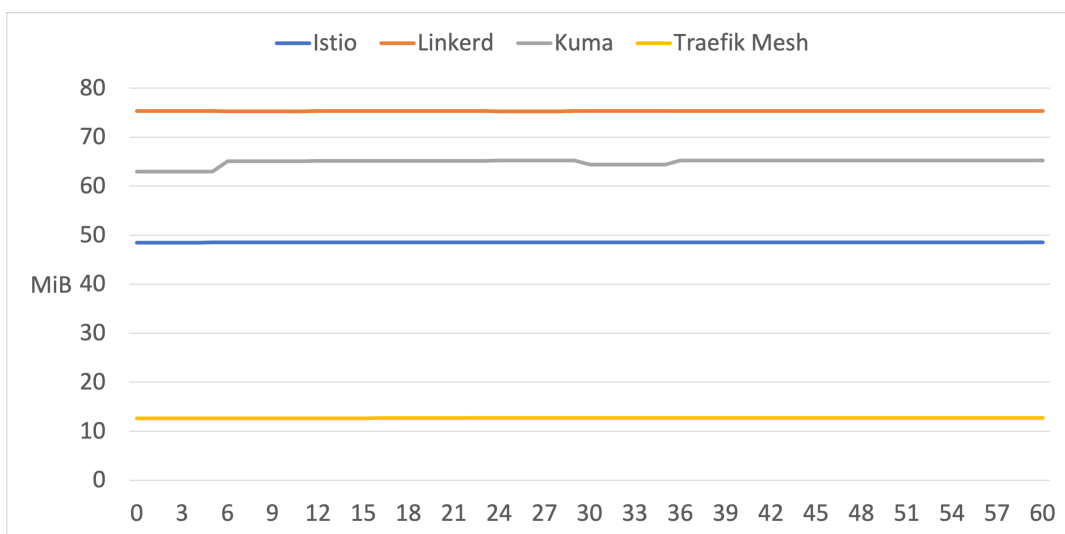


Figure 4.2: Test case 1 - Control plane - Memory usage

The results for the Control Plane in terms of memory usage are shown for test case 1 in Figure 4.2. It is also important to note that the results for test case 2 (Figure 1), test case 3 (Figure 2), and test case 4 (Figure 3) are almost identical. They differ partially only by other numbers in the decimal places. The Traefik Mesh requires the least memory, followed by Istio and Kuma. Linkerd requires the most memory, almost four times as much as Traefik Mesh.

CPU usage

The CPU usage in the Control Plane for test case 1 is relatively close. Traefik Mesh is ahead of Linkerd and Istio. Kuma needs almost 14 times as much CPU as Traefik Mesh. The first three are in the range between 1.5 and 4 ms, as shown in Figure 4.3. As with the memory usage from the previous subchapter, the values for test case 2 (Figure 4), test case 3 (Figure 5) and test case 4 (Figure 6) differ only minimally.

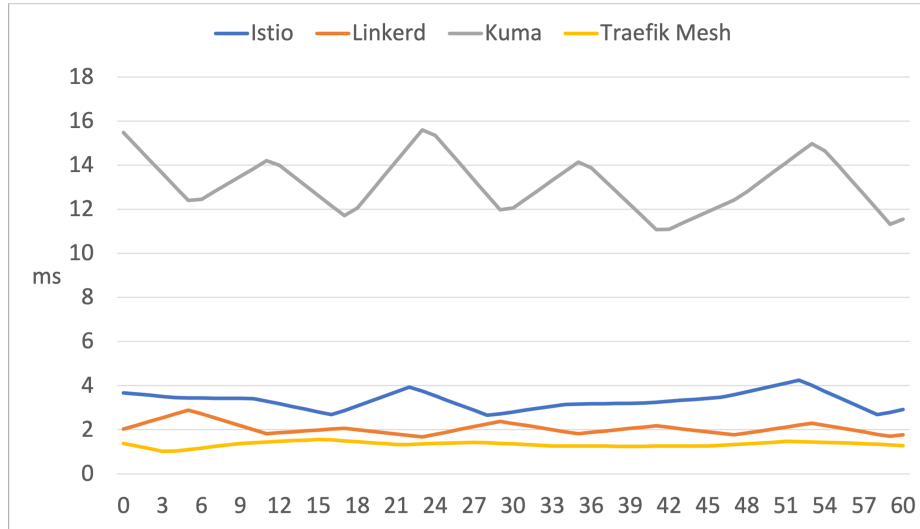


Figure 4.3: Test case 1 - Control plane - CPU usage

4.2.2 Data Plane - Application

Memory usage

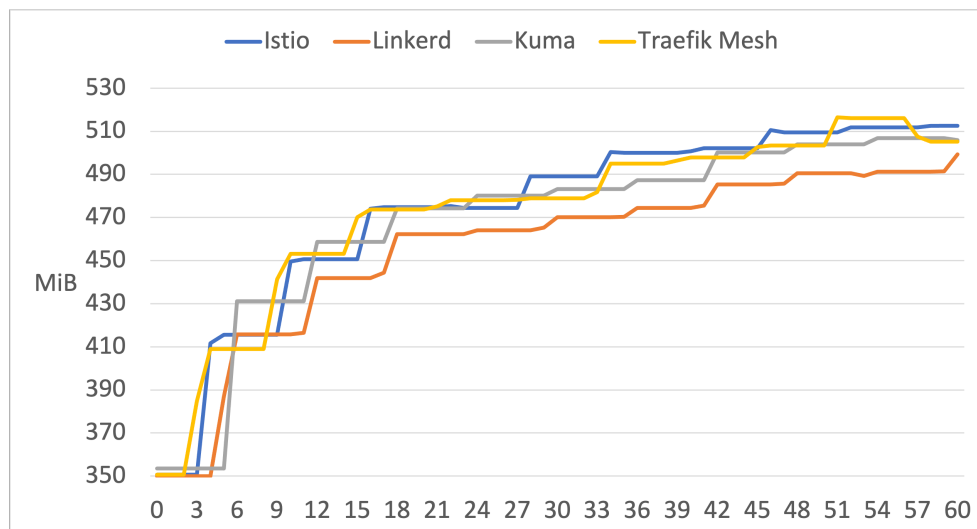


Figure 4.4: Test case 1 - Data plane - Memory usage - Application

The results for memory usage in the application are also scarce. Linkerd is ahead in test case 1 (Figure 4.4) and test case 2 (Figure 7). Istio is ahead in test case 3 (Figure 4.5) and test case 4 (Figure 8). Behind it, Kuma and Traefik Mesh alternate in the results between 3rd and 4th place. Istio is therefore only ahead when the number of virtual users increases, see Figure 4.5.

CPU usage

For CPU Usage, the trend in Figure 4.6 is the same for test case 1 (Figure 9), test case 2 (Figure 10) and test case 4 (Figure 11), although it is more visible with an increase in Virtual

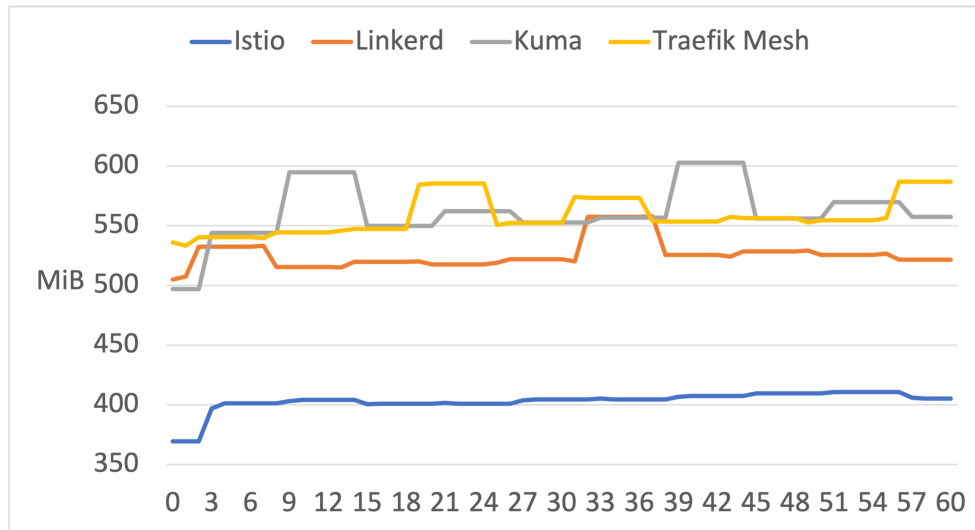


Figure 4.5: Test case 3 - Data plane - Memory usage - Application

User. Istio consumes the least CPU time and Traefik Mesh the most. Linkerd and Kuma are almost always on the same level.

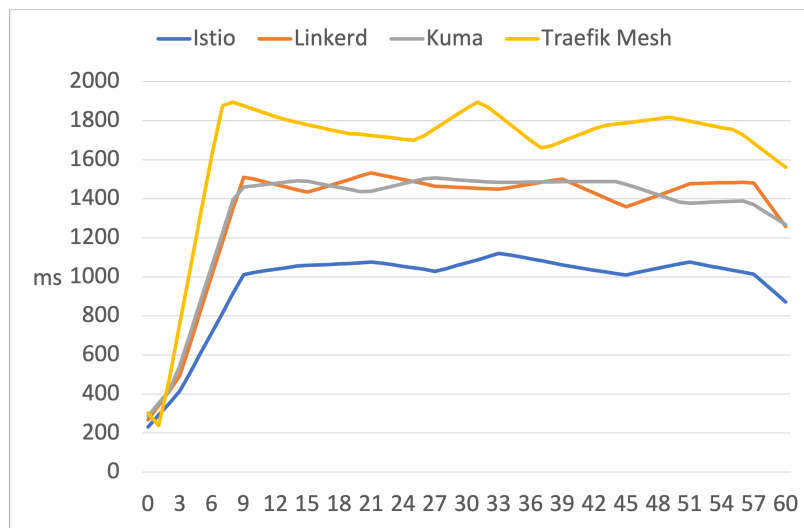


Figure 4.6: Test case 3 - Data plane - CPU usage - Application

4.2.3 Data Plane - Sidecar

Memory usage

For memory usage in Sidecar, the race to see who uses the least is very close at the top. In Test case 1 (Figure 4.7), Linkerd is just ahead of Traefik Mesh. In test case 2 (Figure 12) it is the other way round and Traefik Mesh uses less. From test case 3 (Figure 13) and test case 4 (Figure 14), Traefik Mesh is ahead. Kuma and Istio are always behind in all test cases as shown in Figure 4.7.

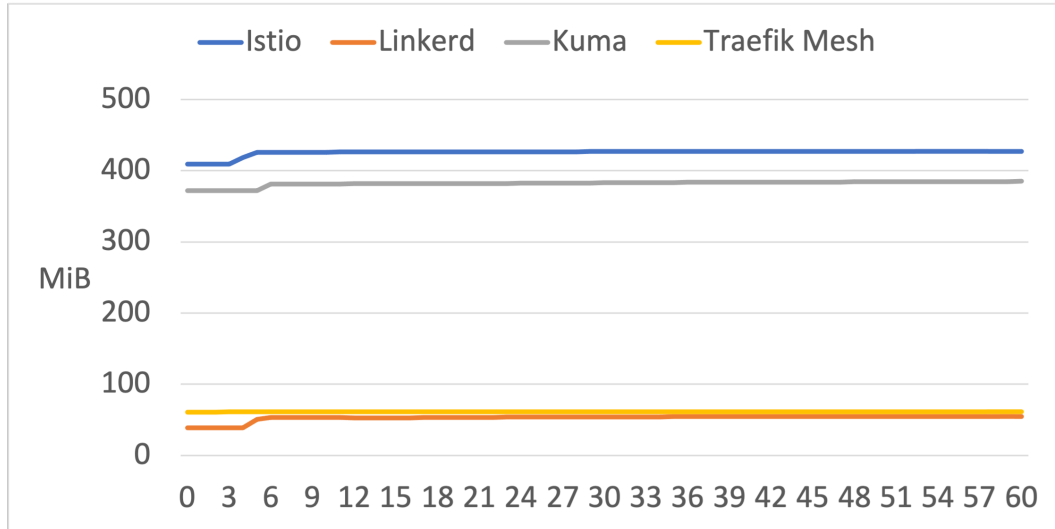


Figure 4.7: Test case 1 - Data plane - Memory usage - Sidecar

CPU usage

The CPU usage for the Sidecar for test case 1 is very clear. The curve as shown in Figure 4.8 is similar to test case 2 (Figure 15), test case 3 (Figure 16) and test case 4 (Figure 17). Traefik Mesh consumes almost no CPU, followed by Kuma, Linkerd, and Istio.

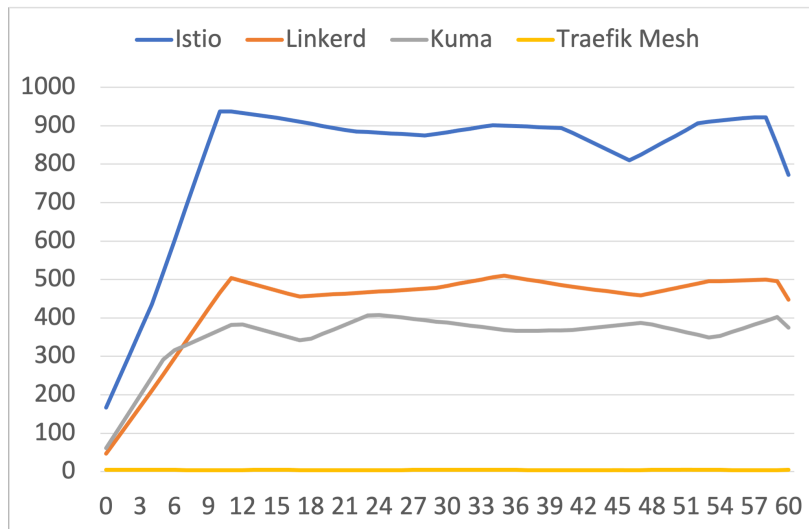


Figure 4.8: Test case 1 - Data plane - CPU usage - Sidecar

4.2.4 Requests per second

In addition to CPU and memory usage, throughput was also measured using RPS. The trend is clear for test case 1, test case 2, test case 3, and test case 4, as Figure 4.9 shows. The gap only increases when more virtual users are used. It should also be mentioned that test case 1, test case 2, and test case 3 had no failed requests. Only in test case 4 (Stress test) did errors occur, as Figure 4.14 shows.

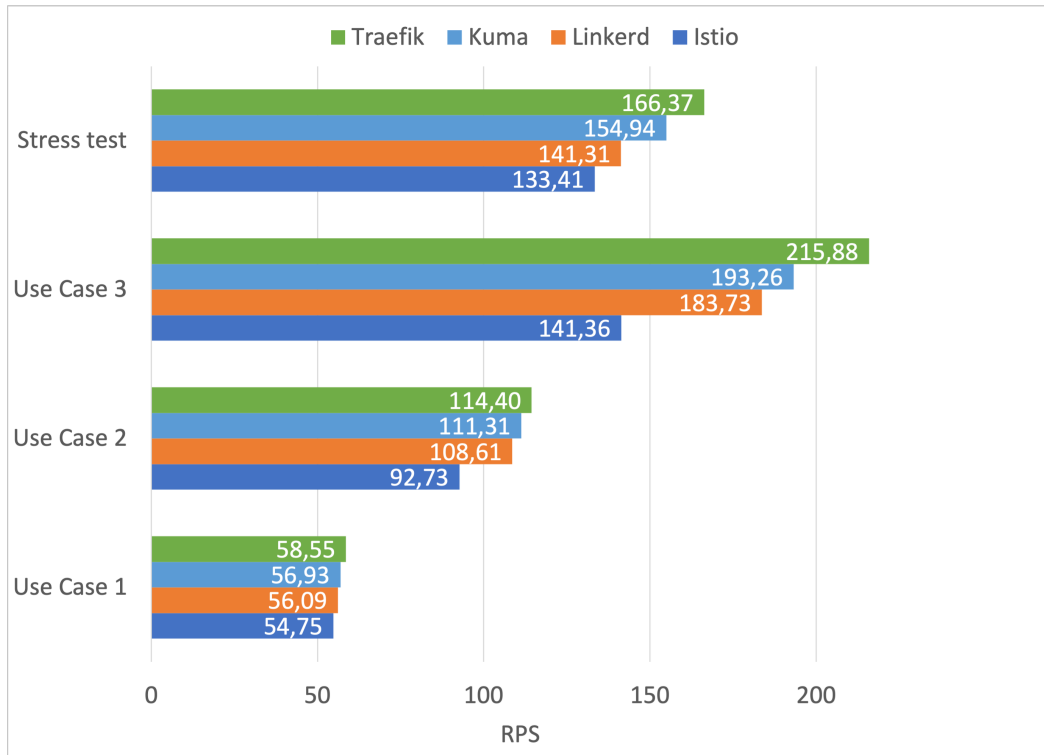


Figure 4.9: Test case 1,2,3 and 4 - Requests per second

4.2.5 GKE vs. HAProxy

Test case 1

As mentioned at the beginning of the chapters, the same tests were also carried out using a different combination of Service Mesh and Ingress. Figure 4.10 for test case 1 shows several things at once. On the one hand, the 90th percentile and the 95th percentile of the response times between the service meshes. Looking at the 90th percentile and 95th percentile by themselves, the trend is the same for both. Traefik Mesh has the shortest response time, followed by Kuma, Linkerd, and Istio. Besides this aspect, only the response times between the two different ingresses are compared. HAProxy is ahead of the GKE Ingress in terms of response times. There is only one exception, namely the red circle in Figure 4.10 for the Traefik Mesh. In test case 4 (Stress test), the GKE behaves better than the HAProxy with Traefik Mesh.

4 Discussion

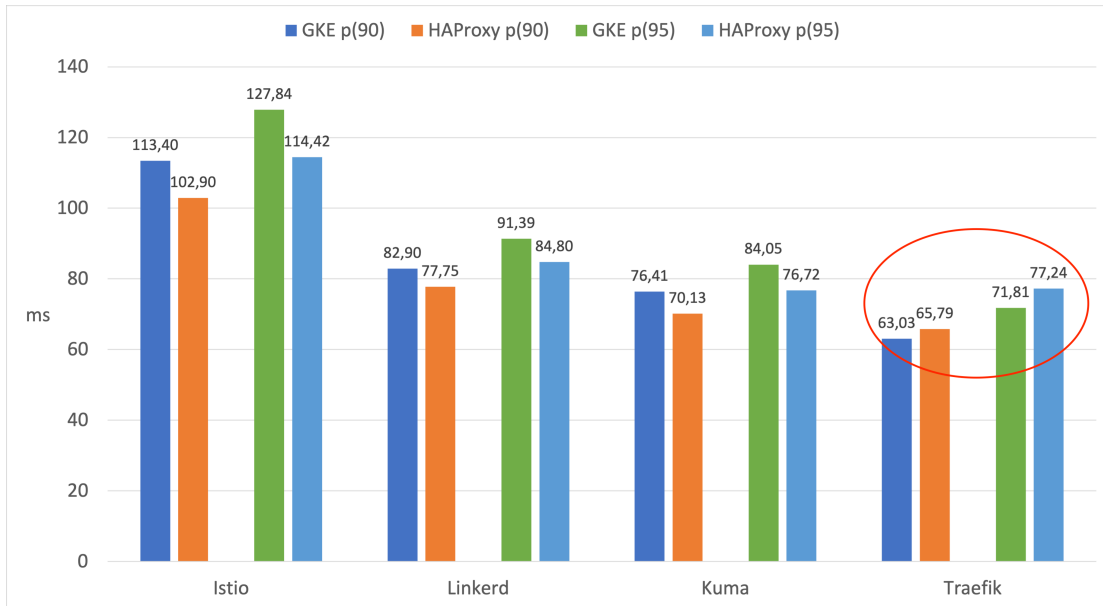


Figure 4.10: Test case 1 - GKE vs. HAProxy - Response time(P90 and P95)

Test case 2

Test case 2 behaves the same as the first Test Case, see Figure 4.11. The behavior of Traefik Mesh is also the same, namely the red circle in Figure 4.11. Only the response time between GKE and HAProxy for Istio and Linkerd is significantly higher when the number of virtual users is increased.

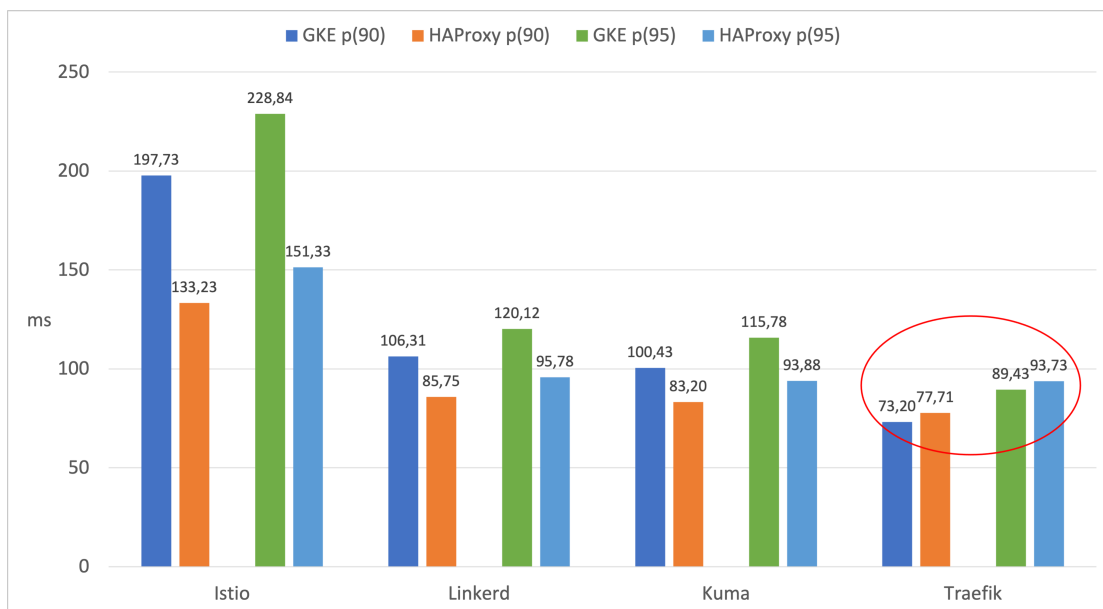


Figure 4.11: Test case 2 - GKE vs. HAProxy - Response time(P90 and P95)

Test case 3

From test case 3 onwards, the trend is generally the same. HAProxy is always faster than GKE even with Traefik Mesh. The order of response times remain unchanged between the service meshes, see Figure 4.12.

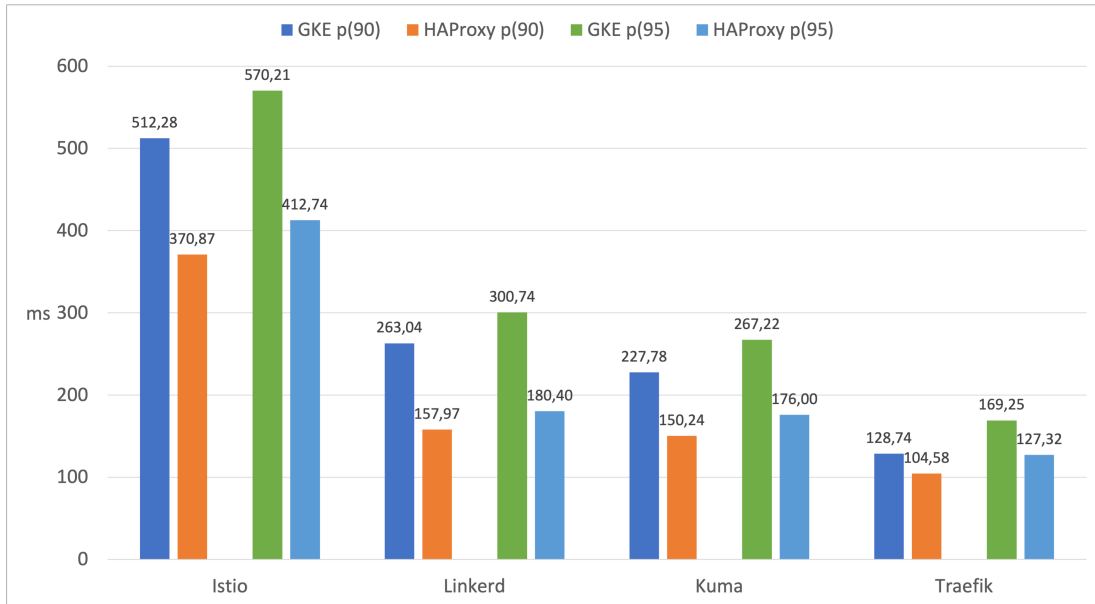


Figure 4.12: Test case 3 - GKE vs. HAProxy - Response time(P90 and P95)

Test case 4

Test case 4 shows a completely different picture than before. Here, the GKE is superior to the HAProxy.

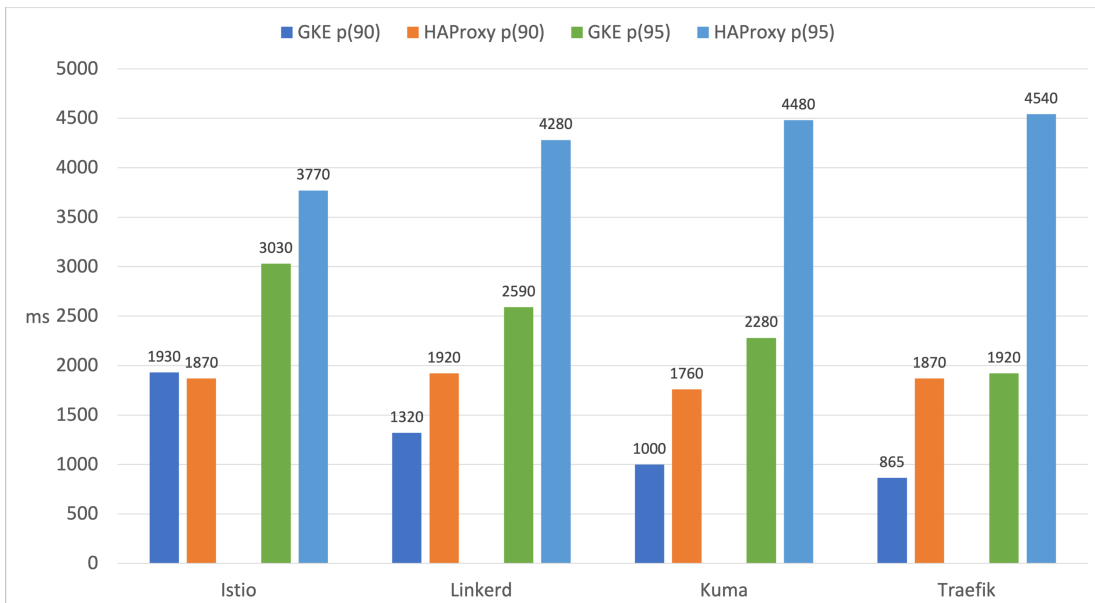


Figure 4.13: Test case 4 - GKE vs. HAProxy - Response time(P90 and P95)

Only in the p90 percentile with Istio is the trend different. Here the HAProxy is minimally faster than the GKE, see Figure 4.13. Figure 4.14 for the failed requests is only available for test case 4, as there were no failed requests for test case 1, test case 2, and test case 3 for both GKE and HAProxy. The results from Figure 4.14 also fit very well with the previous figure that more requests fail for HAProxy than for GKE in all test cases.

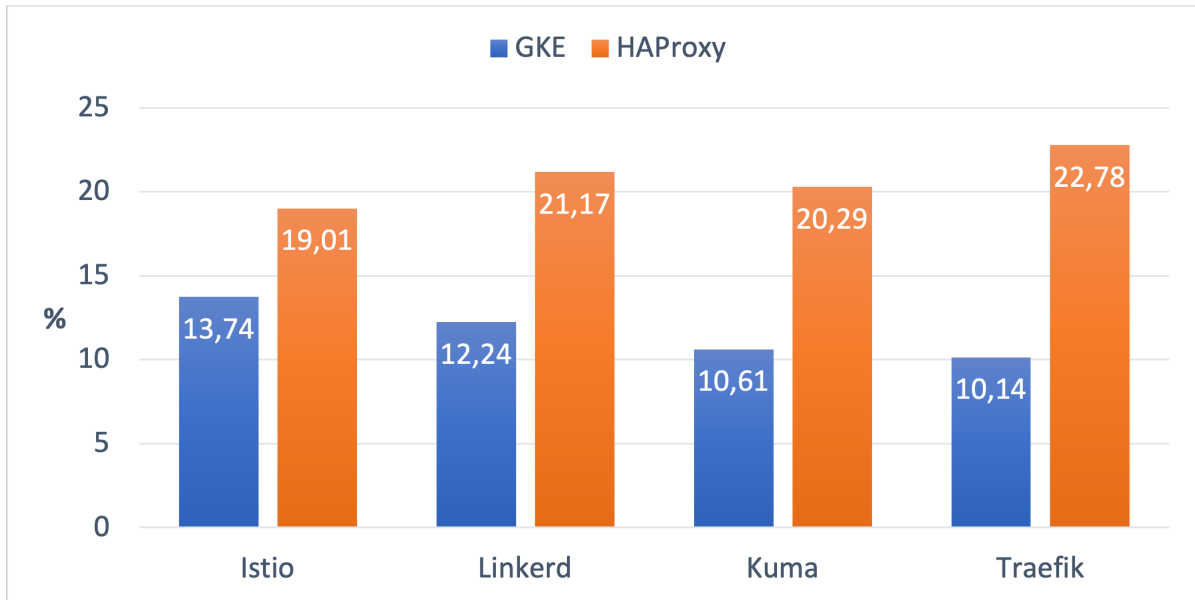


Figure 4.14: Test case 4 - GKE vs. HAProxy - Requests failed

4.3 Summary and Decision tree

4.3.1 Summary

In this chapter, the collected results are once again presented clearly in tabular form and as bar chart. The two tables below list the best and worst service meshes by category / use case. The best service meshes with the lowest memory consumption and CPU load are shown in Table 4.7. The worst service meshes with the highest memory consumption and CPU load are shown in Table 4.8. In the **Request failed - GKE | HAProxy** category, there is no data available for test cases 1, 2 and 3 because there were no errors. In certain categories, two service meshes will be awarded if the results were very close.

The results of the quantitative literature review for the three areas are shown in Figure 4.15. The ranking (the points of the three areas per service mesh summed up) by points is as follows:

- Istio - 15 points
- Linkerd - 12 points
- Kuma - 12 points
- Traefik mesh - 6 points

As you can see in figure 4.15, Kuma and Linkerd are evenly matched, but Kuma is better at security than Linkerd, and the reverse is true for traffic control.

	Test case 1	Test case 2	Test case 3	Test case 4
Control Plane - CPU usage	Traefik Mesh	Traefik Mesh	Traefik Mesh	Traefik Mesh
Data Plane - CPU usage - Sidecar	Traefik Mesh	Traefik Mesh	Traefik Mesh	Traefik Mesh
Data Plane CPU usage - Application	Istio	Istio	Istio	Istio
Control Plane - Memory usage	Traefik Mesh	Traefik Mesh	Traefik Mesh	Traefik Mesh
Data Plane - Memory usage - Sidecar	Linkerd	Traefik Mesh	Traefik Mesh	Traefik Mesh
Data Plane - Memory usage - Application	Linkerd	Linkerd	Istio	Istio
Request failed - GKE	Not available	Not available	Not available	Traefik Mesh
Request failed - HAProxy	Not available	Not available	Not available	Istio
Requests per second	Traefik Mesh	Traefik Mesh	Traefik Mesh	Traefik Mesh
90th percentile - GKE	Traefik Mesh	Traefik Mesh	Traefik Mesh	Traefik Mesh
90th percentile - HAProxy	Traefik Mesh	Traefik Mesh	Traefik Mesh	Kuma
95th percentile - GKE	Traefik Mesh	Traefik Mesh	Traefik Mesh	Traefik Mesh
95th percentile - HAProxy	Kuma	Kuma Traefik Mesh	Traefik Mesh	Istio

Table 4.7: Quantitative best results between service meshes for each test case

	Test case 1	Test case 2	Test case 3	Test case 4
Control Plane - CPU usage	Kuma	Kuma	Kuma	Kuma
Data Plane - CPU usage - Sidecar	Istio	Istio	Istio	Istio
Data Plane CPU usage - Application	Kuma	Traefik Mesh	Traefik Mesh	Traefik Mesh
Control Plane - Memory usage	Linkerd	Linkerd	Linkerd	Linkerd
Data Plane - Memory usage - Sidecar	Istio	Istio	Istio	Istio
Data Plane - Memory usage - Application	Istio	Kuma Traefik Mesh	Kuma Traefik Mesh	Kuma Traefik Mesh
Request failed - GKE	Not available	Not available	Not available	Istio
Request failed - HAProxy	Not available	Not available	Not available	Traefik Mesh
Requests per second	Istio	Istio	Istio	Istio
90th percentile - GKE	Istio	Istio	Istio	Istio
90th percentile - HAProxy	Istio	Istio	Istio	Linkerd
95th percentile - GKE	Istio	Istio	Istio	Istio
95th percentile - HAProxy	Istio	Istio	Istio	Traefik Mesh

Table 4.8: Quantitative worst results between service meshes for each test case

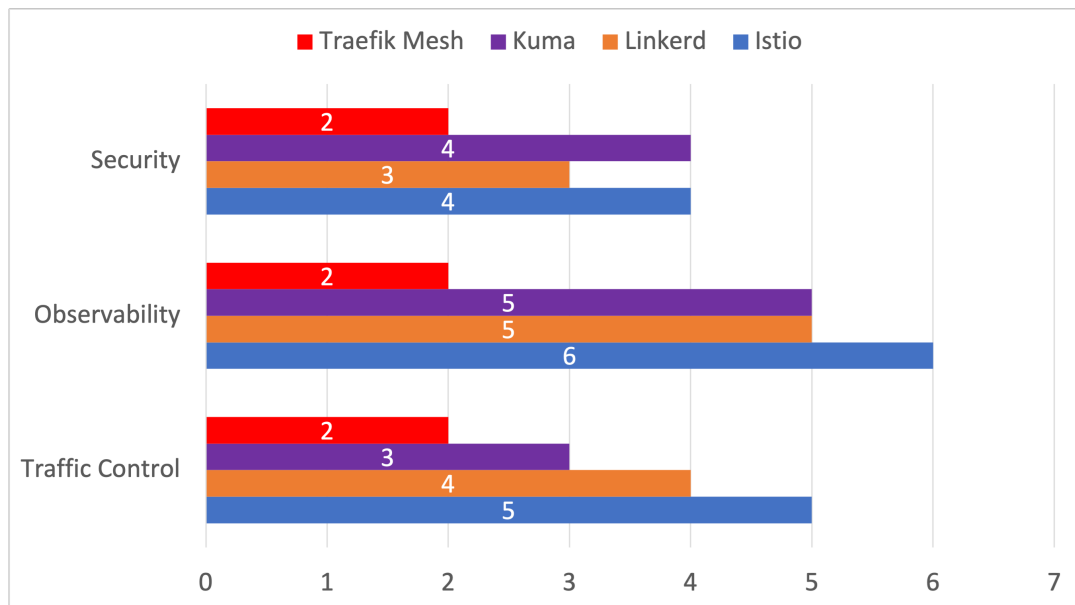


Figure 4.15: Results for Traffic Control, Observability and Security

4.3.2 Decision tree

A decision tree can be derived from the quantitative literature study and quantitative analysis of performance benchmarking for the Istio, Linkerd, Kuma, and Traefik Mesh as shown in Figure 4.16. As a note, there are many more service meshes than the ones mentioned before, so the decision tree is only derived from these. Importantly, the decision for or against a service mesh is based solely on the requirements for the MSA. A service mesh generally makes little sense among e.g. ten microservices because the effort exceeds the benefit. If it does, because management wants to implement a trend, Linkerd should be chosen because it is the easiest to adapt, as it is closely coupled with Kubernetes.

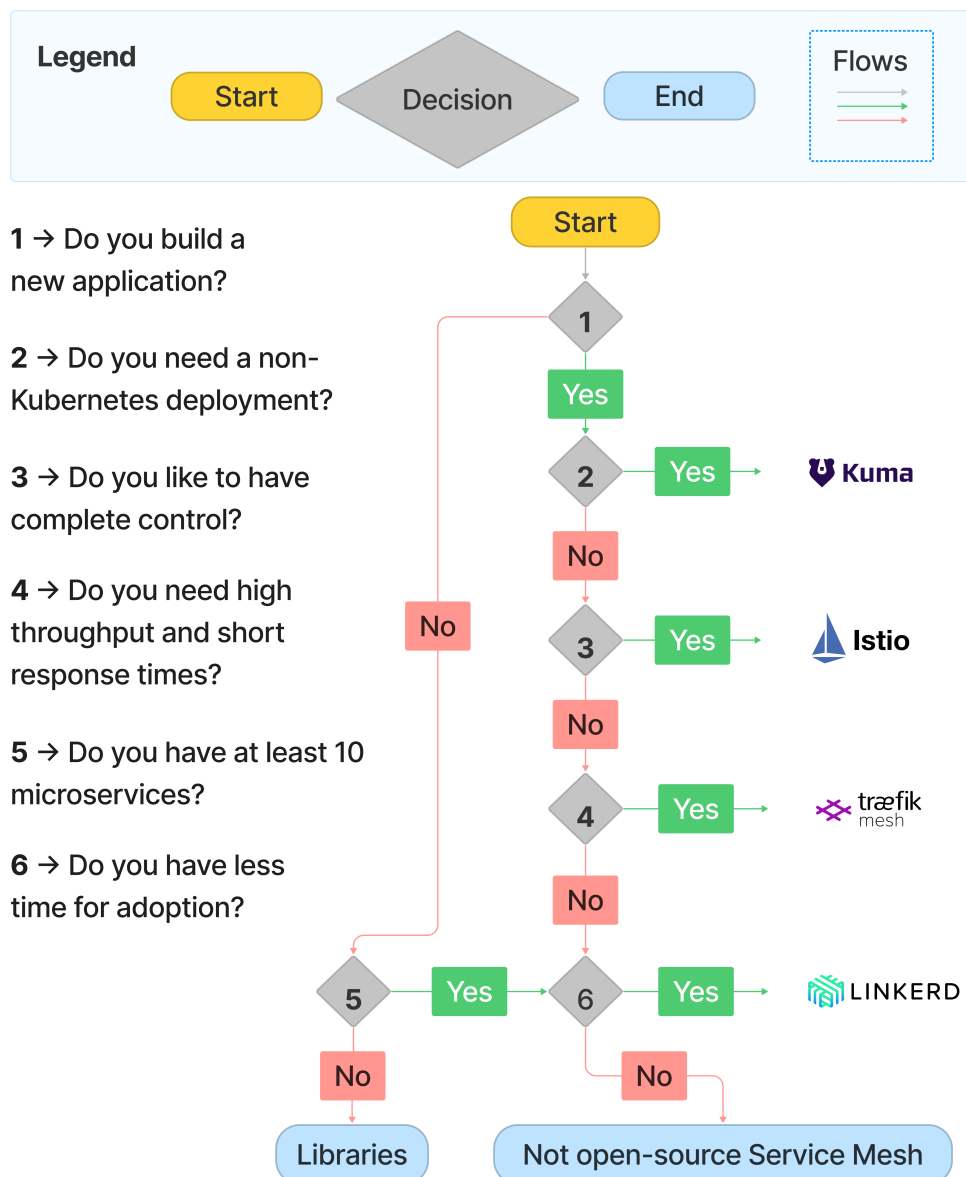


Figure 4.16: Decision tree for service mesh

5 Final Considerations

5.1 Summary

This thesis begins by showing how service meshes actually evolved from problems with microservices. Although there were already solutions in the form of libraries to ensure fault tolerance, this led to drawbacks low reusability of microservice artifacts and limited choice of programming languages. Since 2014, a large number of service meshes have appeared. On the face of it, they all promise the same features, such as improved reliability, observability, and security. Due to the similar features, it is difficult to find the right service mesh in practice. Aiming this problem, this thesis compares the four most popular open-source service meshes under load. Furthermore, there are no approaches how to improve the performance of a service mesh. Chapter 2 gives a detailed look at service meshes and background information about Kubernetes and microservices as well as terminology. Chapter 3 describes the different configurations for the service meshes that are necessary to run it in a hyperscaler like Google Kubernetes Engine. It also explains the test scenarios, how to create a cluster in the Cloud using IaC, and how package management and service meshes work in Kubernetes. In addition, the different types of high-level service mesh architecture are also explained. Furthermore, a proof of concept for the SMP specification is created using a k6 output extension. This solves the problem that there is more than one implementation of the specification that is independent of Meshery. Tools, languages and data collection of aggregated results are also explained. At the end of chapters 3, we look back at the lessons learned. In chapter 4, the quantitative literature study is assessed for traffic control, observability and security using a questionnaire of the selected service meshes. Istio fulfils most of the three criteria. Based on the evaluations, a decision tree results when which service mesh is most suitable or when it is better not to use a service mesh. Traefik Mesh performs best in the quantitative analysis of performance benchmarking. Therefore, the correlation that it performs worst in the quantitative literature study also fits well. It also shows that the performance of a service mesh can be improved by using a different ingress. Based on the evaluation, a decision tree that helps to select a service mesh for a microservice architecture.

5.2 Evaluation of the Objectives

In the following, the defined objectives and research questions from chapter 1.2 are considered as to whether or not they have been achieved:

At least Istio, Linkerd, Kuma and Traefik Mesh with the same microservice architecture in the Google Cloud Platform should be compared for their performance based on previously defined test cases. It is possible. All four service meshes mentioned were compared in the GCP using test cases. For more details, see chapter 4.2. Locally, Consul and Open Service Mesh were also tested, but due to a bug in Ingress in the cloud, which could not be solved, both were not included in the comparison.

Additionally, there is no guideline or best practices to optimize the tuning of service mesh features. [3]

A new best practice would be to change the ingress controller for the service mesh, which improves a non-functional property. In this work, it was shown that the HAProxy ingress is very well suited for performance in load tests compared to the GKE ingress. For resilience in the stress test, the GKE Ingress is better suited. Technical implementation is described in chapter 3.4.4. For more details on the results, see the 4.2.5 chapter. A new guideline on how to improve a service mesh could not be created because the scope was too broad. It would be possible to narrow down the choice to just one service mesh and focus on a non-functional requirement for the guideline.

How do Istio, Linkerd, Kuma, and Traefik Mesh differ in architecture? The architectures are only minimally different because they are all service meshes and therefore all apply the Sidecar pattern. Either Envoy is used as the service proxy or a separate implementation as with Linkerd and Traefik Mesh. Another difference is where the service proxy is deployed. With Istio, Linkerd, and Kuma as a Sidecar per application. Traefik Mesh deploys it as a Sidecar per Kubernetes node. For more details, see chapter 3.2.

Is the SMP specification also applicable to other load testing tools like k6? Yes - The proof of concept showed that the SMP can also be applied to a load test tool such as k6. See more in chapter 3.8

How do the service meshes differ in the areas of traffic control, observability, and security? Istio best implements the three areas mentioned above. Kuma and Linkerd are pretty much the same, with Kuma performing better in the area of security and Linkerd in the area of traffic control. Traefik Mesh fulfils all three categories the least. For more details, see chapter 4.1.

How do Istio, Linkerd, Kuma and Traefik Mesh differ in load testing? In the load test for response times at Test case 1,2 and 3 for 90 percentile and 95 percentiles at GKE Ingress, the following order (fastest to slowest) is obtained: Traefik Mesh, Kuma, Linkerd, and Istio. For HAProxy, the values differ depending on the P90 or P95 test case. For the 90 percentiles, the order is the same as above. For the 95 percentiles, the order for test case 1 and 2 are as follows (fastest to slowest): Kuma, Traefik Mesh, Linkerd, and Istio. For Test case 3, the order of the tests is again the same as for the GKE Ingress. See more in chapters 4.2.5, 4.2.5 or 4.2.5.

How do Istio, Linkerd, Kuma, and Traefik Mesh differ in stress testing? In the Stress Test, the order (fastest to slowest) for GKE (90 percentile) and HAProxy (95 percentile) Ingress is Traefik Mesh, Kuma, Linkerd, and Istio. The order (fastest to slowest) for the GKE Ingress 90 percentile is: Kuma, Linkerd is the same as Traefik Mesh and Istio. For the HAProxy 95 percentile order (fastest to slowest) is: Istio, Linkerd, Kuma, and Traefik Mesh. Find out more in chapter 4.2.5.

How resource intensive (CPU and RAM) are Istio, Linkerd, Kuma, and Traefik Mesh in comparison? In most test cases, Traefik Mesh consumes the least resources, whether CPU or RAM. However, there are two exceptions. Linkerd consumes the least RAM for the

data plane application and the sidecars. Istio consumes the least CPU in the Data Plane application. See more in chapters 4.2.1, 4.2.2 or 4.2.3.

Is it possible to improve the performance of the service mesh? Yes - it is possible. If the Ingress component is changed, a different non-functional requirement can be fulfilled depending on the Ingress. It was shown that the HAProxy ingress is very well suited for performance in load tests, but in terms of resilience, the Google Kubernetes Engine Ingress is better. See chapter 4.2.5.

5.3 Limitations and Reflection of Thesis

One limitation of this work was the operating expenses. No matter which hyperscaler we use, at the end of the day you have to pay for every request in the Cloud. With the Google Cloud Platform, there were also quotas. Quotas are used to limit the resources of a service, a protection for the user if the Kubernetes cluster in the GKE is forgotten. The quotas also limited the work to six vCPUs and 22.5 GB RAM, because the Compute Engine API would have to be increased by means of a business request, or if a service is used for longer, the quota increases automatically.

As already described in chapter 3.1, several service meshes were selected, but only Istio, Linkerd, Kuma, and Traefik Mesh were benchmarked. Originally, Consul and Open Service Mesh would also have been included because the configuration was also created for the local Kubernetes cluster. The configuration itself also worked via port forwarding in GKE, but unfortunately not with Ingress. What the error was could not be determined, but may be related to Google Kubernetes Engine, because the sample applications from both providers were only available for Azure Kubernetes Service and Elastic Kubernetes Service.

The NIST Special Publication - 800-204A was not ideal in retrospect because many of the SM-DR from which the criteria were derived were based on Istio references. But this only concerns the quantitative literature study.

5.4 Related Work

The paper [79] provides benchmark data for various Edge deployments using Bare Metal and virtual machine-based scenarios. It examines the architectural complexities of deploying a service mesh in an Edge environment, including performance implications for types of communication such as north-south and east-west communication, as well as in and out communication of a service mesh. The authors use a simple on-premise Kubernetes cluster with the service mesh Istio. The difference with the current work is that the benchmarking takes place on-premise and the authors have used only one service mesh, namely Istio.

The authors in [3] suggest that the adoption of service meshes, along with their integration with eBPF, represents the next significant trend for meeting most of the industry's networking needs. Their paper examines the challenges associated with this trend, reviews its current status, and discusses future opportunities in that realm. The difference to this thesis is that it is a pure literature comparison and it also deals with eBPF, which is not considered in this thesis, except briefly in chapter 5.5

The authors in [80] look at a service mesh as a governance framework perspective. The paper responds to this challenge by proposing an adaptive strategy approach based on split flow to achieve global load balancing. They also developed a plug-in named DTMA for Istio, a typical representative of Service Mesh, to implement this strategy. Their extensive experiments show that their algorithm can reduce response time by almost 50% while maintaining a stable load of nodes and instances, compared to Istio's native configuration. A common feature of this work is that performance improvement is also achieved. The difference, however, is that the authors of [80] explicitly developed an extension only for Istio. In the present work, the changed configuration through the ingress achieves an improvement for all service meshes.

In [81] the authors presents a proposal for a 5G Core network based on Service Mesh, utilizing the Cilium container network interface plugin to enhance network security and observability. When applied to the 5G Core network, it is also designed to improve the performance of a service mesh. The authors compare their proposed design with a standard design using qualitative characteristics. In contrast to the present work, Cilium was not considered here as a service mesh because, according to CNCF, it is a cloud-native network. We also compared a standard design with a modified design as part of this work.

The objective of [82] is to compare the performance of Istio and Linkerd for microservices latency applications. Their findings demonstrate that Linkerd outperforms Istio in terms of RAM usage efficiency and low latency for application frontend management. One difference is that in the present work, lesser-known service meshes such as Kuma or Traefik Mesh are also used for comparison. What is the same is that the result regarding RAM is similar to this work. The latency results differ in both papers.

5.5 Future Work

In this work, a MSA was chosen for the comparison of the service meshes, in which the individual services communicate synchronously. In [83] Chris Richardson defines this kind of communication as Remote Procedure Invocation (RPI) Pattern. On the one hand, another RPI technology such as REST, Apache Thrift, or GraphQL could be used instead of gRPC. But another pattern could also be used, namely the messaging pattern. Here, asynchronous communication between services would take place and a messaging technology such as Apache ActiveMQ or RabbitMQ would have to be used.

The service mesh is based on the Sidecar pattern see 2.3.2. However, there are also variants without a Sidecar, namely the sidecarless service mesh. Currently, there is no scientific paper on this. However, there is on the one hand the approach of Istio with the Ambient Mesh [84], but also the approach of Cilium [85] with the eBPF. Here, on the one hand, non-functional requirements such as performance could be compared between service meshes with Sidecar and Sidecar-less, but also advantages and disadvantages between the two approaches could be elicited.

Moreover, more than half of the currently available open-source meshes such as Istio, Consul, Kuma, and Open Service Mesh use Envoy as a service proxy. It may be possible to further improve the performance of a service mesh using another proxy such as the Linkerd2 proxy or the Traefik proxy. As of today, there is no quantitative comparison of service proxies like Envoy, Linkerd2-proxy, and Traefik Mesh.

In addition to the two Ingress controllers used, Google Compute Engine and HAProxy, there are many other implementations from different vendors. Here it could be evaluated which controller, improves the non-functional properties of a service mesh.

5.6 Conclusion

The work concludes with several findings. Firstly, the proof of concept demonstrated that SMP can be implemented in a load test tool. This provides an alternative to Meshery, which implements much more than the SMP and is not absolutely necessary for load testing.

It has also been proven that it is possible to improve the performance of a service mesh using a combination of different components such as the Ingress. This work has shown that the HAProxy Ingress improves the performance in load testing, but in resilience, the GKE Ingress is better suited.

Certain trends also emerge in the service meshes selected for this work. Traefik Mesh is the clear winner in terms of response times and throughput but does less well than the others in terms of properties such as security and observability. It is no coincidence that Traefik Mesh and Kuma are ahead in response times, as the manufacturers of both also develop API gateways in addition to service meshes and there are many overlaps here that have a positive effect on performance. Istio plays to its strengths with extensive configuration options and consumes little CPU and memory in the application data plane. Linkerd convinces with minimal memory consumption in the Sidecar proxy and its simplicity in configuration. Kuma achieves very good response times and, unlike the other service meshes, can also be used in non-Kubernetes environments. Furthermore, it must be mentioned that a service mesh was first used in Big Tech companies like Netflix or Uber, where over 100 microservices are in use. This will not always be the case in small and medium-sized companies and will not be necessary there. It should also be mentioned that besides the service meshes mentioned so far, there are several service meshes on the CNCF Landscape alone, but they are not open-source like AWS App Mesh Nginx Service Mesh or Alibaba Cloud Service Mesh. However, due to the limited time available for this work, a limitation had to be made as not every service mesh is the same.

It must be said, however, that a service mesh, no matter which one, is not a silver bullet for a MSA. Under certain aspects, libraries are the better solution, because the learning curve of Kubernetes and only one service mesh is very steep. Therefore, at the beginning of a project, it is necessary to evaluate exactly what is needed and what is not, based on the requirements. The decision tree introduced in this thesis is a good tool to make the right decision for a service mesh.

Bibliography

- [1] A. Koschel, M. Bertram, R. Bischof, K. Schulze, M. Schaaf, and I. Astrova, “A look at service meshes,” in *2021 12th International Conference on Information, Intelligence, Systems and Applications (IISA)*, 2021, pp. 1–8. 1, 13
- [2] W. Li, Y. Lemieux, J. Gao, Z. Zhao, and Y. Han, “Service mesh: Challenges, state of the art, and future research opportunities,” in *2019 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2019, pp. 122–1225. 1
- [3] M. R. S. Sedghpour and P. Townend, “Service mesh and ebpf-powered microservices: A survey and future directions,” in *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*, 2022, pp. 176–184. 2, 59, 60
- [4] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2003. 4, 5
- [5] G. Mohan, *Full Stack Testing*. O’Reilly Media, Inc., 2022. 5, 6
- [6] M. Fowler, *Domain-specific languages*. Pearson Education, 2010. 6
- [7] K. Morris, *Infrastructure as code*. O’Reilly Media, 2020. 6
- [8] M. Butcher, M. Farina, and J. Dolitsky, *Learning Helm*. O’Reilly Media, 2021. 7
- [9] D. Flanagan, *JavaScript: the definitive guide*. O’reilly, 2020. 7
- [10] M. Butcher and M. Farina, *Go in Practice*. Manning Publications Co., 2016. 7
- [11] S. Brown, *Software Architecture for Developers*. Leanpub, 2019, vol. Second. 7
- [12] S. Newman, *Building Microservices*, 2nd ed. O’Reilly Media, Inc., 2021. 8
- [13] M. Luksa, *Kubernetes in Action*. Manning Publications Co., 2018. 9
- [14] B. Ibryam and R. Huß, *Kubernetes Patterns*. O’Reilly Media, Inc., 2019. 9, 68
- [15] kubernetes.io. (2023) Containers. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/containers/> 9
- [16] —. (2023) Pods. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/pods/> 10
- [17] —. (2023) Service. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/service/> 10
- [18] —. (2023) Labels and selectors. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/> 10
- [19] —. (2023) Annotations. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/working-with-objects/annotations/> 11
- [20] —. (2023) Namespaces. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/> 11
- [21] —. (2023) Deployments. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/> 11, 12
- [22] —. (2023) Ingress. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/ingress/> 12

- [23] ——. (2023) Ingress controller. [Accessed 18 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/> 12
- [24] L. Calcote and Z. Butcher, *Istio: Up and running: Using a service mesh to connect, secure, control, and observe*. O'Reilly Media, Inc, 2019. 12, 15, 18, 68
- [25] L. Kysow, *Consul: Up and Running*. O'Reilly Media, Inc., 2022. 13, 68
- [26] K. Indrasiri and S. Suhothayan, *Design Patterns for Cloud Native Applications*. O'Reilly Media, Inc., 2021. 13, 14, 68
- [27] L. Calcote, *The Enterprise Path to Service Mesh Architectures*. O'Reilly Media, Inc, 2021. 16, 17
- [28] C. E. Posta and R. Maloku, *Istio in Action*. Manning Publications Co., 2022. 18
- [29] M. Authors. (2023) Overview | meshery. [Accessed 22 Feb 2023]. [Online]. Available: <https://docs.meshery.io/getting-started/overview> 19, 36
- [30] S. M. P. Authors. (2023) Standardizing service mesh value measurement. [Accessed 19 Feb 2023]. [Online]. Available: <https://smp-spec.io/> 19, 68
- [31] istio.io. (2023) Istio / architecture. [Accessed 15 Mar 2023]. [Online]. Available: <https://istio.io/latest/docs/ops/deployment/architecture/> 21, 68
- [32] N. C. Mendonça, C. Box, C. Manolache, and L. Ryan, “The monolith strikes back: Why istio migrated from microservices to a monolithic architecture,” *IEEE Software*, vol. 38, no. 05, pp. 17–22, 2021. 21
- [33] linkerd.io. (2023) Architecture | linkerd. [Accessed 15 Mar 2023]. [Online]. Available: <https://linkerd.io/2.12/reference/architecture/> 22, 68
- [34] W. Morgan. (2023) Why linkerd doesn't use envoy. [Accessed 16 Mar 2023]. [Online]. Available: <https://linkerd.io/2020/12/03/why-linkerd-doesnt-use-envoy/> 21
- [35] doc.traefik.io. (2023) Traefik mesh: Simpler service mesh. [Accessed 15 Mar 2023]. [Online]. Available: <https://doc.traefik.io/traefik-mesh/> 22, 23, 68
- [36] kuma.io. (2023) Overview | kuma. [Accessed 15 Mar 2023]. [Online]. Available: <https://kuma.io/docs/2.1.x/explore/overview/> 23, 68
- [37] G. C. Platform. (2023) Github repository: Online boutique. [Accessed 08 Mar 2023]. [Online]. Available: <https://github.com/GoogleCloudPlatform/microservices-demo> 24, 25, 68
- [38] developer.hashicorp.com. (2023) Provision a gke cluster. [Accessed 18 Mar 2023]. [Online]. Available: <https://developer.hashicorp.com/terraform/tutorials/kubernetes/gke> 29
- [39] istio.io. (2023) Istio / install with helm. [Accessed 29 Mar 2023]. [Online]. Available: <https://istio.io/latest/docs/setup/install/helm/> 31
- [40] linkerd.io. (2023) Installing linkerd with helm | linkerd. [Accessed 15 Mar 2023]. [Online]. Available: <https://linkerd.io/2.12/tasks/install-helm/> 31
- [41] kuma.io. (2023) Install kuma | kuma. [Accessed 15 Mar 2023]. [Online]. Available: <https://kuma.io/install/2.2.x/> 31
- [42] doc.traefik.io. (2023) Traefik mesh installation documentation. [Accessed 15 Mar 2023]. [Online]. Available: <https://doc.traefik.io/traefik-mesh/install/> 31
- [43] M. Schneider. (2023) Repository for the master thesis - performance benchmarking of service meshes. [Accessed 20 Mar 2023]. [Online]. Available: <https://github.com/schneidermichael/master-thesis-project> 31

- [44] G. Labs. (2023) Output extensions. [Accessed 23 Mar 2023]. [Online]. Available: <https://k6.io/docs/extensions/get-started/create/output-extensions/> 37
- [45] Google. (2023) output package. [Accessed 23 Mar 2023]. [Online]. Available: <https://pkg.go.dev/go.k6.io/k6/output> 37
- [46] kubernetes.io. (2023) Troubleshooting applications | kubernetes. [Accessed 21 Mar 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/> 39
- [47] M. Inc. (2023) Lens | the kubernetes ide. [Accessed 11 April 2023]. [Online]. Available: <https://k8slens.dev/> 39
- [48] Kubeshark. (2023) Kubeshark - the api traffic analyzer for kubernetes. [Accessed 11 April 2023]. [Online]. Available: <https://kubeshark.co/> 39
- [49] S. M. I. Authors. (2023) Smi | a standard interface for service meshes on kubernetes. [Accessed 21 Feb 2023]. [Online]. Available: <https://smi-spec.io/> 39
- [50] Google. (2023) Standard library - go packages. [Accessed 24 Mar 2023]. [Online]. Available: <https://pkg.go.dev/std> 41
- [51] R. Chandramouli, Z. Butcher *et al.*, “Building secure microservices-based applications using service-mesh architecture,” *NIST Special Publication*, vol. 800, p. 204A, 2020. 42, 43, 44, 70
- [52] istio.io. (2023) Istio / protocol selection. [Accessed 28 Feb 2023]. [Online]. Available: <https://istio.io/latest/docs/ops/configuration/traffic-management/protocol-selection/> 44
- [53] —. (2023) Istio / enabling rate limits using envoy. [Accessed 28 Feb 2023]. [Online]. Available: <https://istio.io/latest/docs/tasks/policy-enforcement/rate-limit/> 44
- [54] www.envoyproxy.io. (2023) grpc-json transcoder. [Accessed 28 Feb 2023]. [Online]. Available: https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/grpc_json_transcoder_filter 44
- [55] istio.io. (2023) Istio / authentication policy. [Accessed 28 Feb 2023]. [Online]. Available: <https://istio.io/latest/docs/tasks/security/authentication/authn-policy/> 44
- [56] —. (2023) Istio / envoy access logs. [Accessed 28 Feb 2023]. [Online]. Available: <https://istio.io/latest/docs/tasks/observability/logs/access-log/> 44
- [57] —. (2023) Istio / observability. [Accessed 28 Feb 2023]. [Online]. Available: <https://istio.io/latest/docs/concepts/observability/> 44
- [58] kubernetes.io. (2023) Secrets. [Accessed 28 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/concepts/configuration/secret/> 44, 45, 46, 72
- [59] —. (2023) Configure liveness, readiness and startup probes. [Accessed 28 Feb 2023]. [Online]. Available: <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/> 44, 45, 46, 72
- [60] —. (2023) Istio / virtual service. [Accessed 28 Feb 2023]. [Online]. Available: <https://istio.io/latest/docs/reference/config/networking/virtual-service/> 44
- [61] linkerd.io. (2023) Authorization policy | linkerd. [Accessed 28 Feb 2023]. [Online]. Available: <https://linkerd.io/2.12/reference/authorization-policy/> 45
- [62] —. (2023) Automatic mtls | linkerd. [Accessed 28 Feb 2023]. [Online]. Available: <https://linkerd.io/2.12/features/automatic-mtls/> 45
- [63] —. (2023) Http access logging | linkerd. [Accessed 28 Feb 2023]. [Online]. Available: <https://linkerd.io/2.12/features/access-logging/> 45

- [64] ——. (2023) Telemetry and monitoring | linkerd. [Accessed 28 Feb 2023]. [Online]. Available: <https://linkerd.io/2.12/features/telemetry/> 45
- [65] ——. (2023) Dashboard and on-cluster metrics stack | linkerd. [Accessed 28 Feb 2023]. [Online]. Available: <https://linkerd.io/2.12/features/dashboard/> 45
- [66] ——. (2023) Distributed tracing | linkerd. [Accessed 28 Feb 2023]. [Online]. Available: <https://linkerd.io/2.12/features/distributed-tracing/> 45
- [67] kuma.io. (2023) Data plane on universal | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/explore/dpp-on-universal/> 45
- [68] ——. (2023) Rate limit | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/policies/rate-limit/> 45
- [69] ——. (2023) Protocol support in kuma | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/policies/protocol-support-in-kuma/> 45
- [70] ——. (2023) Mutual tls | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/policies/mutual-tls/> 45
- [71] ——. (2023) Traffic log | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/policies/traffic-log/> 45
- [72] ——. (2023) Traffic metrics | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/policies/traffic-metrics/> 45
- [73] ——. (2023) Traffic trace | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/policies/traffic-trace/> 45
- [74] ——. (2023) Health check | kuma. [Accessed 28 Feb 2023]. [Online]. Available: <https://kuma.io/docs/2.0.x/policies/health-check/> 45
- [75] doc.traefik.io. (2023) Traefik mesh configuration. [Accessed 28 Feb 2023]. [Online]. Available: <https://doc.traefik.io/traefik-mesh/configuration/> 46
- [76] ——. (2023) Traefik logs documentation. [Accessed 28 Feb 2023]. [Online]. Available: <https://doc.traefik.io/traefik/observability/logs/> 46
- [77] ——. (2023) Traefik helm chart documentation. [Accessed 28 Feb 2023]. [Online]. Available: <https://doc.traefik.io/traefik-mesh/migration/helm-chart/> 46
- [78] ——. (2023) Traefik headers documentation. [Accessed 28 Feb 2023]. [Online]. Available: <https://doc.traefik.io/traefik/middlewares/http/headers/> 46
- [79] M. Ganguli, S. Ranganath, S. Ravisundar, A. Layek, D. Ilangoan, and E. Verplanke, “Challenges and opportunities in performance benchmarking of service mesh for the edge,” in *2021 IEEE International Conference on Edge Computing (EDGE)*, 2021, pp. 78–85. 60
- [80] J. Zhou, X. Li, Q. Wang, X. Qin, W. Miao, and J. Tian, “Balancing load: An adaptive traffic management scheme for microservices,” in *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)*, 2023, pp. 641–648. 61
- [81] V.-B. Duong and Y. Kim, “A design of service mesh based 5g core network using cilium,” in *2023 International Conference on Information Networking (ICOIN)*, 2023, pp. 25–28. 61
- [82] F. Adinegoro, C. Rahmania, I. N. Zaini, R. M. Negara, and S. N. Hertiana, “Latency and ram usage comparison of advanced and lightweight service mesh,” in *2022 5th International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*, 2022, pp. 369–372. 61

- [83] C. Richardson, *Microservices patterns: with examples in Java*. Simon and Schuster, 2018. 61
- [84] istio.io. (2023) Istio / introducing ambient mesh. [Accessed 29 Mar 2023]. [Online]. Available: <https://istio.io/latest/blog/2022/introducing-ambient-mesh/> 61
- [85] C. Authors. (2023) ebpf-based networking, security, and observability. [Accessed 29 Mar 2023]. [Online]. Available: <https://cilium.io/> 61
- [86] developer.hashicorp.com. (2023) Service intentions configuration entry. [Accessed 28 Feb 2023]. [Online]. Available: <https://developer.hashicorp.com/consul/docs/connect/config-entries/service-intentions> 72
- [87] ——. (2023) Secure consul agent communication with tls encryption. [Accessed 28 Feb 2023]. [Online]. Available: <https://developer.hashicorp.com/consul/tutorials/security/tls-encryption-secure> 72
- [88] ——. (2023) Configure metrics for consul on kubernetes. [Accessed 28 Feb 2023]. [Online]. Available: <https://developer.hashicorp.com/consul/docs/k8s/connect/observability/metrics> 72
- [89] ——. (2023) Distributed tracing. [Accessed 28 Feb 2023]. [Online]. Available: <https://developer.hashicorp.com/consul/docs/connect/distributed-tracing> 72

List of Figures

2.1	Overview of Kubernetes resources[14]	9
2.2	Service mesh is situated at the first infrastructure layer[25]	13
2.3	Interservice communication over Sidecar[26]	14
2.4	Service Architecture with service proxies[24]	15
2.5	Service Architecture with client libraries [24]	18
2.6	Snippet of the SMP [30]	19
3.1	Istio Architecture [31]	21
3.2	Linkerd Architecture [33]	22
3.3	Traefik Mesh Architecture [35]	22
3.4	Kuma Architecture [36]	23
3.5	Flowchart of Test scenario	24
3.6	Microservice architecture based on [37] illustrated as C4 model	25
3.7	Deployment on GKE	31
3.8	Example of <code>stdout</code> from test case 1 with Istio	35
3.9	Self-built GCP Dashboard for Istio	36
4.1	Comparison of quantitative literature study result	46
4.2	Test case 1 - Control plane - Memory usage	47
4.3	Test case 1 - Control plane - CPU usage	48
4.4	Test case 1 - Data plane - Memory usage - Application	48
4.5	Test case 3 - Data plane - Memory usage - Application	49
4.6	Test case 3 - Data plane - CPU usage - Application	49
4.7	Test case 1 - Data plane - Memory usage - Sidecar	50
4.8	Test case 1 - Data plane - CPU usage - Sidecar	50
4.9	Test case 1,2,3 and 4 - Requests per second	51
4.10	Test case 1 - GKE vs. HAProxy - Response time(P90 and P95)	52
4.11	Test case 2 - GKE vs. HAProxy - Response time(P90 and P95)	52
4.12	Test case 3 - GKE vs. HAProxy - Response time(P90 and P95)	53
4.13	Test case 4 - GKE vs. HAProxy - Response time(P90 and P95)	53
4.14	Test case 4 - GKE vs. HAProxy - Requests failed	54
4.15	Results for Traffic Control, Observability and Security	56
4.16	Decision tree for service mesh	57
1	Test case 2 - Control plane - Memory usage	72
2	Test case 3 - Control plane - Memory usage	73
3	Test case 4 - Control plane - Memory usage	73
4	Test case 2 - Control plane - CPU usage	73
5	Test case 3 - Control plane - CPU usage	74
6	Test case 4 - Control plane - CPU usage	74
7	Test case 2 - Data plane - Memory usage - Application	74
8	Test case 4 - Data plane - Memory usage - Application	75
9	Test case 1 - Data plane - CPU usage - Application	75

List of Figures

10	Test case 2 - Data plane - CPU usage - Application	75
11	Test case 4 - Data plane - CPU usage - Application	76
12	Test case 2 - Data plane - Memory usage - Sidecar	76
13	Test case 3 - Data plane - Memory usage - Sidecar	76
14	Test case 4 - Data plane - Memory usage - Sidecar	77
15	Test case 2 - Data plane - CPU usage - Sidecar	77
16	Test case 3 - Data plane - CPU usage - Sidecar	77
17	Test case 4 - Data plane - CPU usage - Sidecar	78

List of Tables

3.1	Overview of Service Mesh selection	20
3.2	Overview of Load test tool selection	37
4.1	Assessment criteria	42
4.2	Derived criteria catalog from [51]	42
4.2	Derived criteria catalog from [51]	43
4.2	Derived criteria catalog from [51]	44
4.3	Quantitative Assessment for Istio	44
4.4	Quantitative Assessment for Linkerd	45
4.5	Quantitative Assessment for Kuma	45
4.6	Quantitative Assessment for Traefik Mesh	46
4.7	Quantitative best results between service meshes for each test case	55
4.8	Quantitative worst results between service meshes for each test case	55
1	Quantitative Assessment for Consul	72

List of Listings

3.1	Deployment resource of email-service	26
3.2	Service resource of email-service	27
3.3	Automatic Sidecar injection in Istio	27
3.4	Automatic Sidecar injection in Linkerd	28
3.5	Automatic Sidecar injection in Kuma	28
3.6	Ingress resource for Service Meshes with GKE	29
3.7	Ingress resource for Service Meshes with HAProxy	30
3.8	Sidecar configuration in microservices-values.yaml	32
3.9	Ingress configuration in microservices-values.yaml	33
3.10	Command to run k6 with test case 1	35
3.11	Excerpt from the main.go	41
.1	Script for test case 1	79

Appendix

Criterion number	Verifiability through	Assessment
1	[86]	+
2	Not found	0
3	Not found	0
4	[87]	+
5	Not found	0
6	Not found	0
7	Not found	0
8	[88]	+
9	[89]	+
10	[58]	+
11	[59]	+
12	Not found	0

Table 1: Quantitative Assessment for Consul

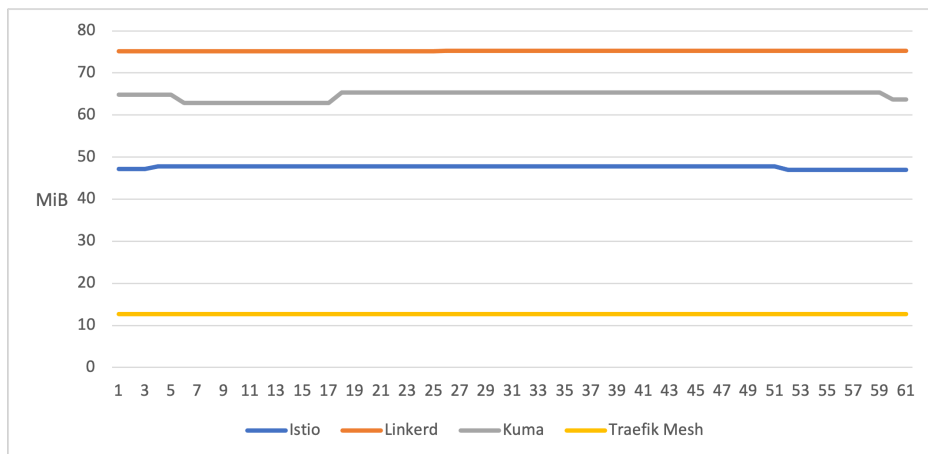


Figure 1: Test case 2 - Control plane - Memory usage

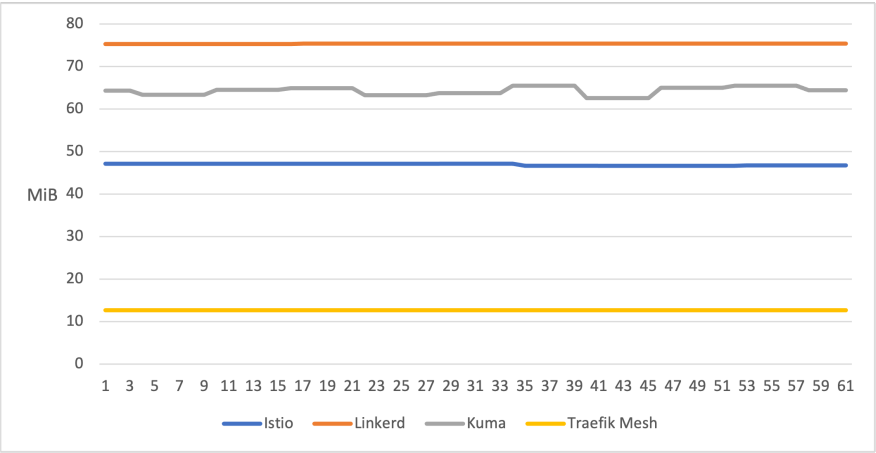


Figure 2: Test case 3 - Control plane - Memory usage

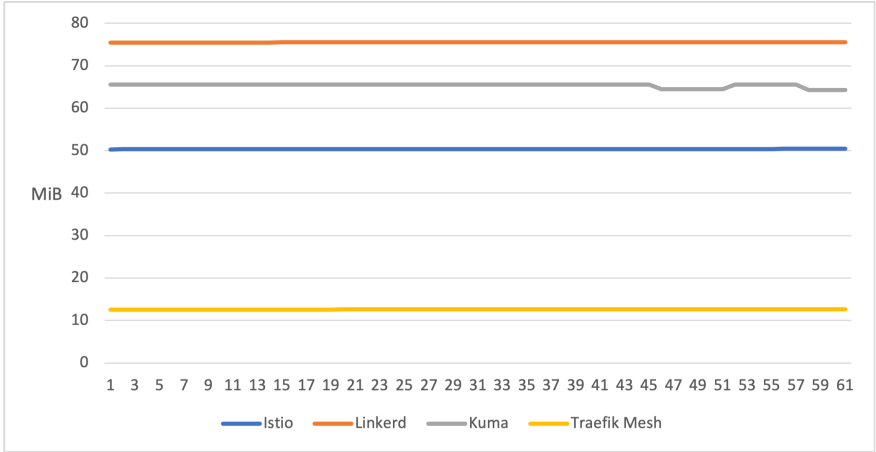


Figure 3: Test case 4 - Control plane - Memory usage

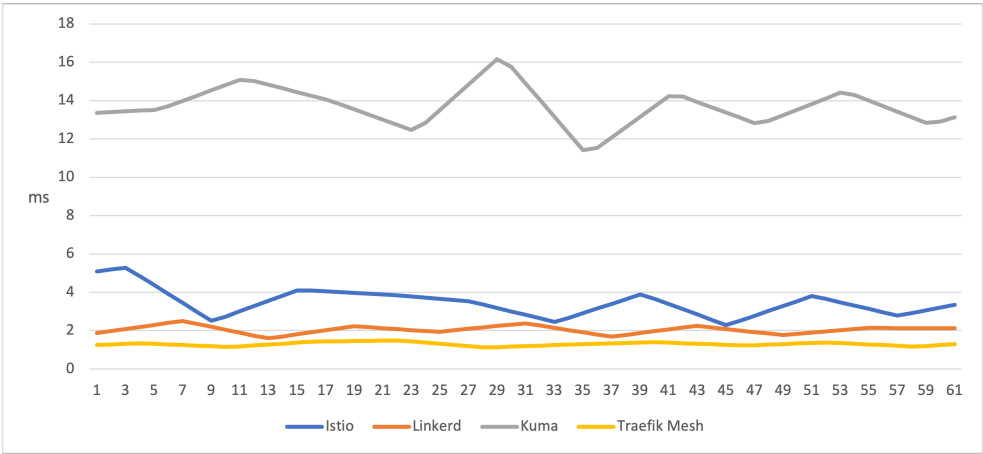


Figure 4: Test case 2 - Control plane - CPU usage

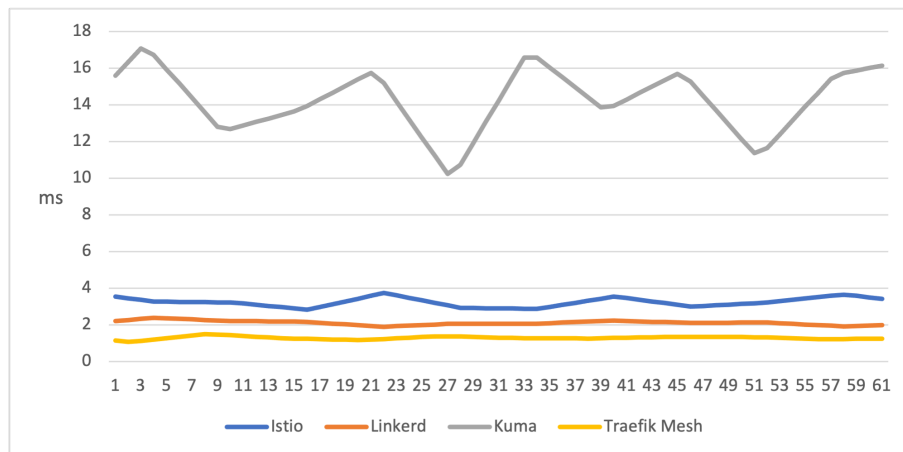


Figure 5: Test case 3 - Control plane - CPU usage

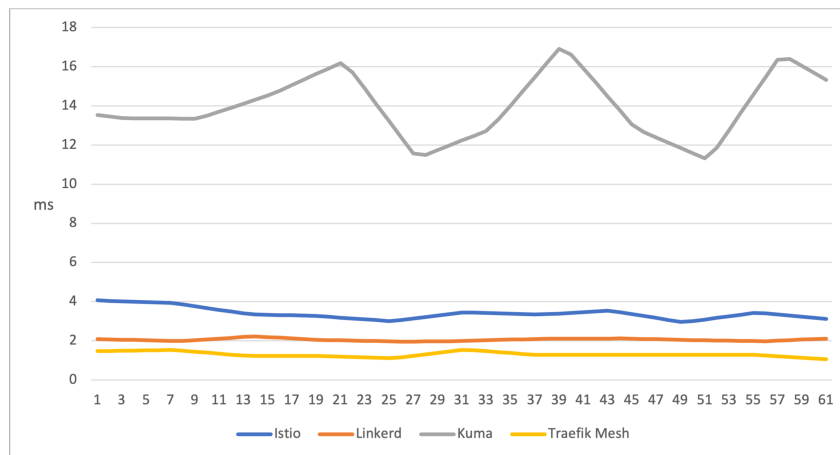


Figure 6: Test case 4 - Control plane - CPU usage

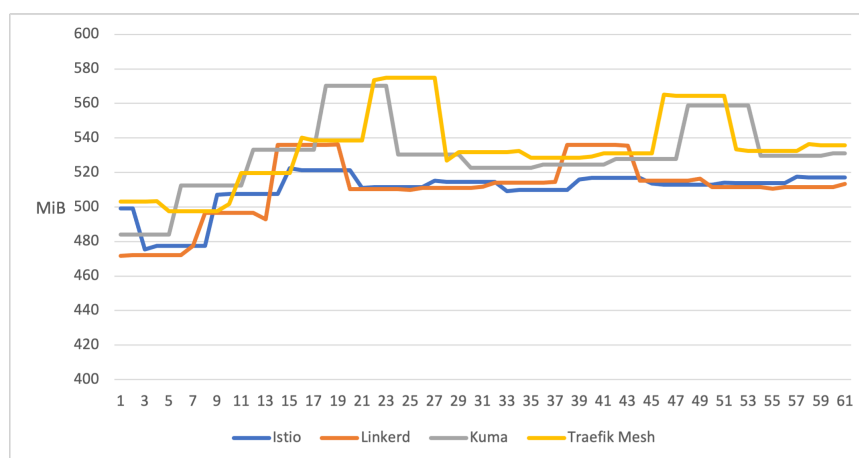


Figure 7: Test case 2 - Data plane - Memory usage - Application

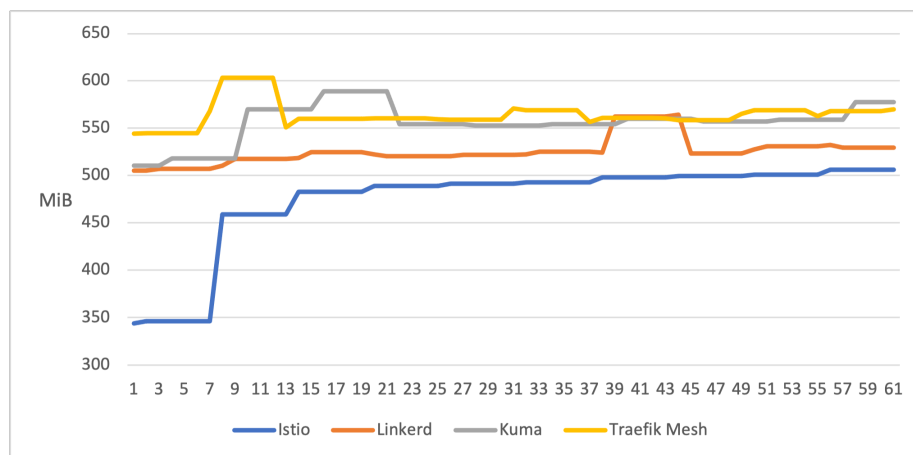


Figure 8: Test case 4 - Data plane - Memory usage - Application

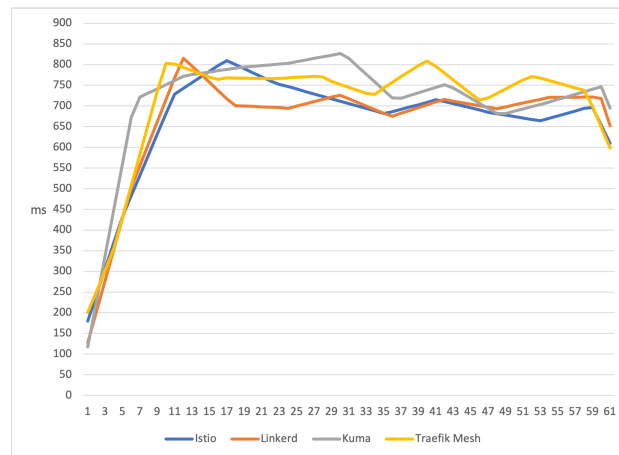


Figure 9: Test case 1 - Data plane - CPU usage - Application

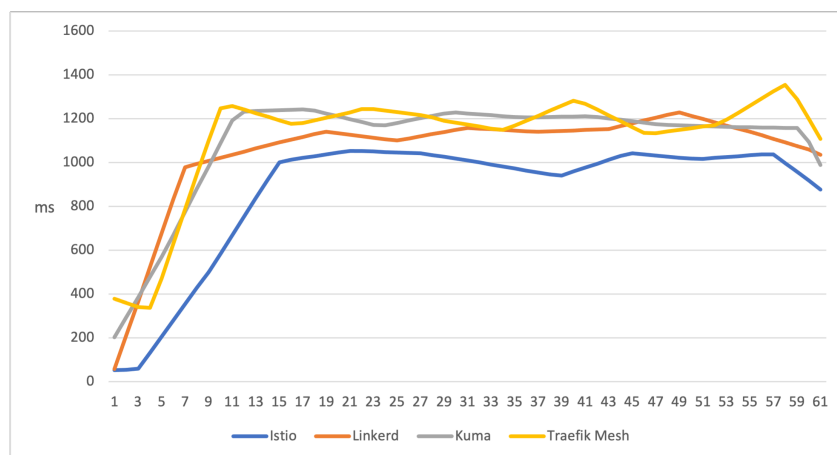


Figure 10: Test case 2 - Data plane - CPU usage - Application

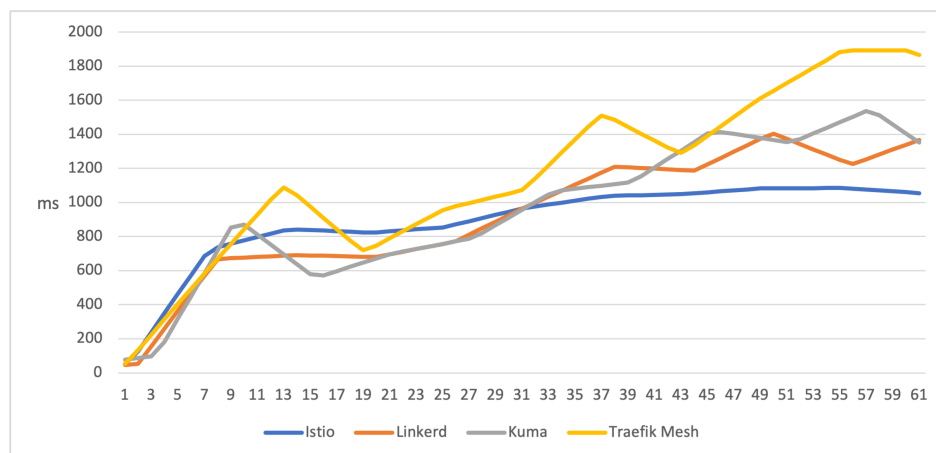


Figure 11: Test case 4 - Data plane - CPU usage - Application

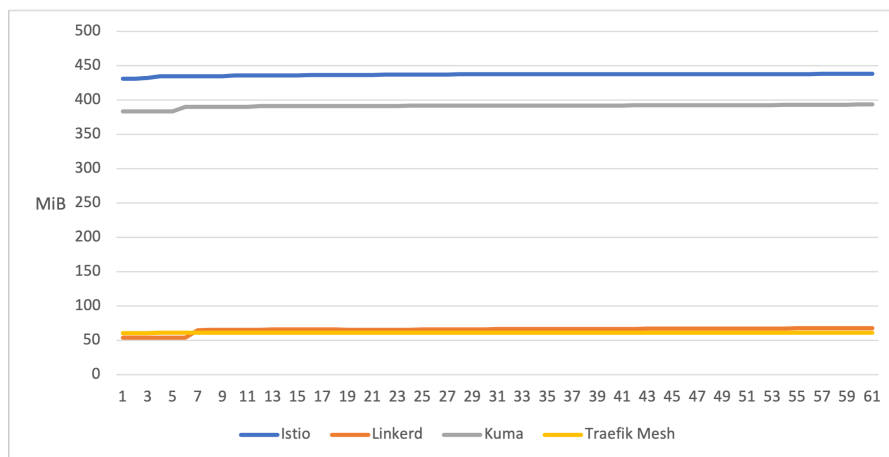


Figure 12: Test case 2 - Data plane - Memory usage - Sidecar

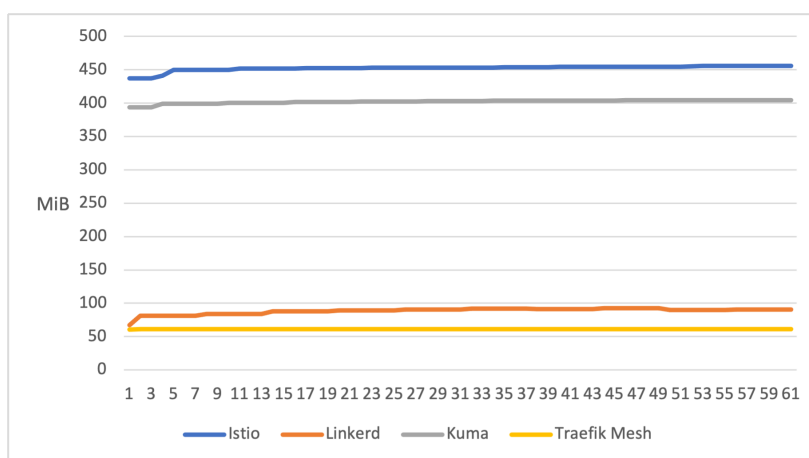


Figure 13: Test case 3 - Data plane - Memory usage - Sidecar

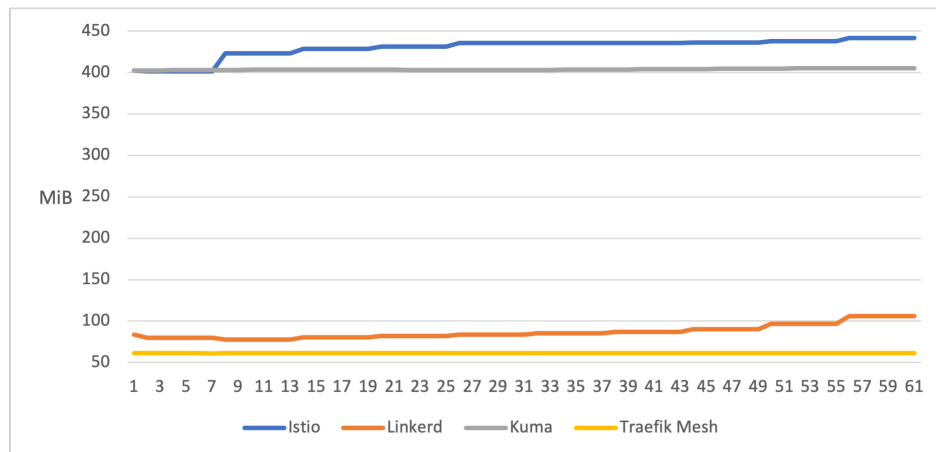


Figure 14: Test case 4 - Data plane - Memory usage - Sidecar

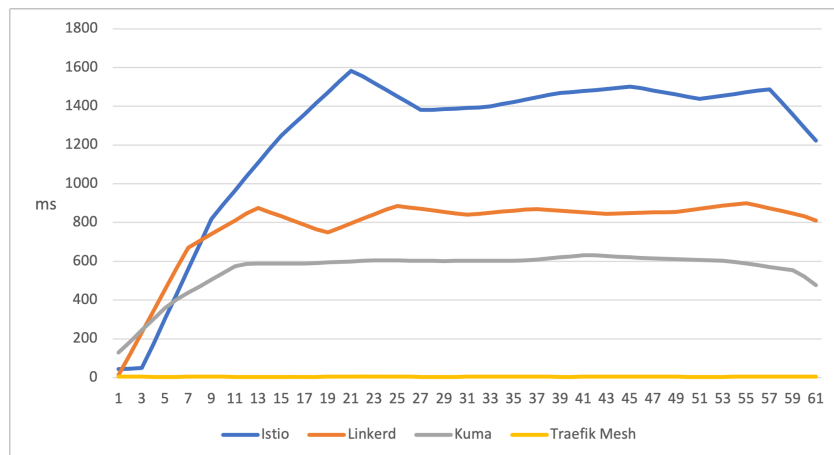


Figure 15: Test case 2 - Data plane - CPU usage - Sidecar

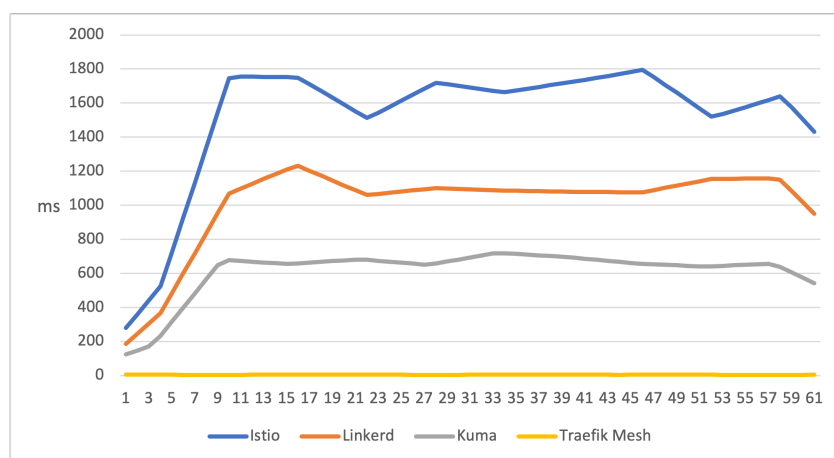


Figure 16: Test case 3 - Data plane - CPU usage - Sidecar

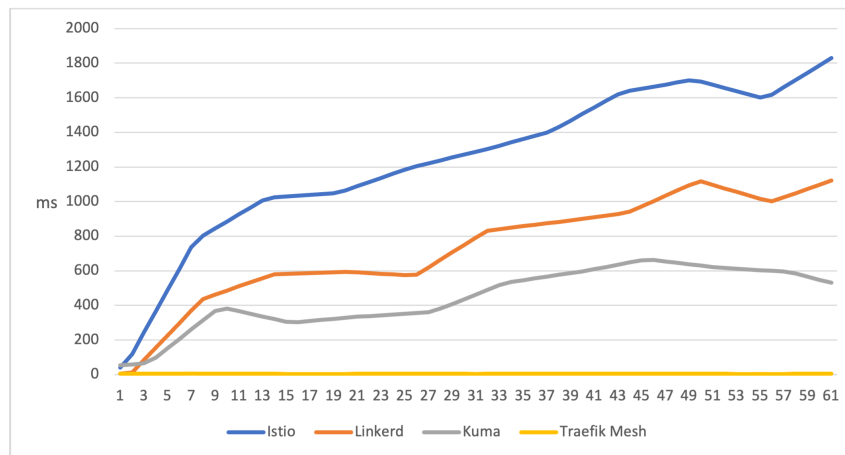


Figure 17: Test case 4 - Data plane - CPU usage - Sidecar

```

1  import http from "k6/http";
2  import { sleep } from "k6";
3
4  export const options = {
5    vus: 25,
6    duration: "600s",
7  };
8
9  const BASE_URL = "http://34.91.85.231";
10
11  const products = [
12    "OPUK6V6EV0",
13    "1YMWWN1N4O",
14    "2ZYFJ3GM2N",
15    "66VCHSJNUP",
16    "6E92ZMYZFZ",
17    "9SIQT8TOJO",
18    "L9ECAV7KIM",
19    "LS4PSXUNUM",
20    "OLJCESPC7Z",
21  ];
22
23  const currencies = ["EUR", "USD", "JPY", "GBP", "TRY", "CAD"];
24
25  export default function () {
26    index();
27    setCurrency();
28    browseProduct();
29    addToCart();
30    viewCart();
31    checkout();
32    sleep(randomise(0,10));
33  }
34
35  function index() {
36    http.get(BASE_URL);
37  }
38
39  function setCurrency() {
40    let data = { currency_code: currencies[randomise(0,3)] };
41    http.post(BASE_URL + "/setCurrency", JSON.stringify(data));
42  }
43
44  function browseProduct() {
45    http.get(BASE_URL + "/product/" + products[randomise(0,8)]);
46  }
47
48  function addToCart() {
49    let product = products[randomise(0,8)];
50    let data = { product_id: product, quantity: randomise(1,5) };
51    http.get(BASE_URL + "/product/" + product);
52    http.post(BASE_URL + "/cart", data);
53  }
54
55  function viewCart() {
56    http.get(BASE_URL + "/cart");
57  }
58
59  function checkout() {
60    addToCart();
61    let data = {
62      email: "master@stud.fh-campuswien.ac.at",
63      street_address: "4498-4400 Cheyenne Ave",
64      zip_code: "52806",
65      city: "Davenport",
66      state: "IA",
67      country: "United States",
68      credit_card_number: "3332-7015-2132-9454",
69      credit_card_expiration_month: "7",
70      credit_card_expiration_year: "2023",
71      credit_card_cvv: "333",
72    };
73    http.post(BASE_URL + "/cart/checkout", data);
74  }
75
76  function randomise(min, max) {
77    min = Math.ceil(min);
78    max = Math.floor(max);
79    return Math.floor(Math.random() * (max - min) + min);
80  }
81
82
83
84

```

Listing .1: Script for test case 1